

Informe Final del Sistema ANE-HX

Documentación técnica, decisiones de diseño, pruebas y transferencia de conocimiento

Equipo de desarrollo

Martín Ramírez Espinosa

Mateo Almeida Gómez

David Ramírez Betancourth

Oscar Andres Gutierrez Estepa

Aldemar Giraldo Pava

2 de junio de 2026

Tabla de contenido

1. Introducción general	14
1.1. Propósito y contexto del proyecto	14
1.2. Problema o necesidad principal	14
1.3. Alcance del documento	15
1.4. Resumen de la arquitectura y los módulos principales	15
1.4.1. Capítulo 2: Análisis de requerimientos, restricciones y uso	16
1.4.2. Capítulo 4: Hardware	16
1.4.3. Capítulo 5: Software del sensor	16
1.4.4. Capítulo 6: Plataforma web en la nube	16
1.4.5. Capítulo 7: Análisis espectral y localización en la nube	17
1.4.6. Anexo A: Técnica de desarrollo con agentes IA	17
1.5. Convenciones del documento	17
2. Análisis de requerimientos, restricciones y uso	18
2.1. Propósito del capítulo	18
2.2. Contexto dentro del sistema general	18
2.3. Desarrollo técnico: requerimientos por categoría	19
2.3.1. Requerimientos de arquitectura	19
2.3.2. Requerimientos funcionales	19
2.3.3. Requerimientos de red	22
2.3.4. Requerimientos de hardware	22
2.3.5. Requerimientos de exactitud y confiabilidad	23

2.3.6. Requerimientos de interoperabilidad	23
2.4. Entradas, salidas e interfaces	23
2.5. Decisiones de diseño o implementación	24
2.6. Problemas presentados y ajustes realizados	25
2.7. Validación, pruebas o evidencias	26
2.8. Aprendizajes y transferencia de conocimiento	26
2.9. Limitaciones	27
2.10. Recomendaciones específicas	27
3. Diseño general del sistema	29
3.1. Propósito del capítulo	29
3.2. Contexto dentro del sistema general	29
3.3. Desarrollo técnico	29
3.4. Entradas, salidas e interfaces	29
3.5. Decisiones de diseño o implementación	29
3.6. Problemas presentados y ajustes realizados	30
3.7. Validación, pruebas o evidencias	30
3.8. Aprendizajes y transferencia de conocimiento	30
3.9. Limitaciones	30
3.10. Recomendaciones específicas	30
4. Hardware	31
4.1. Propósito del capítulo	31
4.2. Contexto dentro del sistema general	31
4.3. Desarrollo técnico	34
4.3.1. Trazabilidad contra requerimientos ANE	34
4.3.2. Arquitectura física del nodo	35
4.3.3. Fichas técnicas de componentes	37
4.3.4. Antenas y montaje mecánico	38
4.3.5. Ruta RF, puertos, filtros, mux y pérdidas	39

4.3.6.	GPS/GNSS y conectividad LTE	41
4.3.7.	Árbol de potencia y gestión térmica	42
4.3.8.	Compatibilidad electromagnética	44
4.4.	Entradas, salidas e interfaces	46
4.5.	Decisiones de diseño o implementación	46
4.6.	Problemas presentados y ajustes realizados	47
4.7.	Validación, pruebas o evidencias	48
4.8.	Aprendizajes y transferencia de conocimiento	50
4.9.	Limitaciones	50
4.10.	Recomendaciones específicas	51
5.	Software de adquisición SDR del sensor	52
5.1.	Propósito del capítulo	52
5.2.	Contexto dentro del sistema general	53
5.2.1.	Arquitectura funcional del software SDR	54
5.3.	Desarrollo técnico	56
5.3.1.	Adquisición	56
5.3.2.	Parámetros SDR configurables	57
5.3.3.	Span, sample rate, FFT y RBW	58
5.3.4.	Uso de buffers	59
5.3.5.	Preprocesamiento	59
5.3.6.	Procesamiento	60
5.3.7.	Postprocesamiento	63
5.3.8.	Calibración en frecuencia	64
5.3.9.	Orquestador	65
5.4.	Entradas, salidas e interfaces	66
5.4.1.	Entradas	66
5.4.2.	Salidas	67
5.4.3.	Interfaces y protocolos	68

5.4.4. Contrato local Python-C	68
5.5. Decisiones de diseño o implementación	69
5.6. Problemas presentados y ajustes realizados	70
5.7. Validación, pruebas o evidencias	71
5.8. Trazabilidad frente a requisitos funcionales	73
5.9. Aprendizajes y transferencia de conocimiento	75
5.10. Limitaciones	75
5.11. Recomendaciones específicas	76
6. Plataforma web en la nube	79
6.1. Propósito del capítulo	79
6.2. Contexto dentro del sistema general	79
6.3. Desarrollo técnico	80
6.3.1. Frontend	81
6.3.2. Backend	83
6.3.3. Base de datos	85
6.3.4. Despliegue	86
6.4. Entradas, salidas e interfaces	87
6.4.1. Entradas	87
6.4.2. Salidas	87
6.4.3. Interfaces	88
6.5. Decisiones de diseño o implementación	88
6.5.1. Separación frontend/backend con API REST	88
6.5.2. WebSocket sobre polling para tiempo real	89
6.5.3. PostgreSQL y TimescaleDB para datos de series temporales	89
6.5.4. Microservicio de postprocesamiento en Python	90
6.5.5. Separación de canales WebSocket para datos y audio	90
6.6. Problemas presentados y ajustes realizados	91
6.7. Validación, pruebas o evidencias	93

6.8. Aprendizajes y transferencia de conocimiento	94
6.9. Limitaciones	95
6.10. Recomendaciones específicas	96
6.11. Repositorios	97
6.12. Objetivo del Capítulo	100
6.13. Alcance y Exclusiones	101
6.13.1. Lo que implementa este módulo	101
6.13.2. Lo que NO implementa este módulo	101
6.14. Relación con Requisitos ANE	102
6.15. Arquitectura Funcional	102
6.16. Entradas del Módulo	104
6.17. RBW, FFT y Resolución Espectral	104
6.17.1. Relación entre parámetros	104
6.17.2. Distinción entre resolución de visualización y exactitud de medición	105
6.18. Calibración y Corrección de Ganancia	105
6.18.1. Qué corrige el archivo de corrección	105
6.18.2. Cómo se genera y periodicidad	106
6.19. Análisis Espectral	106
6.19.1. Frecuencia Central	106
6.19.2. Ancho de Banda	106
6.19.3. Potencia	107
6.19.4. Indicadores Estimados de Calidad (MER/BER)	107
6.20. Modos de Operación	108
6.21. Tolerancias de Cumplimiento contra Licencias	109
6.21.1. Valores y configuración	109
6.21.2. Criterios de cumplimiento técnico	109
6.21.3. Manejo de múltiples licencias y licencias sin vigencia	109
6.21.4. Estados semánticos de licencia	110
6.22. Formato de la Base de Licencias	110

6.23. Localización Administrativa	111
6.23.1. Qué es la localización administrativa	111
6.23.2. Origen del parámetro territorial	111
6.23.3. Jerarquía de prioridad	111
6.23.4. Limitaciones de la localización administrativa	111
6.24. Salidas, API e Integración Cloud	112
6.24.1. Endpoints de API	112
6.24.2. Campos de salida principales	112
6.25. Ejemplos de Entrada y Salida JSON	113
6.25.1. Análisis automático (all_emissions)	113
6.25.2. Análisis por picos	113
6.25.3. Cumplimiento con DANE único	113
6.25.4. Sin emisión detectada	114
6.25.5. Sin DANE — cumplimiento bloqueado	114
6.26. Manejo de Errores y Estados de Excepción	114
6.27. Almacenamiento, Trazabilidad y Reportes	115
6.27.1. Campos de trazabilidad forense	115
6.27.2. Campos que alimentan los reportes de administración de espectro	116
6.28. Seguridad de la Información	117
6.29. Pruebas y Validación	118
6.29.1. Matriz de validación con señales de prueba	118
6.29.2. Métricas de desempeño y metas	119
6.30. Limitaciones del Módulo	119
6.31. Recomendaciones	120
6.32. Criterios de Aceptación Verificables	120
6.33. Tabla de Pendientes para Completar el Capítulo	121
6.34. Conclusión Técnica	121
7. Protocolos de prueba edge-cloud	122

7.1.	Objetivo del capítulo	122
7.2.	Alcance de las pruebas	122
7.3.	Arquitectura bajo prueba	123
7.4.	Criterios generales de aceptación	124
7.5.	Estrategia general de pruebas	125
7.5.1.	Pruebas funcionales	125
7.5.2.	Pruebas de integración edge	125
7.5.3.	Pruebas de integración edge-cloud	125
7.5.4.	Pruebas metrológicas y espectrales	125
7.5.5.	Pruebas pedagógicas	126
7.6.	Matriz general de pruebas	126
7.7.	Protocolos detallados de prueba	129
7.7.1.	PR-EC-001: registro e identificación del sensor	129
7.7.2.	PR-EC-002: heartbeat y salud del sensor	130
7.7.3.	PR-EC-003: configuración de adquisición SDR	131
7.7.4.	PR-EC-004: validación de RBW	132
7.7.5.	PR-EC-005: adquisición IQ y generación de PSD	132
7.7.6.	PR-EC-006: detección de emisiones por umbral	133
7.7.7.	PR-EC-007: medición de ancho de banda ocupado	133
7.7.8.	PR-EC-008: programación y ejecución de campañas	134
7.7.9.	PR-EC-009: recuperación ante pérdida de conectividad	134
7.7.10.	PR-EC-010: exportación de datos	135
7.8.	Pruebas metrológicas recomendadas	136
7.8.1.	DANL	136
7.8.2.	Exactitud de frecuencia	136
7.8.3.	Precisión de amplitud	137
7.8.4.	Rango dinámico	137
7.9.	Pruebas de instalación y operación de campo	137
7.10.	Problemas, decisiones y ajustes esperados	138

7.11. Evidencias mínimas por prueba	139
7.12. Transferencia de conocimiento	140
7.13. Recomendaciones para futuras pruebas	141
7.14. Conclusiones del capítulo	142
8. Integración general del sistema	143
8.1. Propósito del capítulo	143
8.2. Contexto dentro del sistema general	143
8.3. Desarrollo técnico	143
8.4. Entradas, salidas e interfaces	143
8.5. Decisiones de diseño o implementación	143
8.6. Problemas presentados y ajustes realizados	144
8.7. Validación, pruebas o evidencias	144
8.8. Aprendizajes y transferencia de conocimiento	144
8.9. Limitaciones	144
8.10. Recomendaciones específicas	144
9. Conclusiones generales	145
9.1. Síntesis de resultados técnicos	145
9.2. Logros del diseño e implementación	145
9.3. Resultados de la integración	145
9.4. Lecciones aprendidas	145
9.5. Cumplimiento de objetivos	145
10.Recomendaciones	146
10.1. Mejoras de arquitectura	146
10.2. Mejoras de hardware	146
10.3. Mejoras de software	146
10.4. Mejoras de documentación	146
10.5. Reducción de riesgos identificados	146

11. Referencias	147
A. Material de apoyo: uso de agentes de IA para documentación, desarrollo y revisión técnica del proyecto	148
A.1. Nota de alcance y propósito	148
A.2. Contexto y diferenciación funcional	148
A.3. Metodología de trabajo: Ecosistema de Agentes	149
A.3.1. Habilitación del Entorno (Ejemplos CLI)	149
A.3.2. Arquitectura Multi-Agente Asistida	149
A.4. Gobernanza y Uso Responsable de IA	151
A.5. Trazabilidad Metodológica	151
A.6. Flujo de Trabajo Colaborativo (Paso a Paso)	151
A.7. Trazabilidad de Prompts y Relación con GitHub	152
A.8. Problemas, Limitaciones y Ajustes	152
A.9. Métricas de Utilidad y Evidencias	153
A.10. Conclusión del capítulo	153
B. Diagramas adicionales	155
B.1. Diagramas de flujo de la plataforma	155
B.1.1. Frontend	155
B.1.2. Backend	156
B.1.3. Postprocesamiento	157
B.2. Diagramas de despliegue y conectividad	158
B.3. Secuencias operativas	158
B.4. Esquemas detallados de componentes	158
C. Manuales y documentos de apoyo	159
C.1. Propósito del capítulo	159
C.2. Carpeta documental externa	159
C.3. Inventario de documentos disponibles	160

C.4. Relación de los documentos con los capítulos del informe	162
C.5. Clasificación funcional de la documentación	163
C.6. Requisitos previos de consulta e instalación	164
C.7. Instrucciones de operación cotidiana	165
C.8. Solución de problemas frecuentes	166
C.9. Recomendaciones para mantenimiento documental	167
C.10. Control de trazabilidad documental	168
C.11. Nota sobre enlaces individuales	169
C.12. Observación final	169
D. Resultados de pruebas	170
D.1. Tablas de métricas y mediciones	170
D.2. Evidencias visuales de ejecución	170
D.3. Comparación entre escenarios de prueba	170
D.4. Observaciones y anomalías detectadas	170

Lista de figuras

4.1.	Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia.	33
4.2.	Diagrama de bloques de hardware del nodo de sensado. Fuente: elaboración propia.	34
4.3.	Módulos principales del hardware ANE2. Fuente: elaboración propia.	37
4.4.	Puntos críticos de ensamble de la antena TDT. Fuente: elaboración propia.	38
4.5.	Puntos críticos de ensamble de la antena VHF/UHF. Fuente: elaboración propia.	39
4.6.	Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en Mon-RaF2. Fuente: elaboración propia.	40
5.1.	Diagrama de bloques de la arquitectura funcional del software SDR del sensor.	55
6.1.	Diagrama general de la arquitectura de la plataforma web. Fuente: Elaboración propia.	81
A.1.	Diseño conceptual del sistema multi-agente: Relación metodológica entre roles de asistencia. Fuente: Elaboración propia.	150
A.2.	Flujo de trabajo y validación humana en la integración de resultados. Fuente: Elaboración propia.	153
B.1.	Diagrama de arquitectura y flujo del frontend de la plataforma web. Fuente: Elaboración propia.	156
B.2.	Diagrama de arquitectura y flujo del backend de la plataforma web. Fuente: Elaboración propia.	157
B.3.	Diagrama de arquitectura y flujo del servicio de postprocesamiento. Fuente: Elaboración propia.	158

Lista de tablas

2.1. Entradas y salidas principales del sistema ANE-HX.	24
2.2. Problemas encontrados en la implementación de requerimientos.	25
4.1. Trazabilidad de requisitos ANE contra capacidades de hardware.	34
4.2. Inventario funcional de componentes de hardware del nodo ANE2.	36
4.3. Fichas técnicas resumidas de los elementos de hardware.	37
4.4. Presupuesto cualitativo de ruta RF y puntos de pérdida.	41
4.5. Presupuesto de potencia y térmico para operación del nodo.	43
4.6. Criterios mínimos para una versión robusta del árbol de potencia.	44
4.7. Medidas de mitigación EMC y pruebas mínimas.	45
4.8. Interfaces principales del hardware ANE2.	46
4.9. Decisiones de diseño de hardware y justificación.	46
4.10. Problemas presentados, decisiones y ajustes realizados en hardware.	47
4.11. Resumen de telemetría eléctrica y térmica inspeccionada.	48
4.12. Protocolo de validación de hardware.	49
6.1. Problemas presentados, decisiones y ajustes realizados en la plataforma web.	91
6.2. Repositorio asociado al frontend de la plataforma web.	98
6.3. Repositorio asociado al backend de la plataforma web.	98
7.1. Matriz general de pruebas edge-cloud.	126
7.2. Problemas frecuentes durante las pruebas edge-cloud.	139
7.3. Aprendizajes transferibles asociados a las pruebas.	141

A.1. Trazabilidad de actividades de apoyo con IA.	151
A.2. Gestión de problemas y limitaciones de la IA.	152
C.1. Manuales, informes, videos y documentos de apoyo disponibles en Google Drive	160
C.2. Problemas frecuentes en la consulta de documentos externos	166

Capítulo 1

Introducción general

1.1. Propósito y contexto del proyecto

Este documento constituye el informe final del sistema de monitoreo espectral **ANE-HX**, desarrollado como proyecto académico de ingeniería para la Agencia Nacional del Espectro (ANE) de Colombia. El sistema integra componentes de hardware, software de adquisición por radio definida por software (SDR), procesamiento en borde, plataforma web en la nube y algoritmos de análisis espectral para construir una solución funcional de monitoreo del espectro radioeléctrico.

El informe se organiza como memoria técnica del sistema, registro de decisiones de diseño, evidencia de pruebas realizadas e instrumento de transferencia de conocimiento hacia futuros equipos que deban mantener, reproducir o extender el desarrollo.

1.2. Problema o necesidad principal

El monitoreo del espectro radioeléctrico requiere capturar señales en campo, transportarlas a un entorno de análisis, extraer parámetros técnicos de las emisiones detectadas y contrastarlos contra información regulatoria. Un sistema que resuelva este flujo de extremo a extremo debe coordinar hardware de adquisición, conectividad celular, almacenamiento en la nube, algoritmos de procesamiento espectral y una interfaz para operadores.

El proyecto ANE-HX aborda esta cadena completa mediante una arquitectura modular que separa el sensado en borde, la transmisión de datos, la persistencia y visualización en la nube, y el postprocesamiento analítico. Cada módulo se documenta en un capítulo independiente.

1.3. Alcance del documento

El presente informe cubre el diseño, la implementación, las pruebas y los aprendizajes obtenidos durante el desarrollo del sistema. No se incluye el código fuente completo de los módulos de software, el cual reside en repositorios independientes referenciados en cada capítulo.

Los capítulos que contienen contenido técnico desarrollado son:

- Capítulo 2: Análisis de requerimientos, restricciones y uso.
- Capítulo 4: Hardware del nodo de sensado espectral.
- Capítulo 5: Software del sensor (adquisición SDR y preprocesamiento en borde).
- Capítulo 6: Plataforma web en la nube (frontend, backend, base de datos y despliegue).
- Capítulo 7: Análisis espectral y localización en la nube (postprocesamiento de capturas).
- Anexo A: Técnica de desarrollo colaborativo usando agentes de inteligencia artificial.

Los capítulos restantes (3, 8, 9, 10, 11, 12 y anexos B, C, D) contienen la estructura prevista como placeholders para completar la documentación de diseño general, pruebas, integración, conclusiones, recomendaciones, referencias y material complementario.

1.4. Resumen de la arquitectura y los módulos principales

El sistema ANE-HX se organiza en tres dominios:

1. **Borde:** un nodo físico compuesto por una Raspberry Pi 5, un SDR HackRF One, una tarjeta auxiliar MonRaF2 (concentrador RF, LTE, GNSS y multiplexor de antenas) y antenas RF. El software embebido controla la adquisición, ejecuta preprocesamiento y transmite productos espectrales hacia la nube mediante conectividad celular.
2. **Nube:** una plataforma web con frontend SPA (React), backend de orquestación (Node.js), base de datos PostgreSQL con extensión TimescaleDB para series temporales, y un microservicio de postprocesamiento espectral en Python.
3. **Usuarios:** personas operadoras y administradoras que acceden a la plataforma web para visualizar métricas en tiempo real, gestionar campañas de medición, consultar reportes de cumplimiento normativo y administrar sensores.

1.4.1. Capítulo 2: Análisis de requerimientos, restricciones y uso

Consolida los requerimientos funcionales y no funcionales pactados en el pliego técnico del proyecto. Organiza las especificaciones por categoría —parametrización, medición, cumplimiento normativo, gestión, análisis forense, red, hardware, exactitud e interoperabilidad— y las contrasta con el estado de implementación documentado en los capítulos técnicos. Incluye una matriz de problemas encontrados durante la implementación de los requerimientos y sus ajustes correspondientes.

1.4.2. Capítulo 4: Hardware

Documenta el diseño físico del nodo ANE2. Describe la arquitectura de potencia, la ruta RF desde las antenas hasta el SDR, el árbol de alimentación, la conectividad GPIO entre la Raspberry Pi 5 y la MonRaF2, y las mediciones de voltaje y temperatura bajo carga. Incluye decisiones de diseño como la separación de la fuente de potencia del cómputo frente a los periféricos SDR y LTE, y una matriz de problemas encontrados con sus respectivos ajustes.

1.4.3. Capítulo 5: Software del sensor

Describe la arquitectura del código que se ejecuta en el nodo de borde. Cubre el control de la HackRF One, la selección de antena, la adquisición de muestras IQ, el preprocesamiento (balance IQ, corrección DC, estimación de PSD, calibración en frecuencia), la demodulación de FM y AM, y la comunicación con el backend. El software opera mediante un orquestador que gestiona modos de operación como campañas programadas, PSD en tiempo real y calibración.

1.4.4. Capítulo 6: Plataforma web en la nube

Documenta la capa central de coordinación e interfaz de usuario. El frontend, construido con React y Vite, ofrece autenticación, tableros de monitoreo con gráficas en tiempo real vía WebSocket, gestión de sensores y generación de reportes de cumplimiento. El backend, basado en Node.js y Express, expone una API REST, maneja WebSockets para telemetría y audio, y orquesta las campañas de medición. La base de datos TimescaleDB almacena series temporales de métricas espectrales y coordenadas GPS.

1.4.5. Capítulo 7: Análisis espectral y localización en la nube

Describe el módulo de postprocesamiento encargado de convertir capturas espectrales crudas en parámetros técnicos interpretables: frecuencia central, ancho de banda, potencia, MER y BER para señales TDT. El módulo implementa tres modos de operación (detección automática, análisis por picos y cumplimiento contra licencias), utiliza un algoritmo de linealización con umbral fijo optimizado para la Raspberry Pi 5, y soporta filtrado geográfico mediante códigos DANE y nombre de municipio para la validación normativa contra bases de licencias oficiales.

1.4.6. Anexo A: Técnica de desarrollo con agentes IA

Documenta la metodología de trabajo colaborativo del equipo utilizando agentes de inteligencia artificial (Gemini CLI y GitHub Copilot CLI). Describe la instalación, la arquitectura multi-agente basada en roles, la metodología de prompting estructurada (ROLE → OBJECTIVE → CONTEXT → REASONING), los problemas encontrados durante la colaboración con IA y las lecciones aprendidas.

1.5. Convenciones del documento

El informe está redactado en español, compilado con $\text{\LaTeX}$ y utiliza codificación UTF-8. Cada capítulo sigue una estructura homogénea: propósito, contexto en el sistema, desarrollo técnico, entradas y salidas, decisiones de diseño, problemas y ajustes, validación, aprendizajes, limitaciones y recomendaciones. Las referencias a repositorios de software incluyen el enlace real o la indicación explícita de que el repositorio está pendiente de publicación. No se incluye código fuente extenso dentro del documento.

Capítulo 2

Análisis de requerimientos, restricciones y uso

2.1. Propósito del capítulo

Este capítulo consolida los requerimientos funcionales y no funcionales pactados para el sistema ANE-HX, desarrollado para la Agencia Nacional del Espectro (ANE) de Colombia. El objetivo es establecer un marco de referencia verificable que permita evaluar el grado de cumplimiento alcanzado en cada módulo del sistema y servir como guía para futuras iteraciones de desarrollo.

Los requerimientos aquí documentados provienen del pliego técnico oficial del proyecto. Se presentan agrupados por categoría funcional y se contrastan con el estado de implementación documentado en los capítulos técnicos de hardware (Capítulo 4), software del sensor (Capítulo 5), plataforma web (Capítulo 6) y análisis espectral (Capítulo 7).

2.2. Contexto dentro del sistema general

El sistema ANE-HX responde a la necesidad institucional de la ANE de monitorear el espectro radioeléctrico colombiano de forma automatizada, remota y trazable. La arquitectura del sistema se organiza en tres dominios que cubren el flujo completo desde la captura de señales en campo hasta la generación de reportes de cumplimiento:

1. **Borde:** nodos de sensado desplegados en campo que adquieren señales de RF, ejecutan preprocesamiento local y transmiten productos espectrales hacia la nube mediante conectividad celular, WiFi o Ethernet.
2. **Nube:** plataforma central que recibe, almacena, procesa y visualiza los datos espec-

trales. Incluye el backend de orquestación, la base de datos de series temporales, el frontend de usuario y el módulo de postprocesamiento espectral.

3. **Usuarios:** personal técnico de la ANE que opera el sistema mediante la plataforma web, configura campañas de medición, consulta resultados y genera reportes de cumplimiento normativo.

Los requerimientos se distribuyen a lo largo de esta cadena: la parametrización de adquisición y las capacidades de medición recaen en el software del sensor (borde); la visualización, la gestión de campañas y la generación de reportes residen en la plataforma web (nube); la detección de emisiones, el cumplimiento normativo y el análisis forense se implementan en el módulo de postprocesamiento (nube).

2.3. Desarrollo técnico: requerimientos por categoría

A continuación se presentan los requerimientos finales del sistema, organizados según las categorías definidas en el pliego técnico.

2.3.1. Requerimientos de arquitectura

Categoría	Requerimiento
Seguridad de la información	Toda la información en tránsito entre el sensor y la interfaz de monitoreo debe usar SSL/TLS.

2.3.2. Requerimientos funcionales

Parametrización de adquisición

Parámetro	Especificación	Restricción
Frecuencia central	Desde 1 MHz hasta 6 GHz	Límite del hardware SDR
Sample Rate	Hasta 20 MS/s, mínimo 200 kS/s	Seleccionable por el usuario
Ancho de banda instantáneo	Máximo 20 MHz, mínimo 2 MHz	Determinado por el sample rate
Redundancia GPS	Ubicación manual adicional	Conserva la reportada por el GPS

Parametrización de visualización

Parámetro	Especificación
RBW	Ajustable mediante selección de puntos FFT. $RBW = SampleRate/FFTSize$
VBW	Suavizado del espectro en niveles: bajo, medio, alto
Filtros por software	Pasabajo, pasabanda, pasaalto con frecuencia central, ancho de banda y activación/desactivación
Resolución	Pasos de 500 Hz, resolución de hasta 10 Hz
Unidades de medida	Frecuencia: Hz, kHz, MHz, GHz. Potencia: dBm (prioritario), dBmV, dBuV, V, W. Campo eléctrico: dBuV/m (prioritario), dBmV/m, mV/m, W/m ² , dBW/m ² /MHz
Umbral de detección	Configurable a partir de 3 dB, 5 dB u otro valor definido por el usuario

Medición, visualización y audio

Función	Especificación
Indicadores espectrales	Traza máxima, traza mínima, promedio, RMS, máximo y mínimo
Marcadores	Al menos 4 marcadores individuales y tipo delta, con cálculo de delta de potencia y delta de frecuencia
Ancho de banda	Método $-X$ dB (X seleccionable: 3, 10 o 26 dB) y OBW (99 % de potencia)
Potencia de canal	Para emisiones de hasta 20 MHz de ancho de banda
Demodulación FM/AM	Demodular y escuchar emisiones FM y AM. Medir profundidad de modulación AM y excursión de frecuencia FM

Cumplimiento normativo

Verificación	Especificación
Frecuencias sin uso	Detectar ausencia de emisiones autorizadas en bandas con licencia asignada
Emisiones no autorizadas	Detectar emisiones no registradas en la base de datos de operadores autorizados del MinTIC
Incumplimiento de ancho de banda	Detectar señales que excedan los límites de ancho de banda autorizados por servicio
Radiodifusión sonora	Recolección de datos para identificación de intermodulaciones en la banda 88–108 MHz
Servicio móvil aeronáutico	Detección de emisiones desviadas en la banda 108–137 MHz

Herramientas de gestión

Función	Especificación
Monitoreo de sensores	Alertas sobre violación de umbrales de variables del sensor: disco, RAM, CPU
Monitoreo de conectividad	Calidad de conexión IP mediante RTT y estado del canal (alive/not alive)
Agenda de programaciones	Publicar agenda de escaneos, mantener sesiones privadas y permitir cancelación por rol supervisor
Generación de reportes	Al menos 1 reporte con variables definidas por la entidad, entregado en CSV

Análisis forense y almacenamiento

Función	Especificación
Referenciación de información	Timestamp + geotags + identificador del sensor para cada medición
Almacenamiento en sensor	Archivos de muestreo (.cs8) en el almacenamiento local disponible
Almacenamiento centralizado	Resultados de monitoreo programado en servidores institucionales de la ANE

Funciones avanzadas y soporte

Función	Especificación
Datasets para análisis	Recolección y organización de datos para procesos de análisis posteriores y entrenamiento de modelos
Acceso remoto	Acceso vía IP a sensores para actualización de software y aplicación de parches

2.3.3. Requerimientos de red

Requerimiento	Especificación
WiFi	Habilitar conexión inalámbrica
Ethernet	Puerto de red cableado conservando características ambientales del nodo
Enrutamiento automático	Seleccionar automáticamente el canal de comunicación disponible (Ethernet, WiFi o celular)

2.3.4. Requerimientos de hardware

Requerimiento	Especificación
GPS	Conexión móvil mediante SIM card institucional. No depender de tecnologías propietarias. Precisión horizontal ≤ 10 m, vertical ≤ 15 m (estándar L1 GNSS civil)
Multiplexación de antenas	Board que permita usar 4 antenas distintas en procesos de adquisición espectral
Kit de antenas	Tres antenas por sensor: (1) TDT, (2) telefonía móvil Down link ($f > 2$ GHz), (3) VHF/UHF y frecuencias intermedias
EMC	Optimizar susceptibilidad electromagnética para evitar interferencias que falseen mediciones

2.3.5. Requerimientos de exactitud y confiabilidad

Requerimiento	Especificación
Sincronización NTP	Sincronización a reloj de red NTP en cada sensor para etiquetado temporal de muestreos
Calibración	Procedimientos de calibración de frecuencia, ganancia de recepción, balance IQ y corrección de offset, verificados con equipos certificados
DANL	Determinar el nivel más bajo de señal detectable sin señal presente
Rango dinámico	Determinar la diferencia entre la señal máxima sin distorsión y la mínima detectable sobre ruido
Exactitud de frecuencia	Determinar la diferencia máxima aceptable entre frecuencia real y medida
Precisión de amplitud	Determinar el margen de error en la medición de potencia, expresado en dB

2.3.6. Requerimientos de interoperabilidad

Requerimiento	Especificación
Exportación de datos	Escaneos descargables en formato CSV, JSON y XML para procesamiento offline

2.4. Entradas, salidas e interfaces

El sistema ANE-HX opera como una plataforma integral que recibe señales de RF en los nodos de borde y entrega información procesada a los operadores mediante la interfaz web. La Tabla 2.1 resume las entradas y salidas principales.

Tabla 2.1: Entradas y salidas principales del sistema ANE-HX.

Nivel	Entradas	Salidas
Borde	Señales RF (1 MHz – 6 GHz), parámetros de campaña, comandos de control remoto	Muestras IQ, PSD, métricas espectrales, audio demodulado, telemetría del sensor, coordenadas GPS
Nube	Datos espectrales desde sensores, parámetros de postprocesamiento, bases de licencias del MinTIC	Visualizaciones en tiempo real, reportes de cumplimiento (CSV), datos exportables (CSV/JSON/XML), alertas
Usuario	Credenciales de acceso, parámetros de configuración, solicitudes de reporte	Interfaz web con tableros, gráficas, reportes descargables, agenda de campañas

2.5. Decisiones de diseño o implementación

La definición de los requerimientos condicionó las siguientes decisiones de diseño del sistema:

- **Arquitectura borde-nube:** la exigencia de monitoreo remoto con acceso vía IP y transmisión celular determinó la separación entre adquisición local (Raspberry Pi 5 + HackRF One) y procesamiento centralizado en servidores institucionales de la ANE.
- **Modularidad del software:** los requerimientos de parametrización, medición, cumplimiento normativo y gestión se implementaron como módulos independientes (sensor SDR, plataforma web, postprocesamiento espectral) que se comunican mediante API REST y WebSocket.
- **Selección del SDR:** el HackRF One se eligió por su rango de operación (1 MHz – 6 GHz) y ancho de banda instantáneo (hasta 20 MHz), que cubren exactamente las bandas requeridas. Su naturaleza de código abierto evita dependencias de tecnologías propietarias.
- **Base de datos temporal:** la necesidad de almacenar y consultar series temporales de métricas espectrales con timestamp y geolocalización motivó la adopción de PostgreSQL con extensión TimescaleDB.
- **Postprocesamiento en Python:** los algoritmos de detección de emisiones, estimación de parámetros y comparación contra licencias se implementaron en Python por su ecosistema de librerías científicas (NumPy, SciPy) y su facilidad de integración como microservicio.

- **Umbral fijo con linealización:** las restricciones de CPU de la Raspberry Pi 5 llevaron a descartar algoritmos adaptativos costosos en favor de un umbral fijo precedido por una corrección de linealidad del sistema, como se detalla en el Capítulo 7.

2.6. Problemas presentados y ajustes realizados

Tabla 2.2: Problemas encontrados en la implementación de requerimientos.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión del nodo bajo carga SDR/LTE	Caídas de voltaje en EXT5V con HackRF y módem LTE activos simultáneamente	La Raspberry Pi 5 no puede suministrar corriente suficiente a todos los periféricos por sus puertos	Diseñar árbol de potencia externo sobredimensionado con fuente dedicada	Operación estable del nodo con todos los periféricos activos
Falsos positivos en detección de emisiones	Umbral fijo producía detecciones espurias por curvatura del ruido del receptor	No linealidad del piso de ruido del hardware SDR	Implementar corrección de linealidad antes de aplicar umbral fijo	Reducción drástica de falsas alarmas con bajo costo computacional
Timeouts en generación de reportes	Errores 504 en frontend con grandes volúmenes de datos	Procesamiento y armado de reportes excedía el timeout del servidor	Extender timeouts a 3600 s en Node.js y Nginx	Reportes generados exitosamente incluso para campañas extensas
Saturación del canal de subida	Resolución y tasa de refresco limitadas en transmisiones desde borde	Ancho de banda del canal celular insuficiente para tramas de alta resolución	Ajustar resolución espectral según disponibilidad de canal	Compromiso aceptable entre resolución y capacidad de transmisión

2.7. Validación, pruebas o evidencias

La validación de los requerimientos se realizó de forma incremental a medida que cada módulo alcanzaba un estado funcional:

- **Parametrización de adquisición:** verificada mediante campañas de medición configuradas desde la plataforma web con distintas frecuencias centrales, sample rates y ganancias. Los resultados se validaron por inspección espectral en la interfaz de monitoreo en tiempo real.
- **Medición de ancho de banda y potencia:** contrastada contra señales de referencia conocidas (portadoras de FM comercial en la banda 88–108 MHz) cuyos parámetros nominales están documentados en las bases de licencias del MinTIC.
- **Cumplimiento normativo:** validado mediante el procesamiento de capturas reales contra un archivo CSV de licencias de prueba, verificando la correcta asociación por frecuencia, el filtrado por municipio y código DANE, y la emisión de resultados de cumplimiento.
- **Exportación de datos:** verificada la generación de archivos CSV con la estructura de columnas solicitada por la entidad.
- **Conectividad y monitoreo:** el sistema se desplegó en un servidor institucional de la ANE, validando la comunicación extremo a extremo entre el sensor en campo y la plataforma web, incluyendo el enrutamiento automático entre WiFi y Ethernet.

Las evidencias detalladas de cada prueba se documentan en el Capítulo 7 (pruebas del módulo de postprocesamiento) y en las secciones de validación de los capítulos 4, 5 y 6.

2.8. Aprendizajes y transferencia de conocimiento

- **Anticipar restricciones de potencia desde el inicio:** el cuello de botella más recurrente en el hardware fue la alimentación. La lección aprendida es que el árbol de potencia debe diseñarse antes de integrar los periféricos, no como una corrección posterior.
- **El cumplimiento normativo requiere datos administrativos limpios:** la variabilidad en nombres de municipios, códigos DANE y formatos de licencias obligó a implementar normalización de texto y reglas de compatibilidad. Esto debe considerarse desde la fase de diseño de datos.

- **El ancho de banda del canal celular condiciona la resolución espectral:** la capacidad de transmisión desde el borde es el factor limitante para la resolución de las capturas. Futuros despliegues deben dimensionar el canal de subida según la densidad espectral requerida.
- **Modularidad facilita la validación incremental:** la separación en módulos independientes (sensor, plataforma, postprocesamiento) permitió probar y validar cada requerimiento en el contexto del módulo que lo implementa, sin depender del sistema completo.

2.9. Limitaciones

- **MER/BER solo para TDT:** la estimación de calidad de señal (MER/BER) está implementada únicamente para señales de Televisión Digital Terrestre con modulación 64-QAM. No cubre otros servicios de radiodifusión o comunicaciones.
- **Detección de intermodulaciones:** la recolección de datos para identificación de intermodulaciones en la banda de radiodifusión sonora está iniciada pero no se ha completado el análisis automático.
- **Caracterización metrológica:** la determinación formal de DANL, rango dinámico y exactitud de amplitud requiere equipos de calibración certificados no disponibles durante el desarrollo. Las mediciones actuales son operativas, no metrológicas.
- **Almacenamiento de muestras crudas:** el almacenamiento de archivos .cs8 en el sensor está limitado por la capacidad del almacenamiento local de la Raspberry Pi 5 (tarjeta SD).
- **Acceso remoto:** el acceso remoto a sensores vía IP está implementado a nivel de sistema operativo (SSH) pero no integrado en la plataforma web como funcionalidad de usuario.

2.10. Recomendaciones específicas

- **Completar la caracterización metrológica:** realizar una campaña de calibración con equipos certificados para determinar formalmente DANL, rango dinámico, exactitud de frecuencia y precisión de amplitud del sistema.
- **Extender MER/BER a otros servicios:** ampliar la estimación de calidad de señal a modulaciones de radiodifusión sonora (FM) y servicios móviles, utilizando métricas como SINAD o profundidad de modulación.

- **Automatizar detección de intermodulaciones:** desarrollar el módulo de análisis automático de intermodulaciones a partir de los datasets ya recolectados en las bandas 88–108 MHz y 108–137 MHz.
- **Integrar acceso remoto en la plataforma:** incorporar en la interfaz web una funcionalidad de acceso remoto a sensores que permita actualizaciones de software y diagnóstico sin requerir acceso por SSH.
- **Ampliar almacenamiento local:** evaluar la migración a almacenamiento externo (USB o NAS) para aumentar la capacidad de retención de muestras crudas (.cs8) en el nodo de borde.
- **Implementar reportes adicionales:** desarrollar los formatos de reporte complementarios (JSON, XML) según la especificación de interoperabilidad, más allá del CSV actualmente implementado.

Capítulo 3

Diseño general del sistema

3.1. Propósito del capítulo

[[Placeholder]]

3.2. Contexto dentro del sistema general

[[Placeholder]]

3.3. Desarrollo técnico

[[Placeholder]]

3.4. Entradas, salidas e interfaces

[[Placeholder]]

3.5. Decisiones de diseño o implementación

[[Placeholder]]

3.6. Problemas presentados y ajustes realizados

[[Placeholder]]

3.7. Validación, pruebas o evidencias

[[Placeholder]]

3.8. Aprendizajes y transferencia de conocimiento

[[Placeholder]]

3.9. Limitaciones

[[Placeholder]]

3.10. Recomendaciones específicas

[[Placeholder]]

Capítulo 4

Hardware

4.1. Propósito del capítulo

Esta sección documenta el hardware del nodo de sensado espectral ANE2 con un criterio de transferencia de conocimiento. La intención no es solamente listar piezas, sino explicar cómo se organiza el montaje físico, qué función cumple cada módulo, cuáles interfaces deben respetarse y qué decisiones deben mantenerse en una nueva versión del producto.

La idea técnica central es la alimentación. La experiencia de integración mostró que concentrar la entrega de energía de los periféricos en la Raspberry Pi 5 reduce el margen operativo del nodo. Un diseño derivado o una versión posterior debe incluir un **árbol de potencia externo, sobredimensionado y medible**, capaz de alimentar cómputo, SDR, LTE/GNSS, mux RF y futuras extensiones sin depender del margen de los puertos de la Raspberry Pi.

4.2. Contexto dentro del sistema general

ANE2 es un nodo de borde para monitoreo espectral. En campo, el hardware debe recibir señales RF, seleccionar o enrutar la entrada de antena, digitalizar la señal mediante SDR, ejecutar procesamiento local, obtener ubicación y mantener conectividad con la plataforma. Por tanto, el hardware cumple cuatro responsabilidades:

- **Entrada y selección RF:** las antenas llegan a la tarjeta MonRaF2, donde se concentran rutas RF, filtros y una etapa tipo mux/switch controlada por GPIO.
- **Adquisición SDR:** el HackRF One opera como receptor y entrega muestras IQ al computador de borde.

- **Cómputo de borde:** la Raspberry Pi 5 ejecuta control, telemetría, adquisición y comunicación con servicios superiores.
- **Conectividad y ubicación:** el módulo SIM7600G-H aporta LTE y GNSS/GPS para comunicación y georreferenciación.

La Figura 4.1 presenta el conjunto físico del nodo. La lectura correcta del montaje es: antenas hacia MonRaF2, selección o acondicionamiento RF en MonRaF2, adquisición con HackRF One, procesamiento en Raspberry Pi 5 y comunicación por LTE/GNSS.

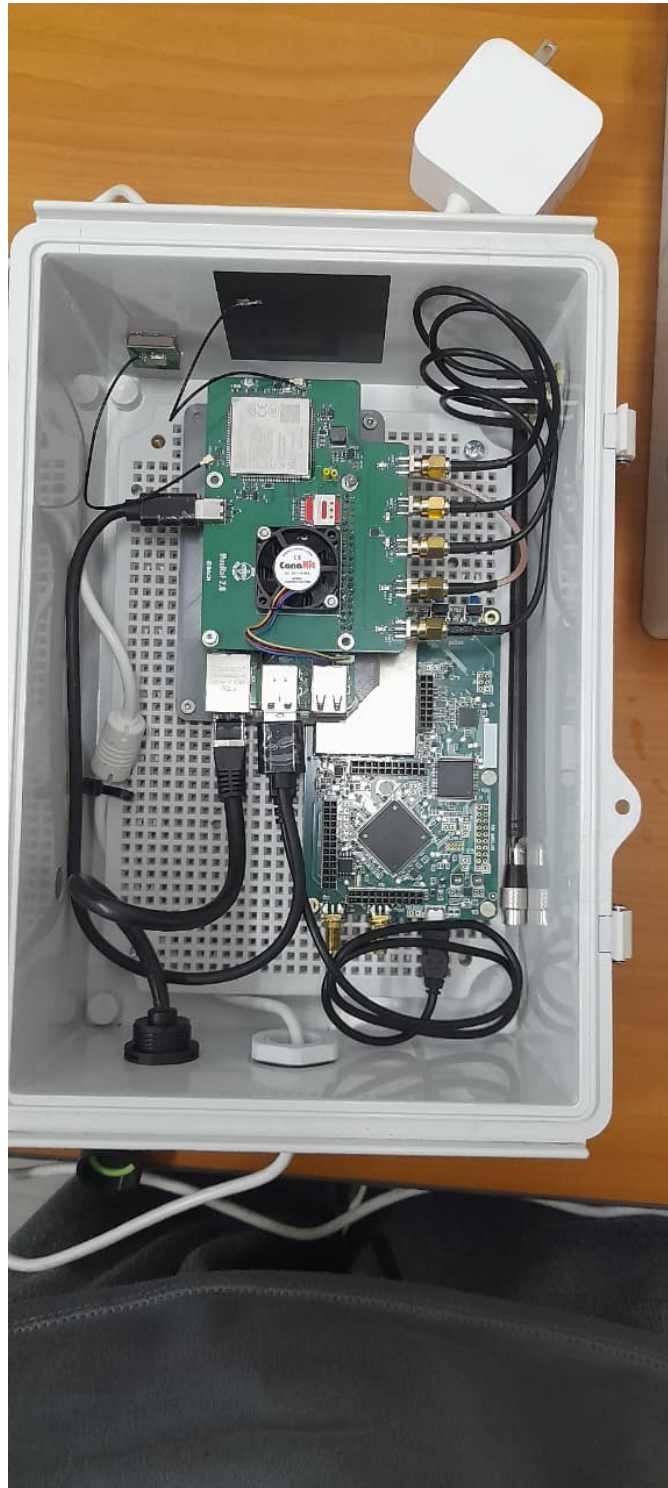


Figura 4.1: Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia.

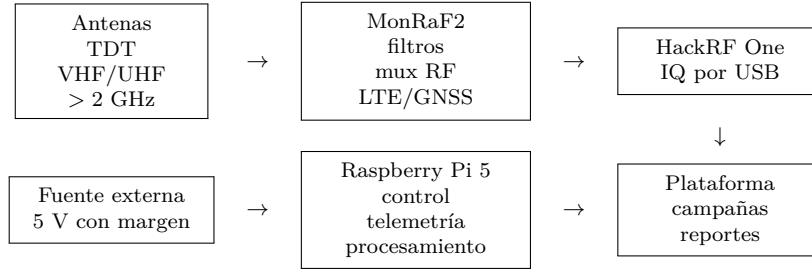


Figura 4.2: Diagrama de bloques de hardware del nodo de sensado. Fuente: elaboración propia.

4.3. Desarrollo técnico

4.3.1. Trazabilidad contra requerimientos ANE

La Tabla 4.1 relaciona los requerimientos finales del sistema que dependen del hardware con los elementos físicos que los soportan. La columna de estado usa cuatro marcas: implementado, parcial, pendiente y recomendado. Esta clasificación separa lo existente en el prototipo de lo que debe endurecerse para una versión de campo.

Tabla 4.1: Trazabilidad de requisitos ANE contra capacidades de hardware.

ID	Requerimiento final asociado	Implementación o evidencia técnica en el nodo	Estado
3.1.1	Integrar conexión móvil mediante SIM institucional para transmisión de datos, sin depender de tecnologías propietarias, y mejorar la precisión horizontal y vertical con GNSS civil L1.	SIM7600G-H aporta LTE Cat-4, ranura nano SIM y GNSS multiconstelación. La ubicación queda disponible para georreferenciación y el enlace celular actúa como canal de datos cuando no hay red fija.	Parcial
3.2.1	Agregar una tarjeta que permita usar cuatro antenas distintas en procesos de adquisición espectral.	MonRaF2 concentra puertos SMA, rutas MAIN/GNSS/AUX, filtros, conmutación RF y control GPIO desde la Raspberry Pi. La ruta documentada cubre TDT, VHF/UHF y > 2 GHz; la cuarta posición queda reservada para expansión.	Parcial
3.2.2	Agregar a cada sensor un kit de tres antenas específicas para recepción: TDT, $f > 2$ GHz para telefonía móvil y VHF/UHF con frecuencias intermedias.	El montaje físico separa antenas por banda objetivo, conserva conectores de $50\ \Omega$ y registra la antena seleccionada durante la campaña de medición.	Implementado
3.3.1	Optimizar la compatibilidad electromagnética para reducir interferencias que puedan falsear las mediciones espectrales del SDR.	El diseño contempla gabinete, separación de rutas RF y digitales, sellado, protección ESD, filtrado y selección de sitio. Falta campaña formal de susceptibilidad e inmunidad.	Parcial

ID	Requerimiento final asociado	Implementación o evidencia técnica en el nodo	Estado
4.1.1	Sincronizar cada sensor con reloj de red NTP para etiquetado temporal de muestreos.	NTP depende del canal IP disponible y GNSS aporta posición y referencia de sitio. La verificación de sello temporal debe acompañar las campañas de captura.	Parcial
4.1.2	Incluir calibración de frecuencia, ganancia de recepción, balance IQ y corrección de offset, con verificación mediante equipos certificados.	Existen mediciones operativas y análisis de balance IQ. La calibración trazable con instrumento certificado queda como requisito de cierre metrológico.	Pendiente
4.2.1	Determinar el DANL, entendido como el nivel más bajo de señal detectable en ausencia de señal presente.	La caracterización requiere medición controlada del piso de ruido por banda, antena, ganancia y ancho de banda configurado.	Pendiente
4.2.2	Determinar el rango dinámico entre la señal más grande medible sin distorsión y la señal mínima detectable sobre el ruido.	HackRF One usa ADC de 8 bits y requiere control estricto de ganancia para evitar saturación; falta barrido formal con señales patrón.	Pendiente
4.2.3	Determinar la exactitud de frecuencia como diferencia máxima aceptable entre la frecuencia real y la medida por el equipo.	La exactitud se debe validar con fuente RF certificada y registro de offset por configuración de muestreo.	Pendiente
4.2.4	Determinar la precisión de amplitud como margen de error en la medición de potencia de una señal, expresado en dB.	La cadena antena, mux, filtros, cables y HackRF requiere presupuesto de pérdidas y comparación contra instrumento patrón.	Pendiente

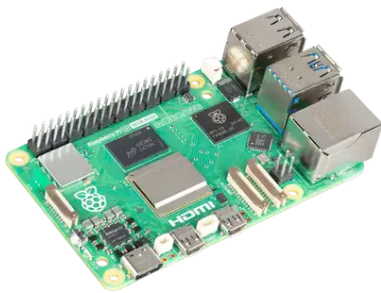
4.3.2. Arquitectura física del nodo

El nodo se compone de una Raspberry Pi 5, una tarjeta HackRF One y una PCB MonRaF2. La Raspberry Pi 5 coordina la ejecución de software, el HackRF One realiza la recepción SDR y MonRaF2 concentra conectividad, geolocalización, señales de control, protecciones y rutas RF.

Tabla 4.2: Inventario funcional de componentes de hardware del nodo ANE2.

Componente	Función principal	Criterio de integración
Raspberry Pi 5	Computador de borde	Ejecuta control, telemetría, adquisición, procesamiento y comunicación. No debe tratarse como fuente principal para todos los periféricos en una versión robusta.
HackRF One	Receptor SDR	Digitaliza señales RF como muestras IQ. Debe recibir la ruta RF ya seleccionada/acondicionada y requiere una alimentación estable para evitar fallas bajo carga.
MonRaF2	Tarjeta de integración	Integra LTE/GNSS, UART/USB/GPIO, protecciones, filtros y una ruta RF con función tipo mux/switch. Es el punto de terminación de antenas y control de rutas RF.
SIM7600G-H	LTE/GNSS	Proporciona datos celulares y ubicación. Su consumo y transitorios deben considerarse dentro del árbol de potencia.
Antenas RF, LTE y GNSS	Interfaz electromagnética	Se conectan al subsistema de MonRaF2. La conexión no debe documentarse como antena directa al HackRF, porque el diseño incluye una etapa de selección/acondicionamiento RF.
Fuente y gabinete	Energía y protección física	La fuente debe proveer margen de corriente; el gabinete debe proteger el montaje, conservar ventilación y sellar puntos vulnerables como alimentación y coaxiales.

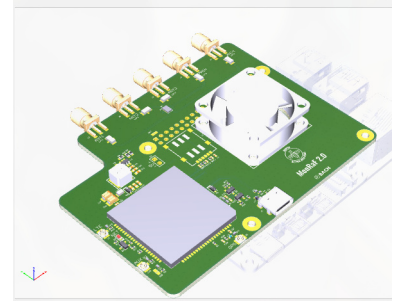
La Figura 4.3 muestra los tres módulos principales del nodo. Su separación ayuda a diagnosticar fallas: no es lo mismo un problema de ruta RF, de alimentación, de procesamiento, de enlace celular o de ubicación.



(a) Raspberry Pi 5.



(b) HackRF One.



(c) MonRaF2.

Figura 4.3: Módulos principales del hardware ANE2. Fuente: elaboración propia.

4.3.3. Fichas técnicas de componentes

Tabla 4.3: Fichas técnicas resumidas de los elementos de hardware.

Elemento	Parámetros relevantes	Rol dentro de ANE2	Estado
Raspberry Pi 5	ARM Cortex-A76 de cuatro núcleos, RAM de 8 GB, USB 3.0, GPIO, Ethernet/WiFi	Computador de borde para adquisición, procesamiento, telemetría, control RF y comunicación con la plataforma.	Implementado
HackRF One	1 MHz–6 GHz, hasta 20 MHz de ancho instantáneo, ADC de 8 bits, LNA 0–40 dB, VGA 0–62 dB	Receptor SDR principal; entrega muestras IQ y permite campañas de monitoreo multibanda.	Implementado
MonRaF2	PCB de integración con puertos SMA, rutas RF, filtros, mux/switch, GPIO, UART/USB, SIM7600G-H y protecciones	Concentra antenas, conectividad celular, GNSS, control auxiliar y adaptación de rutas hacia el receptor.	Implementado
SIM7600G-H	LTE Cat-4, LTE-FDD/TDD, GSM/GPRS/EDGE, WCDMA/HSPA+, USB 2.0, UART, GNSS multi-constelación	Enlace celular de datos, registro de red, ubicación y soporte de sincronización operacional.	Implementado
Antena TDT	470–698 MHz, recepción UHF, adaptación a SMA, ganancia típica 5–7.5 dBi	Recepción de televisión digital terrestre y pruebas RMTDT.	Implementado
Antena VHF/UHF	25 MHz–1.3 GHz, 50 Ω , adaptadores SO239–SMA/BNC, ganancia promedio cercana a 3.5 dBi	Monitoreo de VHF, UHF, FM y frecuencias intermedias.	Implementado
Antena > 2 GHz	600–960 MHz y 1710–6000 MHz, 50 Ω , SMA macho, ganancia máxima cercana a 6 dBi	Cobertura de bandas móviles, LTE/5G sub-6 y servicios de microondas.	Implementado

Elemento	Parámetros relevantes	Rol dentro de ANE2	Estado
Fuente	5 V DC por USB-C, adaptador recomendado de 45 W, reserva para cargas SDR/LTE	Alimentación del nodo; debe evolucionar a rieles medibles y distribuidos para campo.	Parcial
Gabinete	Protección ambiental tipo IP68 en chasis, sellado externo de alimentación, separación de antenas y ventilación	Protección mecánica, instalación en campo y reducción de exposición a polvo, humedad e interferencias.	Parcial

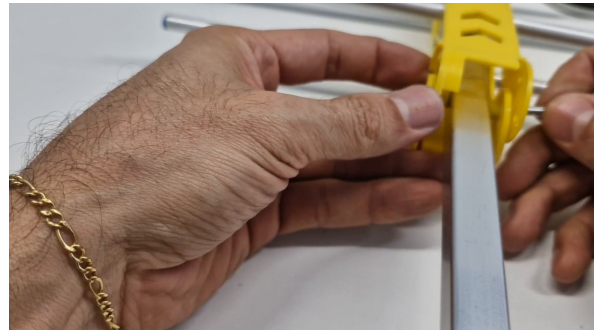
4.3.4. Antenas y montaje mecánico

El kit de antenas se organiza por banda de trabajo: TDT para UHF de televisión, VHF/UHF para monitoreo amplio y una antena de alta frecuencia para bandas móviles y servicios por encima de 2 GHz. Esta separación evita exigir a una sola antena un comportamiento plano en todo el rango del HackRF y facilita que la plataforma seleccione puerto y preajuste según la campaña.

La caracterización disponible muestra que la antena VHF/UHF tiene mejor comportamiento entre 200 MHz y 600 MHz, con degradación progresiva por encima de 1 GHz. La antena TDT mantiene adaptación aceptable en la banda UHF de televisión, con mínimos de SWR en frecuencias clave de la banda. Para antenas referenciadas a tierra, el plano de tierra no es un accesorio: mejora la adaptación, estabiliza resonancias y reduce potencia reflejada.



(a) Alineación de elemento en antena TDT.



(b) Fijación del soporte direccional TDT.

Figura 4.4: Puntos críticos de ensamble de la antena TDT. Fuente: elaboración propia.



(a) Base magnética y puerto de conexión.



(b) Instalación de radiales y elemento vertical.

Figura 4.5: Puntos críticos de ensamble de la antena VHF/UHF. Fuente: elaboración propia.

4.3.5. Ruta RF, puertos, filtros, mux y pérdidas

La ruta RF no debe describirse como una antena conectada directamente al HackRF. El esquemático de MonRaF2 muestra redes de antena como `MAIN_ANT`, `GNSS_ANT` y `AUX_ANT`, además de conectores RF, filtros LPF y una etapa de conmutación/mux RF controlada por señales GPIO. En términos operativos, MonRaF2 actúa como la tarjeta que recibe y ordena las rutas de antena antes de que la señal sea usada por el subsistema de adquisición o por el módulo celular/GNSS.

El flujo físico recomendado queda entonces:

1. Las antenas externas se conectan a puertos SMA de MonRaF2.
2. MonRaF2 selecciona, protege o acondiciona rutas RF mediante su red de filtros, conectores y mux/switch RF.
3. La ruta RF seleccionada alimenta el subsistema correspondiente: adquisición SDR, LTE o GNSS, según el modo de operación.
4. La Raspberry Pi 5 controla la operación y recibe los datos del HackRF por USB, pero no debe asumirse como concentrador físico de antenas.

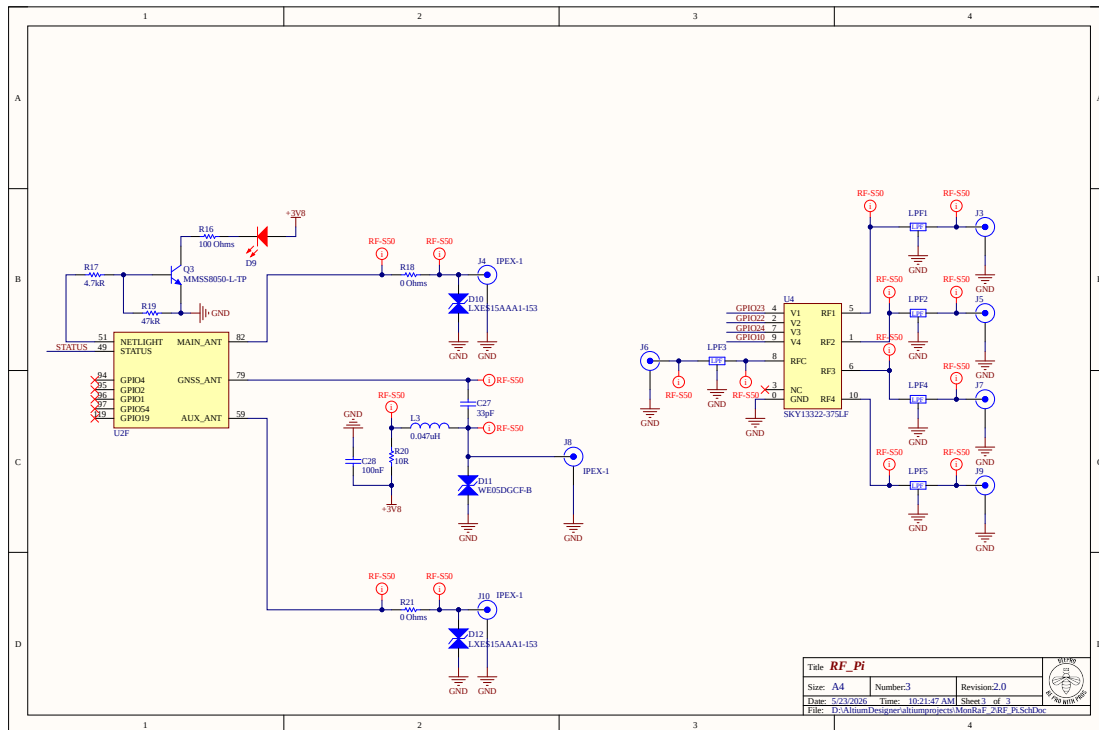


Figura 4.6: Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en MonRaF2.
Fuente: elaboración propia.

Tabla 4.4: Presupuesto cualitativo de ruta RF y puntos de pérdida.

Tramo	Pérdida o res- tricción	Criterio de control
Antena a coa- xial	Adaptación de impedancia y conectores	Mantener $50\ \Omega$ en la cadena de medición; cuan- do la antena original sea $75\ \Omega$, usar adaptación hacia SMA.
Coaxial y adap- tadores	Pérdida por lon- gitud, transición y apriete	Usar cables cortos, conectores firmes y verifica- ción visual antes de cada campaña.
Puerto SMA de MonRaF2	Selección física de antena	Registrar puerto/antena en la plataforma para no confundir una medición de banda con otra.
Filtros LPF y protección	Limitación de banda y protec- ción de entrada	Validar que el filtro corresponde a la ruta selec- cionada y que no bloquea la banda objetivo.
Mux/switch RF	Pérdida típica de inserción cerca- na a 1.25 dB	Incluir esta pérdida en el margen de sensibilidad y mantener GPIO en el estado correcto.
Entrada HackRF	Saturación y pi- so de ruido	Ajustar LNA/VGA por escenario; conservar la configuración durante comparaciones.
LTE/GNSS	Separación de rutas y antenas	Evitar que transmisión celular o mala ubicación GNSS contaminen mediciones de RF.

4.3.6. GPS/GNSS y conectividad LTE

El módulo SIM7600G-H cumple dos funciones distintas: comunicación celular y georreferenciación. En conectividad, soporta GSM/GPRS/EDGE, WCDMA/HSPA+ y LTE-FDD/LTE-TDD, con categoría LTE Cat-4 de hasta 150 Mbps de bajada y 50 Mbps de subida en condiciones ideales de red. En ubicación, soporta GPS, GLONASS, BeiDou, Galileo y QZSS, con salida NMEA por UART o USB según la configuración.

La operación correcta de GNSS no depende solamente del módulo. La antena debe tener vista despejada al cielo, evitar techos de losa, marquesinas, árboles densos y superficies metálicas cercanas, y conservar un cono de visibilidad amplio hacia el cenit. Para disminuir multipath e interferencia, el sitio debe mantener separación frente a antenas transmisoras de alta potencia y evitar fachadas metálicas o vidrios extensos que reflejen señales satelitales.

La conectividad LTE debe validarse como una condición de sitio, no como una propiedad fija del hardware. Antes de dejar un nodo operativo se debe confirmar señal celular sufi-

ciente, estabilidad de túnel o canal IP, estado de la SIM, registro de red y comportamiento de LED del módem. Para despliegue, una señal RSRP mayor a -95 dBm es un criterio práctico de aceptación inicial.

4.3.7. Árbol de potencia y gestión térmica

El principal aprendizaje del hardware es que el diseño de potencia debe preceder al diseño mecánico y de software. En las primeras integraciones, gran parte de la energía de periféricos se entregaba desde la Raspberry Pi 5. Ese enfoque funciona para pruebas de bajo alcance, pero se vuelve frágil cuando se combinan SDR, LTE, GNSS, procesamiento sostenido y tráfico de red.

La evidencia de telemetría muestra tres hechos prácticos:

- Las pruebas con HackRF y peticiones desde servidor elevaron la temperatura del procesador hasta el orden de 75 °C.
- El riel EXT5V presentó caídas bajo cargas pesadas, aun cuando no todas las corridas terminaron marcando una bandera persistente de subtensión.
- Las pruebas con alimentación externa para periféricos tuvieron mejor margen para sostener carga combinada.

Por ello, el criterio para un nuevo producto o una siguiente versión debe ser:

El nodo debe tener un árbol de potencia externo y generoso, con capacidad para la carga actual y para extensiones futuras. La Raspberry Pi 5 debe ser tratada como carga y controlador, no como fuente primaria para el resto del sistema.

Tabla 4.5: Presupuesto de potencia y térmico para operación del nodo.

Elemento	Demanda o riesgo	Criterio de diseño
Raspberry Pi 5	Carga de CPU, USB y red; límite térmico operativo	Tratarla como carga crítica, mantener ventilación y evitar superar 85 °C en reposo post-instalación.
HackRF One	Consumo sostenido durante captura IQ y presión sobre USB	Alimentación estable y separación de riel frente a cargas pulsantes.
SIM7600G-H	Transitorios de red celular y registro LTE	Reservar margen de corriente y desacople cerca del módulo.
Mux RF y GPIO	Carga baja, pero sensible a estados lógicos incorrectos	Alimentar desde riel lógico estable y registrar estado de selección.
Antenas activas o futuras extensiones	Consumo adicional no siempre presente	Dejar reserva de potencia para expansión sin rediseño completo.
Gabinete	Retención térmica y sellado	Combinar protección ambiental con ventilación o disipación; no encerrar el nodo sin camino térmico.

Tabla 4.6: Criterios mínimos para una versión robusta del árbol de potencia.

Criterio	Implicación de diseño
Fuente principal sobredimensionada	Debe alimentar simultáneamente Raspberry Pi 5, HackRF, Mon-RaF2, LTE/GNSS y periféricos futuros sin operar en el límite.
Rieles distribuidos	Las cargas de alto consumo o con transitorios, como SDR y LTE, deben alimentarse desde rieles dedicados o reguladores con margen, no desde el puerto de la Raspberry Pi.
Medición de rieles	El sistema debe conservar telemetría de voltaje, corriente, temperatura y eventos de subtensión para diagnóstico.
Protección y desacople	Se deben incluir protecciones, filtrado y capacitancia local cerca de cargas ruidosas o pulsantes.
Reserva para extensiones	El diseño debe contemplar nuevos servicios, antenas, sensores, almacenamiento o comunicaciones sin rediseñar la alimentación desde cero.

4.3.8. Compatibilidad electromagnética

El hardware de ANE2 comparte en un mismo volumen físico recepción sensible, procesamiento digital, USB, LTE, GNSS y alimentación conmutada. Por esa razón, la compatibilidad electromagnética no puede tratarse como revisión estética del gabinete: afecta directamente la confiabilidad de las mediciones.

Tabla 4.7: Medidas de mitigación EMC y pruebas mínimas.

Riesgo	Mitigación	Prueba mínima
Acoplamiento LTE sobre RF	Separar físicamente antena LTE/GNSS de entrada SDR y conservar coaxiales cortos.	Comparar piso de ruido con LTE inactivo, registrado y transmitiendo.
Multipath GNSS	Evitar metal, vidrio extenso y obstrucciones sobre la antena GNSS.	Verificar fix con al menos 4 satélites y coordenadas coherentes con el sitio.
Ruido de fuente conmutada	Usar desacople local, ferritas cuando aplique y rieles separados para cargas pulsantes.	Medir estabilidad de EXT5V y observar cambios en PSD con distintas cargas.
ESD y manipulación de antenas	Mantener protección de entrada, conectores firmes y manipulación con equipo apagado.	Inspección de continuidad, apriete y ausencia de daño antes de energizar.
Encierro térmico	No instalar en caja estanca sin disipación o ventilación natural.	Verificar temperatura en reposo y bajo adquisición sostenida.

4.4. Entradas, salidas e interfaces

Tabla 4.8: Interfaces principales del hardware ANE2.

Interfaz	Tipo	Contrato operacional
Antenas MonRaF2	a RF coaxial	Las antenas se conectan a MonRaF2. La etapa RF de la tarjeta selecciona o acondiciona rutas antes de la adquisición o uso LTE/GNSS.
MonRaF2 HackRF	a RF seleccionado	El HackRF debe recibir una ruta RF controlada, no una conexión de antena asumida como directa.
HackRF Raspberry Pi 5	a USB	Transporta muestras IQ y comandos de configuración. El <i>sample rate</i> aumenta simultáneamente el ancho instantáneo y la presión sobre USB/CPU.
Raspberry Pi 5 a MonRaF2	GPIO, UART, USB	Permite control auxiliar, comunicación con LTE/GNSS y señales de estado.
MonRaF2 red celular	a LTE	Provee conectividad de datos para estado, mediciones, campañas y reintentos.
MonRaF2 GNSS	a GNSS/GPS	Entrega ubicación del nodo para reportes y trazabilidad de mediciones.
Fuente externa a cargas	DC regulado	Debe distribuir potencia a Raspberry Pi 5, HackRF, MonRaF2 y expansiones con margen suficiente.

4.5. Decisiones de diseño o implementación

Tabla 4.9: Decisiones de diseño de hardware y justificación.

Decisión	Justificación	Implicación práctica
Usar MonRaF2 como concentrador RF y de conectividad	La tarjeta contiene rutas de antena, filtros, mux/switch RF, LTE/GNSS y señales de control.	Las antenas se documentan y cablean hacia MonRaF2; el HackRF queda como receptor SDR dentro de la cadena, no como punto directo de antena.
Usar Raspberry Pi 5 como computador de borde	Tiene capacidad suficiente para orquestación, telemetría y control del SDR.	Debe protegerse de sobrecarga eléctrica; no debe alimentar todos los periféricos en una versión robusta.

Decisión	Justificación	Implicación práctica
Usar HackRF One como receptor SDR	Permite recepción flexible por software y adquisición IQ.	Requiere ganancia configurada con criterio y alimentación estable.
Separar potencia de control	Los problemas de sub-tensión aparecen cuando la distribución de energía queda demasiado acoplada a la Raspberry Pi.	El árbol de potencia externo pasa a ser requisito de producto.
Mantener telemetría de salud	Temperatura, voltajes, corrientes y banderas de <i>throttling</i> permiten diagnosticar fallas reales.	Cada prueba de RF o conectividad debe acompañarse de telemetría eléctrica.
Usar antenas por banda	Las antenas no son verdaderamente planas en todo el rango del SDR.	La plataforma y el operador deben seleccionar la antena adecuada para cada campaña.

4.6. Problemas presentados y ajustes realizados

Tabla 4.10: Problemas presentados, decisiones y ajustes realizados en hardware.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión y bajo margen de potencia	Caídas de EXT5V bajo cargas SDR/LTE y escenarios con eventos de alimentación insuficiente.	Demasiadas cargas alimentadas o condicionadas desde la Raspberry Pi 5.	Diseñar un árbol de potencia externo, generoso y medible.	Requisito obligatorio para una versión robusta.
Aumento de temperatura bajo carga	Pruebas con HackRF y peticiones desde servidor alcanzaron temperaturas cercanas a 75 °C.	Procesamiento sostenido, actividad USB y carga RF/LTE simultánea.	Mantener disipación, ventilación y telemetría de temperatura.	La temperatura queda como restricción operativa.
Ambigüedad en ruta de antenas	El diseño incluye redes MAIN/GNSS/AUX, filtros y mux/switch RF en MonRaF2.	Interpretar la antena como conexión directa al HackRF simplifica de forma incorrecta la cadena RF.	Documentar MonRaF2 como entrada y selección RF.	La conexión física queda alineada con el esquemático.
Dependencia del enlace LTE	Las pruebas de campo usan conectividad celular para operación y envío de datos.	La red celular puede variar por cobertura, antena, SIM y ubicación.	Separar diagnóstico de RF, cómputo, potencia y LTE.	Las fallas se pueden atribuir con mejor evidencia.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Riesgo de saturación RF	HackRF no usa AGC de hardware en esta cadena de operación.	Ganancia excesiva o emisores fuertes pueden saturar el ADC de 8 bits.	Ajustar ganancia por escenario y conservar configuraciones durante comparaciones.	Menor riesgo de confundir artefactos con emisiones reales.
Adaptación de antenas	VHF/UHF, TDT y > 2 GHz tienen bandas óptimas distintas.	Una antena única no cubre con buen desempeño todo el rango 1 MHz–6 GHz.	Mantener kit por banda y registrar puer-to/antena en campaña.	Mejor trazabilidad entre medición, banda y condición física.

4.7. Validación, pruebas o evidencias

La validación usada para esta sección combina inspección de esquemático, fotografías del montaje, ensamble de antenas, caracterización de antenas, pruebas de aceptación funcional y telemetría de nodos bajo reposo, carga SDR, carga LTE y carga combinada. Los indicadores más útiles fueron temperatura ARM, EXT5V, corrientes de rieles, banderas de subtensión, frecuencia de CPU y estados de *throttling*.

Tabla 4.11: Resumen de telemetría eléctrica y térmica inspeccionada.

Escenario	Temperatura ARM	EXT5V	UV	Lectura para operación
Nodo en reposo	54.3–57.6 °C	5.006–5.041 V	No	Referencia base con sistema operativo y telemetría.
HackRF en carga alta	54.9–74.7 °C	4.748–4.965 V	No	La carga SDR aumenta demanda y temperatura.
Periféricos conectados en reposo	45.0–52.1 °C	4.903–4.963 V	No	El margen mejora cuando no hay procesamiento pesado.
Petición media con periféricos	61.5–75.7 °C	4.721–5.083 V	No	El sistema se acerca a límites térmicos y de alimentación.
Carga alta con periféricos externos	49.4–75.2 °C	4.904–5.122 V	No	La alimentación externa mejora el margen frente a carga combinada.

Tabla 4.12: Protocolo de validación de hardware.

Req.	Condición de prueba	Instrumento de medio	Resultado observado	Criterio de aceptación	Estado
3.1.1	LTE de sitio	Panel de red, SIM y estado del módem	Registro de red y comunicación LTE cuando hay cobertura.	SIM institucional activa, canal estable por más de 10 minutos y RSRP mayor a -95 dBm.	Parcial
3.1.1	GNSS de sitio	Coordenadas reportadas y condición de cielo abierto	El nodo reporta ubicación cuando la antena tiene visibilidad suficiente.	Coordenadas coherentes con sitio, lock de al menos 4 satélites y error dentro de la meta horizontal y vertical definida.	Parcial
3.2.1	Inspección de ruta RF y multiplexación	Esquemático, revisión visual y tabla de puertos	MonRaF2 concentra MAIN/GNSS/AUX, filtros, mux RF y control GPIO.	Cada puerto debe mapearse a una banda, conmutar desde software y dejar reservada la cuarta ruta.	Parcial
3.2.2	Ensamble de antena TDT	Inspección de boom, elementos y soporte	Elementos y soporte quedan fijados y alineados.	Sin piezas flojas, conector firme y orientación coherente con campaña TDT.	Implementado
3.2.2	Ensamble de antena VHF/UHF	Inspección de base, radiales y elemento vertical	Base y radiales quedan instalados en el plano de referencia.	Radiales presentes, elemento vertical fijo y sin contacto accidental con metal externo.	Implementado
3.2.2	Selección de antena por campaña	Prueba funcional de monitoreo	El usuario selecciona sensor online, puerto/antena y parámetros de ganancia.	El panel habilita adquisición, confirma configuración aplicada y conserva la antena usada en la evidencia.	Implementado
3.3.1	EMC mínima	PSD comparativa, LTE activo/inactivo e inspección de sitio	Riesgo identificado por cercanía de antenas, fuente, cableado digital y gabinete.	Sin cambio anómalo de piso de ruido al activar LTE o cargas; separación física y puesta a tierra documentadas.	Recomendado
4.1.1	Sello temporal de captura	Registro NTP y hora de adquisición	La hora depende de conectividad IP estable y configuración del sensor.	El muestreo debe quedar etiquetado con hora sincronizada antes de iniciar campaña.	Parcial
4.1.2	Calibración de frecuencia, IQ y offset	Generador RF, analizador o equipo certificado	Hay análisis IQ operativo; falta cierre con instrumento certificado.	Error de frecuencia, ganancia, balance IQ y offset dentro de tolerancia definida para el sensor.	Pendiente
4.2.1	Medición DANL	Carga de 50 Ω , configuración SDR y registro PSD	Pendiente de barrido formal por banda, ganancia y ancho de banda.	Piso de ruido documentado y repetible para cada condición de operación.	Pendiente
4.2.2	Rango dinámico	Generador RF con barrido de potencia	Pendiente de determinar el punto de saturación y señal mínima útil.	Diferencia documentada entre señal máxima sin distorsión y señal mínima sobre DANL.	Pendiente
4.2.3	Exactitud de frecuencia	Fuente RF certificada	Pendiente de comparación entre frecuencia patrón y frecuencia medida.	Error máximo de frecuencia documentado por banda y configuración.	Pendiente

Req.	Condición de prueba	de Instrumento o medio	Resultado observado	Criterio de aceptación	Estado
4.2.4	Precisión de amplitud	Instrumento patrón, presupuesto de pérdidas y señal conocida	Pendiente de comprar antena, cables, mux, filtros y ganancia SDR.	Error de potencia expresado en dB y trazado contra configuración de medición.	Pendiente

4.8. Aprendizajes y transferencia de conocimiento

- El árbol de potencia define la robustez del nodo. Si se alimentan periféricos exigentes desde la Raspberry Pi, el diseño queda sin margen.
- Las antenas pertenecen a la cadena de MonRaF2 y su etapa RF. El HackRF debe verse como receptor dentro de una ruta seleccionada, no como destino directo de todas las antenas.
- La telemetría eléctrica debe acompañar cualquier prueba RF. Sin voltajes, corrientes y temperatura es fácil confundir fallas de potencia con fallas de software.
- La caracterización de antenas debe leerse por banda. El plano de tierra y los adaptadores físicos cambian SWR, S11 y eficiencia real de recepción.
- La validación de campo debe separar sitio, potencia, RF, LTE y GNSS. Si se prueban juntos desde el inicio, las fallas quedan mezcladas.
- El diseño debe reservar margen para servicios futuros. Cada extensión de software o hardware aumenta demanda de energía, temperatura y tráfico.

4.9. Limitaciones

- Las cifras de telemetría resumen pruebas disponibles; no reemplazan una caracterización eléctrica completa con instrumentos de laboratorio.
- No se presentan curvas completas de consumo por banda, ganancia, temperatura ambiente, antena o ubicación.
- El diseño RF requiere validar pérdidas, aislamiento, adaptación de impedancias y compatibilidad electromagnética antes de una versión final de campo.
- La estabilidad LTE/GNSS depende de cobertura, antenas, SIM, ubicación y configuración del operador.
- La trazabilidad EMC está en estado parcial porque falta una campaña formal de susceptibilidad, inmunidad y emisiones conducidas/radiadas.

4.10. Recomendaciones específicas

1. Diseñar una fuente principal externa con margen amplio para carga simultánea y extensiones futuras.
2. Alimentar HackRF, LTE/GNSS y cargas auxiliares desde rieles dedicados o reguladores adecuados, no desde el margen de la Raspberry Pi 5.
3. Mantener medición de voltaje, corriente, temperatura y banderas de subtensión como parte del producto.
4. Conectar y documentar antenas desde MonRaF2 y su ruta tipo mux/switch RF.
5. Revisar disipación térmica para escenarios con SDR y LTE activos al mismo tiempo.
6. Separar pruebas de reposo, SDR, LTE/GNSS y carga combinada para poder atribuir fallas con evidencia.
7. Completar la campaña EMC con pruebas de LTE activo/inactivo, fuente bajo carga, sensibilidad GNSS y comparación de piso de ruido por antena.
8. Mantener el protocolo de aceptación como tabla viva del capítulo, de modo que cada mejora futura cambie de parcial o recomendado a implementado solo cuando exista evidencia verificable.

Capítulo 5

Software de adquisición SDR del sensor

5.1. Propósito del capítulo

El propósito de este capítulo es describir, de forma técnica y verificable, la arquitectura del código que corre directamente en el sensor de monitoreo espectral remoto. El sensor está compuesto por una Raspberry Pi 5, una HackRF One y una tarjeta auxiliar que integra comunicación LTE, receptor GPS y un multiplexor para selección de antena.

La sección del proyecto descrita en este capítulo no corresponde a toda la plataforma. El sistema completo también incluye componentes externos, como backend, frontend, bases de datos y servicios de visualización. Este capítulo se concentra en el código embebido del sensor: la parte que recibe instrucciones de la plataforma, controla el hardware de radiofrecuencia, adquiere muestras, procesa señales y devuelve resultados.

El propósito general del código es convertir al sensor físico en un nodo autónomo de medición espectral. Para ello, el software debe recibir órdenes del servidor, configurar la HackRF One, seleccionar la antena adecuada, capturar señales IQ, procesarlas localmente y enviar a la plataforma central productos útiles como PSD, métricas de modulación, audio demodulado, estado del dispositivo y coordenadas GPS.

De manera específica, el software del sensor busca cumplir los siguientes objetivos:

- Controlar la adquisición de radio mediante la HackRF One, ajustando frecuencia central, tasa de muestreo, ganancias, amplificador y puerto de antena.
- Ejecutar mediciones espectrales en modo *realtime* y en campañas programadas desde el servidor.
- Procesar muestras IQ crudas para estimar densidad espectral de potencia mediante

métodos como Welch y banco de filtros polifase.

- Demodular señales AM o FM cuando el servidor solicita audio en tiempo real.
- Aplicar etapas de limpieza y corrección para mejorar la calidad de las mediciones, incluyendo balance IQ y remoción del pico DC.
- Mantener una calibración de frecuencia que compense desviaciones del oscilador de la HackRF.
- Coordinar la comunicación con el backend, el envío de resultados, los reintentos ante fallos de red y el reporte de estado del sensor.
- Gestionar recursos físicos del sensor, como LTE, GPS, GPIO y multiplexor de antena.

El capítulo describe el software actualmente implementado en el repositorio. La capa de control se encuentra principalmente en `orchestrator.py`, `campaign_runner.py`, `status.py`, `retry_queue.py`, `functions.py`, `cfg.py` y `utils/`. La capa de adquisición y procesamiento RF se encuentra en `rf/rf.c` y `rf/libs/*.c`. Los servicios de posicionamiento y conectividad se encuentran en `gps-lte/`, mientras que el audio en tiempo real se articula con `server_webrtc.py`. Por tanto, las funcionalidades descritas no son una especificación abstracta, sino una lectura organizada del código del sensor.

5.2. Contexto dentro del sistema general

El software del sensor hace parte del sistema de monitoreo espectral multisensor remoto. Dentro de la arquitectura general, el sensor actúa como un nodo de adquisición y procesamiento local, mientras que la plataforma central se encarga de coordinar tareas, almacenar resultados, visualizar información y permitir la interacción con el usuario.

La arquitectura del sensor se puede entender como una cadena de trabajo: el servidor decide qué se debe medir, el orquestador traduce esa instrucción a una configuración de radio, el motor RF adquiere y procesa las muestras, y finalmente los resultados regresan al servidor.

La división principal del sistema es la siguiente:

- **Orquestación en Python:** gestiona la relación con el backend, consulta instrucciones, programa campañas, controla estados del sistema, envía comandos al motor RF y prepara los resultados para publicarlos.
- **Motor RF en C:** controla la HackRF One, recibe muestras IQ, administra buffers de alta velocidad, ejecuta procesamiento DSP, calcula PSD, demodula audio y devuelve resultados al orquestador.

- **Servicios auxiliares:** atienden funciones de soporte como GPS, LTE, estado del sistema, reintentos de datos no enviados, WebRTC para audio y control del multiplexor de antena.

La decisión de separar Python y C responde a una necesidad práctica. Python es adecuado para coordinación, comunicación HTTP, manejo de estados y programación de tareas. C es adecuado para la parte sensible a latencia: captura de muestras, buffers, FFT, filtrado, PSD y demodulación.

El sensor opera con varios servicios o modos complementarios:

- **PSD en tiempo real:** el backend solicita una configuración y el sensor responde con una estimación espectral de la banda capturada.
- **PSD y audio en tiempo real:** además de la PSD, el sensor demodula FM o AM y genera tramas de audio reproducibles por la plataforma.
- **Campañas:** el backend programa mediciones periódicas; el sensor guarda la configuración, agenda ejecuciones y sube resultados en cada intervalo.
- **Calibración en frecuencia:** el sensor estima el error de sintonía y aplica una corrección para que las mediciones queden mejor alineadas en frecuencia.
- **GPS y LTE:** el sensor mantiene conectividad celular y reporta ubicación geográfica al servidor.
- **Estado del dispositivo:** se reportan variables de salud como conectividad, almacenamiento, temperatura, memoria y actividad del software.

5.2.1. Arquitectura funcional del software SDR

La ruta principal de una medición puede resumirse como una cadena de control y una cadena de datos. La cadena de control inicia en el backend, pasa por el orquestador Python y llega al motor RF mediante mensajería local. La cadena de datos inicia en la HackRF One, produce muestras IQ, ejecuta procesamiento digital de señales en C y retorna productos resumidos hacia la plataforma.

En el diagrama, el orquestador Python se muestra como el proceso de coordinación: consulta y publica por API REST, activa servicios de tiempo real, campañas y audio, agenda campañas mediante cron y aplica el postproceso final de PSD. El motor RF en C concentra el camino crítico de adquisición y DSP: configura la HackRF, administra buffers, preprocesa IQ, estima PSD, produce métricas y usa multihilo para las etapas de cómputo intensivo. El servicio WebRTC corre en paralelo al orquestador y se activa cuando hay demodulación; toma audio demodulado del camino RF y lo entrega mediante WebRTC

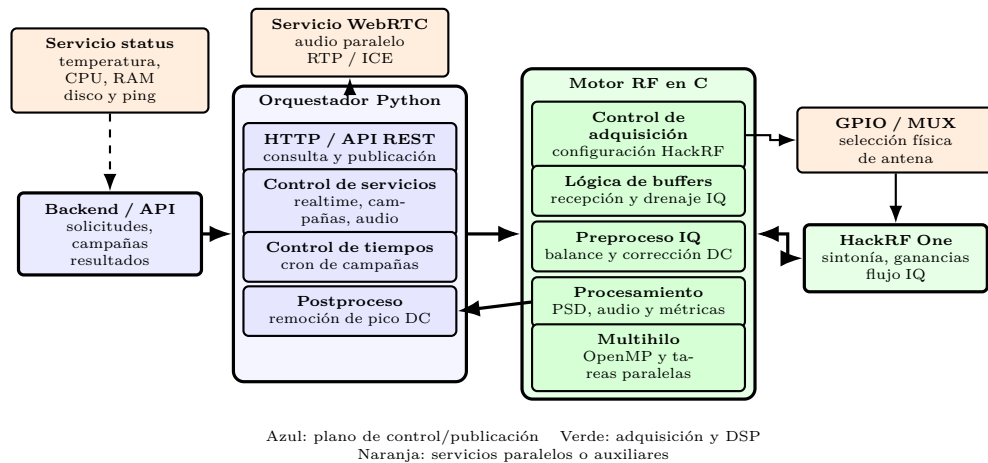


Figura 5.1: Diagrama de bloques de la arquitectura funcional del software SDR del sensor.

usando RTP e ICE. El servicio de estado también corre de forma continua y reporta métricas de salud del sistema hacia la plataforma.

Bloque	Responsabilidad técnica
Backend	Publica solicitudes de medición, campañas, parámetros RF y recibe resultados.
Orquestador Python	Consulta tareas por API REST, coordina servicios, agenda campañas y aplica postproceso de PSD.
ZeroMQ IPC	Transporta comandos JSON locales entre Python y C usando un flujo estricto REQ/REP.
Motor RF C	Configura HackRF, controla adquisición, buffers, preproceso IQ, PSD, métricas, audio y multihilo.
libhackrf/USB	Ejecuta la comunicación de bajo nivel con la HackRF One y entrega muestras IQ intercaladas.
WebRTC/Opus	Corre en paralelo, se activa con demodulación y entrega audio mediante WebRTC, RTP e ICE.
Servicio status	Reporta temperatura, carga de CPU, RAM usada, espacio en disco y ping hacia la plataforma.
ShmStore	Comparte estado en <code>/dev/shm/persistent.json</code> , como <code>ppm_error</code> , GPS y campañas.
GPS/LTE/GPIO	Mantiene ubicación, conectividad celular y control físico del multiplexor de antenas.

Esta separación define una frontera importante: el sensor ejecuta adquisición, procesamiento básico y control local; la plataforma central se encarga de persistencia histórica,

visualización, comparación institucional, reportes y analítica de mayor nivel. La tabla siguiente resume esa distribución.

Nivel	Funciones principales
Sensor/edge	Adquisición IQ, PSD local, filtrado, demodulación AM/FM, GPS, estado, reintentos y audio.
Backend/cloud	Almacenamiento histórico, campañas, gestión de sensores, APIs, consolidación y reglas institucionales.
Frontend	Visualización, selección de parámetros, marcadores visuales, navegación de resultados y operación del usuario.

5.3. Desarrollo técnico

El código puede leerse como un conjunto de bloques funcionales. Cada bloque cumple una responsabilidad dentro del camino completo de la señal: desde la instrucción del backend hasta la publicación del resultado.

5.3.1. Adquisición

El bloque de adquisición es el encargado de hablar con la HackRF One y convertir una instrucción de medición en muestras IQ reales.

Conceptualmente, este bloque realiza cinco acciones:

1. Recibe una configuración de medición enviada por el orquestador. Esa configuración contiene parámetros como frecuencia central, ancho de muestreo, ganancias, método de PSD, puerto de antena, filtro solicitado y modo de demodulación.
2. Aplica la configuración sobre la HackRF One. Esto incluye sintonía de frecuencia, tasa de muestreo, ganancias de recepción y corrección de frecuencia cuando existe una calibración disponible.
3. Inicia la recepción de muestras IQ crudas desde el dispositivo SDR.
4. Deposita esas muestras en buffers circulares para separar la velocidad de llegada del hardware de la velocidad de procesamiento del software.
5. Entrega bloques de muestras suficientemente grandes a los algoritmos de procesamiento, evitando que cada operación pesada interrumpa directamente la captura.

Las muestras IQ representan la señal recibida en banda base compleja. La componente I corresponde a la parte en fase y la componente Q a la parte en cuadratura. En conjunto permiten conservar información de amplitud y fase, que es necesaria para estimar el espectro, filtrar, demodular FM o demodular AM.

En términos operativos, una adquisición completa sigue el siguiente procedimiento:

1. Validar que la máquina de estados global permita la medición solicitada. El sistema evita ejecutar adquisiciones simultáneas entre **REALTIME**, **CAMPAIGN** y **KALIBRATING**.
2. Construir una configuración JSON con frecuencia central, tasa de muestreo, RBW, ventana, ganancias, antena, filtro, modo de demodulación y error de frecuencia `ppm_error` cuando está disponible.
3. Enviar la configuración al motor RF por ZeroMQ y esperar una respuesta antes de emitir otro comando, porque el contrato local es REQ/REP.
4. Seleccionar el puerto de antena mediante GPIO cuando el hardware auxiliar lo requiere.
5. Configurar la HackRF One con `libhackrf`: frecuencia central, *sample rate*, LNA, VGA, amplificador y corrección de frecuencia.
6. Iniciar el flujo USB de muestras IQ, llenar buffers circulares y vigilar condiciones de saturación, ausencia de dispositivo o error de streaming.
7. Cerrar de forma controlada la captura, liberar recursos temporales y devolver al orquestador una respuesta JSON con resultados o error.

5.3.2. Parámetros SDR configurables

Los parámetros de medición llegan desde el backend, son validados por el orquestador y finalmente son interpretados por el parser del motor RF. La tabla resume los parámetros más relevantes para adquisición y procesamiento.

Parámetro	Unidad/formato	Uso en el sensor
Frecuencia central	Hz	Sintonía nominal de la HackRF. Puede corregirse internamente con <code>ppm_error</code> .
Sample rate	muestras/s	Ancho de adquisición instantáneo y base para calcular resolución frecuencial.

Parámetro	Unidad/formato	Uso en el sensor
Span	Hz	Banda observada alrededor de la frecuencia central; en la práctica queda limitado por el sample rate útil.
FFT size / <code>nperseg</code>	muestras	Número de puntos por segmento para Welch o tamaño de FFT en procesamiento espectral.
RBW	Hz	Resolución aproximada de frecuencia; se relaciona con <code>sample_rate/FFT size</code> .
Overlap Welch	fracción	Porcentaje de solape entre segmentos para estabilizar la PSD.
Ventana	texto	Tipo de ventana espectral: Hamming, Hann, Blackman, Kaiser, Tukey, entre otras.
Método PSD	texto	Selección entre Welch y banco de filtros polifase cuando se requiere mayor selectividad.
LNA/VGA	dB configurados	Ganancias de recepción de la HackRF; afectan nivel relativo y riesgo de saturación.
Amplificador RF	booleano	Habilita o deshabilita el amplificador frontal de la HackRF.
Puerto de antena	entero	Selección física del camino RF mediante el multiplexor de antenas.
Filtro digital	Hz inicio/fin	Recorta la región de interés dentro de la banda realmente capturada.
Demodulación	AM/FM/ninguna	Activa el camino de audio en tiempo real.
Cooldown	segundos	Pausa solicitada entre capturas o ciclos de consulta.
<code>ppm_error</code>	ppm	Corrección de frecuencia estimada durante calibración.

5.3.3. Span, sample rate, FFT y RBW

En el sensor, *sample rate*, span, tamaño de FFT y RBW no son parámetros independientes. La tasa de muestreo define el ancho instantáneo máximo que puede observarse en una sola

captura. El tamaño de FFT o `nperseg` define cuántos puntos se usan para dividir esa banda en contenedores de frecuencia. Como regla práctica:

$$\text{RBW} \approx \frac{\text{sample_rate}}{\text{FFT size}}$$

Una RBW menor mejora la separación entre componentes cercanas, pero exige más muestras por segmento, más memoria temporal, más cómputo FFT y normalmente mayor latencia. Una RBW mayor reduce costo computacional y mejora la respuesta temporal, pero disminuye la capacidad de distinguir señales próximas. Por esta razón, el motor RF no debe reducir unilateralmente la tasa de muestreo o el tamaño de FFT solicitados para ahorrar recursos: esos parámetros hacen parte de la medición definida por el servidor.

5.3.4. Uso de buffers

La lógica de buffers es central en el sensor. La HackRF entrega datos de forma continua y rápida, mientras que la PSD, el filtrado o la demodulación requieren procesamiento por bloques. Por eso el motor RF usa buffers circulares:

- Un buffer principal recibe muestras IQ para estimación espectral.
- Un buffer de audio conserva muestras necesarias cuando se activa demodulación AM o FM.
- El software drena estos buffers cuando ya hay suficientes muestras para construir una respuesta.

Esta estructura permite que el sensor atienda procesos en tiempo real, como PSD y audio, sin tratar cada muestra de manera aislada. La idea pedagógica es similar a una banda transportadora: el hardware deposita muestras, el procesamiento toma paquetes completos y el orquestador recibe resultados ya elaborados.

5.3.5. Preprocesamiento

El preprocesamiento agrupa las operaciones que preparan la señal antes de extraer productos finales. Su objetivo es reducir artefactos propios del hardware y mejorar la calidad de la medición.

Balance IQ

En un receptor SDR real, las ramas I y Q no siempre quedan perfectamente balanceadas. Puede haber diferencias de ganancia, pequeños desplazamientos DC o correlación no deseada entre ambas componentes. Si estos efectos no se corrigen, pueden aparecer artefactos en el espectro o degradación en la demodulación.

El balance IQ busca compensar esos errores antes del cálculo de PSD o del filtrado. De forma conceptual, el algoritmo:

- remueve desplazamientos promedio en las ramas I y Q;
- compara la energía relativa de ambas ramas;
- corrige diferencias de amplitud;
- reduce correlaciones espurias entre I y Q.

No se trata de cambiar la información de la señal recibida, sino de disminuir imperfecciones introducidas por la cadena de recepción.

Corrección de componente DC

La componente DC aparece cerca del centro de la banda base y suele estar asociada a efectos internos del receptor. En el flujo del sensor se atiende en dos niveles:

- En el preproceso IQ se reduce el offset de las muestras antes de calcular PSD o demodular.
- En el postproceso de PSD se repara el pico central visible en el espectro cuando aparece como un artefacto de medición.

Esta separación es importante: una cosa es limpiar la señal cruda y otra es pulir el producto espectral que se enviará al servidor.

5.3.6. Procesamiento

El bloque de proceso transforma muestras IQ en productos útiles para la plataforma. En este repositorio se implementan tres familias principales de procesamiento: estimación de PSD, demodulación de audio y filtrado de señales.

Estimación de PSD

La PSD, o densidad espectral de potencia, permite observar cómo se distribuye la energía de la señal dentro de un rango de frecuencias. Es el producto principal para monitoreo espectral, porque permite identificar ocupación de banda, niveles relativos de potencia, portadoras y posibles emisiones.

El sensor contempla métodos como:

- **Welch:** divide la señal en segmentos, aplica ventanas, calcula FFT y promedia resultados para obtener una estimación estable del espectro.
- **Banco de filtros polifase:** mejora la separación entre canales y ayuda a controlar fugas espectrales cuando se requiere una estimación más selectiva.

En la implementación, el método Welch es el camino principal para entregar una PSD estable en tiempo real y en campañas. El banco de filtros polifase se reserva para casos donde se requiere mejor separación espectral o menor fuga entre canales, asumiendo un mayor costo computacional. La configuración incluye tamaño de segmento, solape, tipo de ventana, tasa de muestreo y método PSD; con estos datos se genera el eje de frecuencia y el vector de potencia relativo.

La PSD es una de las tareas más costosas del sistema, porque requiere operar sobre grandes bloques de muestras y ejecutar transformadas de Fourier. Por eso el motor RF está escrito en C y aprovecha una lógica de trabajo multihilo, planes FFTW y buffers reutilizables por hilo. El uso de cachés locales reduce la sobrecarga de planificación FFT, pero exige cuidado con memoria dinámica, OpenMP y almacenamiento local de hilo.

Escala de potencia y unidades

El resultado logarítmico que produce el sensor debe interpretarse como una escala relativa al rango digital del receptor, es decir, como dBFS o potencia relativa, no como una medición absoluta en dBm. En este proyecto no se realizó una calibración de potencia de la cadena RF completa. Por tanto, no se conoce con certeza la tensión equivalente de saturación del ADC, las atenuaciones internas de la HackRF, las pérdidas de cables, el factor de antena, la ganancia real de cada etapa ni la respuesta en frecuencia del conjunto antena-cable-SDR.

Esta aclaración es crítica: aunque el sistema puede mostrar diferencias relativas entre señales, detectar ocupación y comparar niveles dentro de una misma configuración, no debe reportarse como instrumento calibrado en dBm, dB μ V/m, mV/m, W/m² o dBW/m²/MHz sin una calibración externa. Para convertir la escala actual a unidades absolutas se requeriría caracterizar el sistema con un generador RF calibrado, niveles conocidos, varias

frecuencias, varias ganancias, registro de pérdidas y una curva de corrección con incertidumbre asociada.

Demodulación FM y AM

Cuando el backend solicita audio en tiempo real, el sensor no solo calcula PSD. También toma las muestras IQ, extrae la información modulada y la convierte en audio reproducible.

En FM, la información está principalmente en la variación de fase o frecuencia instantánea. El proceso conceptual consiste en observar cómo cambia la fase entre muestras consecutivas, convertir esa variación en una señal de audio y acondicionar el resultado para reproducción.

En AM, la información está principalmente en la envolvente de la señal. El proceso conceptual consiste en calcular la magnitud de la señal compleja, separar las variaciones útiles de la portadora y normalizar el audio resultante.

Después de demodular, el audio se convierte a un formato de transmisión. El sensor genera tramas de audio, las codifica con Opus y las entrega al servicio local que publica el flujo mediante WebRTC. Esto permite que la plataforma escuche la señal con baja latencia sin que el backend tenga que procesar IQ crudo.

Filtrado

El filtrado permite concentrarse en una parte de la banda capturada y rechazar contenido fuera del rango de interés. El servidor puede solicitar un rango de frecuencia y el motor RF aplica un filtro pasabanda antes de calcular la PSD o de preparar la señal.

Desde una perspectiva conceptual, el filtro funciona como una ventana selectiva: conserva la región que interesa y atenúa lo que queda por fuera. En el código se apoya en procedimientos de dominio frecuencial y validaciones para que el rango solicitado sea coherente con la banda realmente capturada por la HackRF.

Actualmente el filtro se expresa como un intervalo de frecuencias de inicio y fin dentro de la banda capturada. Si el rango solicitado cae por fuera de los límites de Nyquist definidos por la frecuencia central y la tasa de muestreo, el parser ajusta el intervalo al rango físicamente observable. Filtros pasabajo o pasaaltos pueden representarse como casos particulares de ese intervalo, pero una caracterización formal de orden, ancho de transición y atenuación fuera de banda debería documentarse como mejora futura si se requiere trazabilidad metrológica.

Trabajo multihilo

El sistema separa tareas para mantener capacidad de respuesta:

- Un flujo atiende la llegada de muestras desde la HackRF.
- Otro flujo procesa bloques para PSD.
- Cuando hay audio, un hilo adicional drena muestras, demodula y envía tramas codificadas.
- El orquestador permanece disponible para comunicarse con el backend y manejar cambios de modo.

Esta organización evita que una tarea pesada, como una PSD de alta resolución, bloquee completamente la captura o la transmisión de audio.

La lógica de multitarea y multihilo usada para paralelizar regiones de cómputo dentro del motor RF se apoya en la librería OpenMP para C. Esta herramienta permite distribuir operaciones repetitivas, como acumulaciones, compensación IQ y segmentos de PSD, entre varios hilos de ejecución de forma relativamente sencilla, sin tener que implementar manualmente toda la administración de hilos mediante interfaces de menor nivel.

El uso de OpenMP aporta varias ventajas al desarrollo del sensor. En primer lugar, facilita la paralelización de operaciones repetitivas o costosas, como cálculos sobre bloques de muestras, procesamiento espectral o tareas que pueden dividirse entre núcleos de CPU. En segundo lugar, reduce la complejidad del código frente a una implementación completamente manual con hilos, bloqueos y sincronización explícita. Finalmente, es una solución ampliamente soportada en entornos de compilación en C, lo que permite mantener una implementación portable y fácil de integrar dentro del motor RF. No obstante, OpenMP se usa con cautela: el código evita depender de referencias inseguras a almacenamiento local de hilo dentro de regiones paralelas y controla la cantidad de hilos para no saturar la Raspberry Pi 5 durante PSD de alta resolución o audio simultáneo.

5.3.7. Postprocesamiento

El postprocesamiento corresponde a la pulida final aplicada a los resultados antes de enviarlos al servidor. En este sistema, la etapa más importante es la corrección del pico DC en la PSD.

Remoción del pico DC en PSD

Aunque el preproceso reduce offset en las muestras IQ, en la PSD puede quedar un pico artificial cerca del centro de la banda. Ese pico no siempre representa una emisión real;

con frecuencia es un artefacto propio del receptor SDR.

El algoritmo de postproceso analiza el comportamiento estadístico de la PSD como una señal con variabilidad local. En lugar de borrar siempre un número fijo de puntos, estima la región afectada y decide cómo reconstruirla.

Conceptualmente, el procedimiento realiza:

- detección de la región central afectada usando simetría y cambios locales en la pendiente;
- análisis de ocupación espectral alrededor del centro;
- validaciones con descriptores estadísticos como media, mediana, dispersión y comportamiento de la cola alta;
- ampliación de la ventana de reparación cuando la zona tiene baja ocupación espectral;
- reconstrucción local mediante interpolación lineal o ajuste polinomial.

El resultado enviado al backend conserva una PSD más limpia. Cuando aplica, el software también puede conservar información de diagnóstico sobre la corrección, como la ventana reparada y el modo de reconstrucción usado.

5.3.8. Calibración en frecuencia

La calibración en frecuencia busca compensar la desviación entre la frecuencia nominal solicitada y la frecuencia real a la que queda sintonizado el receptor. En SDR de bajo costo, esta desviación puede aparecer por tolerancias del oscilador, temperatura o condiciones de operación.

El sensor usa una referencia conocida para estimar el error de frecuencia. En el caso de emisiones FM comerciales, una referencia útil es el tono piloto estéreo de 19 kHz que aparece después de demodular FM. Si una estación FM fuerte contiene ese tono, el sensor puede usarlo como punto de comparación.

El flujo conceptual es:

1. Capturar una porción amplia de la banda donde se esperan emisoras FM.
2. Identificar candidatos con potencia suficiente.
3. Bajar cada candidato a banda base y demodular FM.
4. Buscar el tono piloto alrededor de 19 kHz en la señal demodulada.

5. Comparar la posición observada con la posición esperada.
6. Estimar un error de frecuencia y expresarlo como factor de corrección.

Cuando hay varias emisiones candidatas, el sistema puede usar más de una referencia para evitar depender de una sola estación. Las emisiones más confiables reciben mayor peso, por ejemplo si tienen mejor potencia o una detección más clara del tono piloto. Con esa información se obtiene una estimación más robusta del error.

El resultado de la calibración se guarda como un error de frecuencia, normalmente expresado en partes por millón. Luego, cuando el backend solicita una frecuencia central, el orquestador incluye ese factor en la configuración enviada al motor RF.

El motor usa la corrección para ajustar la sintonía de la HackRF. De esta manera, la frecuencia física de adquisición queda más alineada con la frecuencia nominal que la plataforma espera visualizar.

5.3.9. Orquestador

El orquestador es la lógica de coordinación del sensor. Su función no es procesar IQ de forma intensiva, sino dirigir el trabajo de todos los bloques.

Sus responsabilidades principales son:

- Consultar al backend para saber si hay solicitudes en tiempo real.
- Consultar y sincronizar campañas programadas.
- Validar configuraciones antes de enviarlas al motor RF.
- Enviar comandos al motor RF usando mensajería local ZeroMQ.
- Recibir resultados de PSD y métricas desde el motor RF.
- Aplicar postproceso sobre los resultados cuando corresponde.
- Formatear y publicar datos hacia el backend mediante HTTP REST.
- Controlar el inicio y parada del servicio de audio WebRTC.
- Mantener estados globales como `IDLE`, `REALTIME`, `CAMPAIGN`, `KALIBRATING` y `ERROR`.
- Gestionar memoria compartida local mediante `ShmStore`; el archivo usado es `/dev/shm/persistent.json`. Allí se conservan parámetros como `ppm_error`, coordenadas GPS o configuraciones de campaña.

Las campañas son mediciones programadas por el servidor. El orquestador consulta las campañas activas, selecciona la que corresponde ejecutar, guarda sus parámetros y agenda ejecuciones periódicas. Cada ejecución captura PSD y envía el resultado al backend. Si la red falla, el resultado puede quedar en una cola local para ser reenviado posteriormente.

En modo *realtime*, el servidor entrega una configuración inmediata. El orquestador la envía al motor RF, espera la respuesta, aplica postproceso y sube el resultado. Mientras el backend siga enviando una solicitud válida, el sensor mantiene este modo activo.

Cuando la solicitud incluye demodulación, el orquestador activa también el camino de audio. En ese caso, el motor RF demodula AM o FM y genera tramas Opus; el servicio WebRTC se encarga de hacer disponible ese audio para la plataforma.

Antes o durante ciertos ciclos de operación, el orquestador puede solicitar al motor RF una calibración de frecuencia. El resultado se guarda localmente y se usa en adquisiciones posteriores.

5.4. Entradas, salidas e interfaces

El software del sensor se comunica con diferentes elementos internos y externos. Por esta razón, se puede describir en términos de entradas, salidas e interfaces.

5.4.1. Entradas

Las principales entradas del sistema son:

- Configuraciones de medición enviadas desde el backend.
- Parámetros de campañas programadas.
- Solicitudes de PSD en tiempo real.
- Solicitudes de PSD con audio en tiempo real.
- Parámetros de frecuencia central, span, ganancias, filtro, puerto de antena y modo de demodulación.
- Muestras IQ crudas entregadas por la HackRF One.
- Datos GPS recibidos desde el módulo de posicionamiento.
- Estado de conectividad entregado por el servicio LTE.
- Parámetros persistentes almacenados localmente, como el error de frecuencia en partes por millón.

5.4.2. Salidas

Las principales salidas del sistema son:

- Estimaciones de PSD enviadas al backend.
- Métricas de modulación y resultados derivados del procesamiento.
- Audio demodulado en tiempo real cuando se solicita AM o FM.
- Tramas de audio codificadas con Opus.
- Coordenadas GPS del sensor.
- Estado general del dispositivo.
- Información de conectividad, almacenamiento, temperatura, memoria y actividad del software.
- Resultados de calibración en frecuencia.
- Datos almacenados localmente para reintento cuando hay fallos de red.

Estas salidas tienen formatos y alcances diferentes. Para evitar ambigüedad, la tabla siguiente resume qué producto genera el sensor y qué significado tiene.

Salida	Formato	Observación técnica
PSD	JSON con eje de frecuencia y potencia relativa	Producto principal de monitoreo. La escala logarítmica es relativa/dBFS si no existe calibración de potencia.
Audio	Tramas Opus/WebRTC	Disponible solo cuando se solicita demodulación AM o FM en tiempo real.
GPS	Campos de estado y coordenadas	Debe asociarse a la medición con estado de fix cuando esté disponible.
Estado	JSON HTTP	Incluye variables de salud como conectividad, temperatura, memoria, almacenamiento y actividad.
Calibración	ppm_error	Corrección de frecuencia persistida en memoria compartida. No equivale a calibración de potencia.
Reintentos	JSON local/cola	Resultados no enviados por falla de red, para posterior publicación.

Salida	Formato	Observación técnica
Logs	Archivos en Logs/	Evidencia operativa para diagnóstico y soporte.

5.4.3. Interfaces y protocolos

Los protocolos y procedimientos principales que conectan los bloques del sensor son los siguientes:

Elemento	Uso conceptual en el sensor
HTTP REST	Comunicación entre sensor y backend para consultar instrucciones y publicar datos.
ZeroMQ	Canal local de comandos entre el orquestador Python y el motor RF en C.
libhackrf/USB	Interfaz de control y adquisición de muestras desde la HackRF One.
WebRTC	Entrega de audio demodulado hacia cliente/plataforma con baja latencia.
Opus	Codificación de audio antes de enviarlo al servicio WebRTC.
GPIO	Control de líneas físicas para multiplexor de antena y soporte de hardware.
PPP/LTE	Conectividad celular del sensor hacia la red.
GPS/NMEA	Obtención y conversión de coordenadas geográficas.
Welch	Método de estimación de PSD por segmentos y promediado.
Banco polifase	Método de estimación espectral con mejor control de separación entre canales.
OpenMP	Paralelización de tareas de cómputo dentro del motor RF desarrollado en C.

5.4.4. Contrato local Python-C

La comunicación entre el orquestador y el motor RF usa ZeroMQ mediante el transporte `ipc://` y el endpoint local `/tmp/rf_engine`. Aunque la clase Python se llama `ZmqPairController`, el socket usado por Python es `REQ` y el socket del motor C es `REP`.

Por tanto, el flujo es estrictamente secuencial: por cada solicitud debe recibirse una respuesta antes de emitir el siguiente comando. Esta restricción complementa la máquina de estados global y evita adquisiciones concurrentes sobre la misma HackRF.

Comando lógico	Parámetros principales	Respuesta esperada
PSD/realtime	Frecuencia central, sample rate, RBW, ventana, método PSD, ganancias, antena y filtro	JSON con frecuencia, PSD, estado y metadatos de adquisición.
PSD con audio	Parámetros PSD más <code>demodulation=am/fm</code>	PSD y activación del flujo Opus/WebRTC para audio.
Campaña	Configuración guardada por campaña, ventana temporal e intervalo	Resultado PSD por ejecución y publicación o reintento.
Calibración de frecuencia	<code>calibrate=true</code> y configuración RF base	JSON con estado de calibración completa y <code>ppm_error</code> .
Filtro	<code>start_freq_hz</code> , <code>end_freq_hz</code>	Procesamiento limitado al rango válido dentro de la banda capturada.
Parada/reset	Cambio de modo o solicitud sin demodulación	Cierre del camino de audio o retorno a estado de espera según corresponda.

Los errores relevantes de esta interfaz incluyen JSON inválido, timeout de ZeroMQ, respuesta tardía, HackRF no disponible, frecuencia fuera del rango soportado, tasa de muestreo inválida, saturación USB, falla de streaming, filtro fuera de banda, error de demodulación o falla de publicación HTTP. Ante timeout, el controlador Python recrea el socket para descartar respuestas tardías y restaurar el estado REQ/REP.

5.5. Decisiones de diseño o implementación

Una de las decisiones principales de diseño fue dividir el software del sensor en dos capas: una capa de orquestación en Python y un motor RF en C. Esta separación permite aprovechar las ventajas de cada lenguaje. Python facilita la comunicación con el backend, el manejo de estados, la programación de campañas y la integración con servicios auxiliares. C permite ejecutar tareas de adquisición y procesamiento digital de señales con mayor control sobre memoria, buffers y rendimiento.

Otra decisión importante fue usar buffers circulares para desacoplar la velocidad de llegada de las muestras desde la HackRF de la velocidad de procesamiento del software. Esta estrategia permite atender flujos de datos continuos y evitar que operaciones costosas, como la estimación de PSD o la demodulación, interrumpan la captura de muestras.

También se decidió usar ZeroMQ como canal local de comunicación entre el orquestador Python y el motor RF en C. Esto permite separar responsabilidades, mantener independencia entre procesos y enviar comandos o recibir resultados sin integrar todo el sistema en un único ejecutable. La elección de REQ/REP mantiene un contrato simple y controlado: una solicitud de adquisición, calibración o audio debe cerrarse con una respuesta antes de iniciar otra operación RF.

Para la entrega de audio en tiempo real, se optó por demodular localmente la señal en el sensor, codificar el audio con Opus y publicarlo mediante WebRTC. Esta decisión evita enviar IQ crudo al backend para tareas de escucha y reduce la carga de procesamiento de la plataforma central.

En cuanto a procesamiento paralelo, se decidió usar OpenMP para implementar la lógica de multitarea y multihilo dentro del motor RF. Esta elección se tomó porque OpenMP facilita la paralelización de regiones de código en C, tiene soporte amplio en compiladores comunes y permite aprovechar mejor los núcleos disponibles de la Raspberry Pi 5 sin aumentar excesivamente la complejidad del código.

Finalmente, se incluyeron etapas de preprocesamiento y postprocesamiento para mejorar la calidad de los productos enviados al backend. Entre estas etapas se encuentran el balance IQ, la reducción de componente DC en las muestras y la remoción del pico DC en la PSD.

5.6. Problemas presentados y ajustes realizados

Durante el diseño del software fue necesario considerar varios problemas propios de un sistema SDR remoto.

Uno de los primeros problemas es la presencia de imperfecciones en la cadena de recepción. En un receptor SDR real, las ramas I y Q pueden presentar diferencias de ganancia, desplazamientos DC o correlación no deseada. Para mitigar estos efectos se incorporó una etapa de balance IQ y reducción de offset antes del procesamiento espectral o la demodulación.

Otro problema identificado es la aparición de un pico artificial cerca del centro de la PSD. Este pico no necesariamente corresponde a una emisión real, sino que puede ser un artefacto de medición asociado al receptor. Para atenderlo se incluyó una etapa de postprocesamiento que analiza la región central de la PSD, estima la zona afectada y reconstruye localmente el resultado mediante interpolación lineal o ajuste polinomial.

También se contemplaron fallos de red asociados a la conectividad LTE. Por esta razón,

el orquestador debe manejar reintentos, estados internos y almacenamiento temporal de resultados cuando el envío al backend no puede realizarse de forma inmediata.

La desviación de frecuencia de la HackRF también fue considerada como un problema relevante. Debido a tolerancias del oscilador, temperatura o condiciones de operación, la frecuencia real de adquisición puede diferir de la frecuencia nominal solicitada. Para corregir este efecto se implementó una lógica de calibración en frecuencia basada en referencias presentes en emisiones FM comerciales.

Además, la operación en tiempo real exige que el sistema no bloquee la adquisición mientras procesa datos. Para esto se ajustó la arquitectura mediante buffers circulares, separación de tareas y paralelización con OpenMP dentro del motor RF.

En la operación real también deben considerarse errores de adquisición y de periféricos: HackRF no detectada, saturación del bus USB, overflow de buffers, frecuencia o sample rate inválidos, antena no disponible, falla de GPIO, pérdida de LTE, ausencia de fix GPS, timeout de ZeroMQ, caída del backend o temperatura alta de la Raspberry Pi. El software atiende parte de estas condiciones con estados, reintentos, logs, recreación de sockets y retorno a IDLE; otras condiciones requieren diagnóstico operativo o criterios de aceptación definidos por el despliegue.

5.7. Validación, pruebas o evidencias

La validación del software del sensor puede entenderse desde el flujo completo de funcionamiento. De forma resumida, el ciclo operativo del sistema es el siguiente:

1. El sensor se identifica ante la plataforma mediante su dirección MAC y su estado local.
2. El orquestador consulta si hay una tarea *realtime* o una campaña activa.
3. Si hay tarea, se construye una configuración de radio y se envía al motor RF.
4. El motor RF configura la HackRF, selecciona antena si aplica y captura IQ.
5. Las muestras pasan por preprocesamiento para mejorar su condición de medición.
6. El bloque de proceso calcula PSD, aplica filtros y, si se solicita, demodula audio.
7. El postproceso corrige artefactos finales, especialmente el pico DC en la PSD.
8. El orquestador empaqueta el resultado y lo envía al backend.
9. Si el envío falla, el dato puede quedar almacenado localmente para reintento.

10. Los servicios auxiliares mantienen GPS, LTE, estado del sensor y audio según el modo activo.

Este flujo permite validar que las principales etapas del sensor están conectadas: recepción de instrucciones, adquisición IQ, procesamiento, postprocesamiento, publicación de resultados y soporte de servicios auxiliares.

Como evidencias funcionales del sistema se consideran:

- La capacidad de recibir configuraciones desde el backend.
- La configuración efectiva de la HackRF One.
- La adquisición de muestras IQ.
- La generación de PSD mediante métodos como Welch o banco de filtros polifase.
- La activación de demodulación AM o FM cuando se solicita audio.
- La entrega de audio codificado mediante Opus y publicado por WebRTC.
- El reporte de estado del sensor, GPS, conectividad y variables de salud.
- La aplicación de corrección de frecuencia cuando existe una calibración disponible.
- El manejo de reintentos ante fallos de comunicación con el backend.

Para que la validación sea repetible, se propone registrar cada prueba con entrada, procedimiento, salida esperada y criterio de aceptación. La siguiente matriz resume pruebas mínimas para el software del sensor.

Prueba	Procedimiento	Criterio de aceptación
Conexión HackRF	Iniciar motor RF y enviar configuración PSD básica.	El motor responde JSON válido y no reporta dispositivo ausente.
PSD con señal conocida	Injectar o sintonizar una portadora conocida.	El pico aparece en la frecuencia esperada dentro de la tolerancia de calibración.
RBW/FFT	Ejecutar capturas con distinto RBW y mismo sample rate.	La separación de bins cambia según sample rate/FFT sin degradar parámetros solicitados.

Prueba	Procedimiento	Criterio de aceptación
Demodulación FM/AM	Solicitar audio con señal compatible.	Se activa WebRTC/Opus y se recibe audio sin bloquear la PSD.
GPS/LTE	Ejecutar reporte de estado en campo.	Se reporta conectividad y coordenadas cuando existe fix válido.
Campaña	Programar ventana de campaña y esperar ejecución.	Se genera medición en el intervalo esperado y se publica o encola.
Caída de red	Interrumpir LTE/backend durante envío.	El resultado queda en cola local y se reintenta posteriormente.
Calibración de frecuencia	Ejecutar calibración con emisora FM fuerte.	Se obtiene <code>ppm_error</code> razonable o se rechaza la calibración si no hay referencia.
Carga térmica	Ejecutar PSD de alta resolución por tiempo prolongado.	El sensor mantiene operación y reporta temperatura/estado para diagnóstico.

5.8. Trazabilidad frente a requisitos funcionales

La tabla siguiente relaciona funciones esperadas en un sistema de monitoreo espectral con el estado del software descrito en este capítulo. La columna de estado diferencia lo implementado en el sensor, lo parcialmente cubierto y lo que debe tratarse como recomendación o desarrollo futuro.

Funcionalidad	Estado	Comentario técnico
Frecuencia central	Implementado	Configurable desde backend y aplicada por HackRF con corrección <code>ppm_error</code> .
Sample rate/span hasta 20 MHz	Implementado	La captura instantánea depende de la tasa de muestreo útil de la HackRF.
RBW y FFT	Implementado	RBW derivada de sample rate y tamaño FFT/ <code>nperseg</code> .

Funcionalidad	Estado	Comentario técnico
Ventanas y Welch	Implementado	Soporta ventanas y solape para estabilizar PSD.
Banco polifase	Implementado / parcial	Disponible como método PSD alternativo con mayor costo de CPU.
VBW/suavizado	Parcial	Puede aproximarse por promediado; falta formalizar como parámetro metrológico separado.
Filtros digitales	Parcial	Implementado como rango pasabanda validado; falta caracterización formal LP/HP/PB.
Demodulación AM/FM	Implementado	Audio local con codificación Opus y publicación WebRTC.
GPS	Implementado	Se reportan coordenadas; se recomienda documentar fix, precisión y HDOP si están disponibles.
LTE/reintentos	Implementado	Envío HTTP y cola local ante fallos de red.
Unidades dBm/dB μ V/m	Pendiente	Requieren calibración de potencia, antena, pérdidas y curva de corrección.
Escala dBFS relativa	Implementado	Es la interpretación válida actual de los niveles logarítmicos del sensor.
Marcadores, OBW y potencia de canal	Parcial/futuro	Pueden calcularse sobre PSD; debe definirse si se ejecutan en sensor, backend o frontend.
Sweep mayor a 20 MHz	Futuro	Requiere barrido por ventanas, solape, tiempo de permanencia y unión espectral.
Almacenamiento IQ	Parcial/futuro	Debe formalizar formato .cs8/metadatos si se requiere evidencia forense.
NTP/timestamping	Parcial	Se usan tiempos del sistema; se recomienda documentar fuente, zona horaria y tolerancia.
Monitoreo del sensor	Implementado / parcial	Estado, temperatura, memoria, almacenamiento y conectividad reportados para diagnóstico.

Funcionalidad	Estado	Comentario técnico
Seguridad HTTPS/TLS	Implementado / parcial	La URL base usa HTTPS; falta documentar autenticación, certificados y manejo de credenciales.

5.9. Aprendizajes y transferencia de conocimiento

El desarrollo del software del sensor deja varios aprendizajes importantes para la continuidad del proyecto.

En primer lugar, se evidencia la conveniencia de separar las tareas de orquestación y las tareas de procesamiento intensivo. Python resulta adecuado para la integración con servicios, consultas HTTP, manejo de estados y coordinación general. C resulta más adecuado para las tareas críticas de adquisición, procesamiento DSP, manejo de buffers y demodulación.

En segundo lugar, el uso de buffers circulares es fundamental en sistemas de adquisición continua. En un sensor SDR, el hardware puede entregar muestras de forma sostenida y a alta velocidad, por lo que el software debe estar preparado para recibir datos sin bloquearse mientras ejecuta procesamiento por bloques.

En tercer lugar, se confirma la importancia del preprocesamiento y el postprocesamiento. La señal entregada por un SDR real puede contener artefactos como desbalance IQ, offset DC o picos centrales en la PSD. Por tanto, el software no solo debe adquirir y calcular espectros, sino también aplicar correcciones que mejoren la calidad de los productos entregados al backend.

Otro aprendizaje importante es que las funcionalidades de tiempo real dependen tanto del procesamiento local como de la comunicación con la plataforma. Aunque el motor RF procese los datos rápidamente, la experiencia de visualización también depende de la latencia de red, la publicación al backend y la actualización de la interfaz.

Finalmente, el uso de OpenMP muestra que es posible mejorar el aprovechamiento de los recursos de cómputo disponibles sin aumentar excesivamente la complejidad del código. Esta herramienta permite introducir paralelización en C de forma más sencilla que una implementación manual completa basada en hilos.

5.10. Limitaciones

El sistema presenta varias limitaciones que deben tenerse en cuenta para su operación y evolución futura.

Una primera limitación está asociada a la latencia de comunicación con el servidor. Aunque el sensor pueda adquirir y procesar señales en tiempo real, la tasa de actualización observada en la plataforma siempre estará limitada por los tiempos de envío, recepción, procesamiento en backend y visualización en frontend. Por tanto, el tiempo real del sistema no depende únicamente de la velocidad del motor RF, sino también de la infraestructura de comunicación y de la plataforma central.

Otra limitación corresponde al span configurable. El sistema trabaja con un límite de span de hasta 20 MHz, asociado al ancho de banda práctico de adquisición de la HackRF One y a la configuración implementada en el software. Actualmente no se incluye una funcionalidad de *sweep* que permita recorrer secuencialmente varias porciones del espectro para observar un ancho de banda mayor al que permite una sola captura directa del hardware.

También existe una limitación en la calibración en frecuencia. La estrategia implementada depende de detectar emisiones FM comerciales y, en particular, referencias como el tono piloto estéreo de 19 kHz. Por esta razón, en zonas aisladas o en lugares donde no existan emisoras FM suficientemente fuertes o claras, el sensor puede no contar con referencias adecuadas para ejecutar la calibración. Alternativas futuras incluyen una referencia RF externa, GPSDO, beacons conocidos, estaciones patrón, comparación contra analizador certificado o calibración periódica en laboratorio.

Otra limitación importante corresponde a la calibración en potencia. Las HackRF One usadas en el sistema no son instrumentos calibrados en potencia absoluta y en este proyecto no se realizó una calibración de amplitud de la cadena RF. Por esta razón, al calcular el logaritmo de la potencia de la PSD, el resultado no debe interpretarse como dBm ni como dB absolutos referenciados. La interpretación correcta actual es relativa al rango digital de la adquisición, es decir, dBFS o nivel relativo. No se conoce con certeza la tensión de saturación equivalente del ADC, los rangos internos de tensión, las atenuaciones de la HackRF, las pérdidas del cableado, el factor de antena ni la ganancia real del conjunto. Para obtener mediciones robustas de potencia absoluta, sería necesario realizar una calibración externa con señales de referencia, instrumentación adecuada y una curva de corrección que relacione los niveles medidos por el sensor con niveles reales de potencia.

Finalmente, al tratarse de un sistema remoto basado en conectividad LTE, el envío de resultados puede verse afectado por cobertura, disponibilidad de red, pérdida de paquetes o variaciones de latencia. Aunque el software contempla reintentos y almacenamiento local, la operación continua depende de la calidad de la conexión en el sitio de despliegue.

5.11. Recomendaciones específicas

A partir del análisis del software del sensor, se proponen las siguientes recomendaciones específicas:

- Mantener la separación entre orquestación en Python y procesamiento RF en C, ya que esta división permite conservar claridad arquitectónica y rendimiento en las tareas críticas.
- Documentar con mayor detalle las interfaces entre el orquestador y el motor RF, especialmente los mensajes ZeroMQ, los parámetros de configuración y los formatos de respuesta.
- Conservar el uso de buffers circulares y revisar periódicamente sus tamaños, tiempos de drenaje y condiciones de saturación para evitar pérdidas de muestras.
- Continuar usando OpenMP para paralelizar tareas de procesamiento en C, evaluando cuidadosamente qué secciones del código realmente se benefician de la paralelización.
- Implementar en el futuro una funcionalidad de *sweep* si se requiere observar un span mayor a 20 MHz. Esta funcionalidad debería coordinar capturas sucesivas en diferentes frecuencias centrales y reconstruir una vista espectral más amplia.
- Desarrollar una metodología de calibración en potencia si el sistema se va a usar para mediciones absolutas. Esta calibración debería usar un generador RF calibrado, niveles conocidos, varias frecuencias, varias combinaciones de ganancia, registro de pérdidas de cable, factor de antena, incertidumbre y una curva de corrección para convertir dBFS a unidades referenciadas como dBm.
- Complementar la calibración en frecuencia con otros métodos de referencia para escenarios donde no existan emisoras FM comerciales disponibles o donde el tono piloto de 19 kHz no pueda detectarse de manera confiable, por ejemplo GPSDO, señal patrón externa, beacons conocidos o comparación con un instrumento certificado.
- Medir y caracterizar la latencia extremo a extremo del sistema, desde la adquisición en el sensor hasta la visualización en la plataforma, para definir con precisión el límite real de actualización en tiempo real.
- Mantener registros de diagnóstico sobre corrección de pico DC, calibración de frecuencia, fallos de red y estado del dispositivo, con el fin de facilitar soporte técnico y depuración.
- Formalizar timestamping y georreferenciación por medición, incluyendo fuente de tiempo, sincronización NTP, zona horaria, formato ISO 8601, estado de fix GPS, número de satélites y precisión cuando estén disponibles.
- Definir responsabilidades para marcadores, OBW, potencia de canal, umbrales y exportación: qué se calcula en el sensor, qué se deriva en backend y qué se visualiza o ajusta en frontend.

- Evaluar periódicamente el desempeño térmico y de recursos de la Raspberry Pi 5, especialmente durante tareas de PSD de alta resolución, demodulación de audio y operación multihilo.

Capítulo 6

Plataforma web en la nube

6.1. Propósito del capítulo

Este capítulo documenta la plataforma web que constituye la capa de interfaz humana y de coordinación central del sistema de monitoreo espectral ANE-HX. La plataforma está compuesta por un frontend de aplicación de página única (SPA) y un backend de servicios que actúa como centro de orquestación entre las personas operadoras, los sensores RF en campo, la base de datos y los servicios de postprocesamiento.

El capítulo describe la arquitectura general, las funcionalidades ofrecidas a los usuarios, las interfaces de comunicación entre componentes, las decisiones de diseño que guiaron la implementación, los problemas encontrados durante el desarrollo, los aprendizajes obtenidos y las limitaciones actuales. El objetivo es proporcionar una comprensión completa de cómo la plataforma web hace posible el ciclo operativo completo: desde la autenticación de una persona usuaria hasta la generación de un reporte de cumplimiento normativo basado en mediciones de espectro capturadas en campo.

6.2. Contexto dentro del sistema general

La plataforma web ocupa la posición central en la arquitectura del sistema ANE-HX. Como se describe en el capítulo de diseño general (Capítulo 3), el sistema se organiza en tres dominios principales: borde (sensores RF y adquisición), nube (plataforma web, base de datos y postprocesamiento) y usuarios (personas operadoras y administradoras). La plataforma web constituye la totalidad del dominio de nube desde la perspectiva de coordinación y la totalidad del dominio de usuarios desde la perspectiva de interfaz.

La relación con los demás módulos del sistema es la siguiente:

- **Hardware y sensores RF (Capítulo 4):** la plataforma recibe los datos de espectro, estado, GPS y audio demodulado que los sensores capturan en campo. También entrega a los sensores las configuraciones de adquisición y las campañas programadas que deben ejecutar.
- **SDR y adquisición en borde (Capítulo 5):** el flujo de adquisición definido en el borde depende de los parámetros que la plataforma configura y transmite. La plataforma es la consumidora de los datos generados por la cadena de procesamiento SDR.
- **Análisis espectral y localización en la nube (Capítulo 7):** la plataforma invoca al servicio de postprocesamiento Python para el análisis de emisiones, detección de señales, cruce con licencias y evaluación de cumplimiento normativo. Los resultados del análisis se integran en los reportes que la plataforma entrega a las personas usuarias.
- **Protocolos de prueba (Capítulo 8):** la plataforma es el objeto de pruebas de integración extremo a extremo y pruebas de usabilidad. Los protocolos de prueba definidos en ese capítulo se aplican sobre la plataforma para verificar su correcto funcionamiento en conjunto con los demás módulos.
- **Integración general (Capítulo 9):** la plataforma es el componente integrador por excelencia. El capítulo de integración detalla cómo todos los módulos se conectan a través de la plataforma para formar el sistema completo.

En el flujo extremo a extremo del sistema, la plataforma web es el punto de entrada para las personas y el punto de convergencia para los datos. Sin la plataforma, los sensores podrían capturar datos pero no habría forma de configurarlos, visualizar sus mediciones, planificar campañas, detectar alertas ni generar reportes.

6.3. Desarrollo técnico

La plataforma web se implementa como dos aplicaciones independientes que se despliegan como contenedores Docker coordinados por un proxy inverso Nginx: un frontend de aplicación de página única (SPA) y un backend de servicios API.

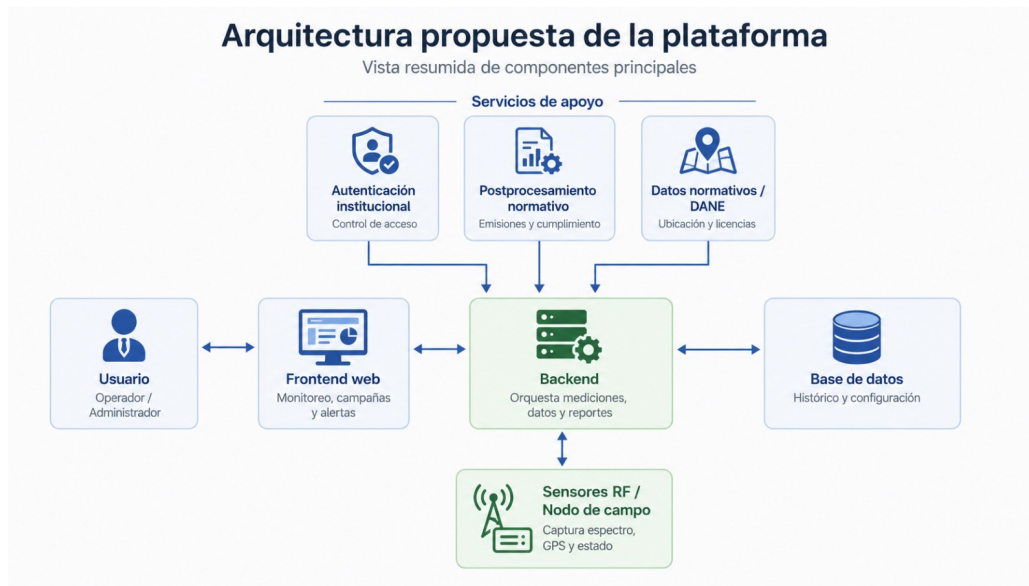


Figura 6.1: Diagrama general de la arquitectura de la plataforma web. Fuente: Elaboración propia.

6.3.1. Frontend

El frontend es una aplicación web de página única desarrollada con React 19 y TypeScript, construida con el empaquetador Vite. La interfaz sigue un diseño de panel de control con menú lateral de navegación que da acceso a siete pantallas principales.

Tecnologías

La pila tecnológica del frontend está conformada por:

- **React 19** como librería principal de componentes de interfaz, seleccionado por su modelo eficiente de actualización del DOM virtual y su capacidad para manejar datos en tiempo real sin re-renderizaciones innecesarias.
- **TypeScript** como lenguaje de desarrollo, proporcionando tipado estático que mejora la detección temprana de errores y documenta implícitamente las interfaces de datos entre componentes.
- **Vite 5** como empaquetador y servidor de desarrollo, seleccionado por su velocidad de compilación basada en ESM nativo y su configuración simplificada.
- **Tailwind CSS 3** como framework de estilos mediante utilidades atómicas, facilitando la iteración rápida sobre el diseño y la consistencia visual entre componentes.

- **Plotly.js** para la visualización interactiva de espectro de potencia y gráficas waterfall (espectrograma), seleccionada por su soporte nativo para gráficos científicos con miles de puntos de datos y actualización en tiempo real.
- **Leaflet** y **React-Leaflet** para la visualización de mapas geográficos con marcadores de sensores, permitiendo la ubicación espacial de la red de monitoreo.
- **Recharts** para gráficas estadísticas complementarias en el panel de inicio.
- **React Router 7** para el enrutamiento del lado del cliente, permitiendo navegación entre pantallas sin recarga completa de la página.
- **Microsoft Authentication Library (MSAL)** para la integración con Azure AD como proveedor de identidad institucional.
- **Axios** como cliente HTTP para la comunicación con el backend.

Pantallas principales

La interfaz se organiza en las siguientes pantallas, accesibles desde un menú lateral cuya visibilidad se adapta al rol de la persona usuaria:

Inicio (Dashboard): pantalla de bienvenida que presenta un mapa geográfico con la ubicación de todos los sensores registrados, indicadores de estado por colores, y un resumen de estadísticas del sistema (sensores activos, campañas en curso, alertas recientes). Es el punto de partida para la navegación.

Dispositivos: vista de gestión de la red de sensores. Muestra el listado completo de sensores con su estado actual, ubicación y antenas asociadas. Incluye un mapa interactivo con marcadores de estado y acceso al detalle de cada sensor. Desde esta pantalla se puede iniciar un monitoreo en vivo sobre un sensor seleccionado.

Monitoreo en vivo: flujo central de la plataforma para realizar mediciones inmediatas. La persona selecciona un sensor y una antena, define los parámetros de adquisición (rango de frecuencia, ancho de banda, resolución, ganancias) e inicia la captura. Durante la medición se visualiza en tiempo real el espectro de potencia (PSD) mediante Plotly.js, la evolución temporal en waterfall, y el audio demodulado AM/FM cuando está habilitado. La persona puede detener la medición o guardar la configuración como una nueva campaña.

Campañas: gestión de mediciones programadas. Permite crear campañas definiendo nombre, fechas de inicio y fin, horas de operación, intervalo entre mediciones, parámetros espectrales y sensores asignados. Incluye listado con filtros por estado, visualización de detalle, inicio y detención manual, y consulta de datos capturados. Los datos de campaña son la fuente para la generación de reportes de cumplimiento.

Alertas: revisión de incidentes y eventos operativos detectados por el sistema. Presenta un listado cronológico con filtros por sensor, tipo de alerta y rango de fechas. Cada alerta se vincula con los datos del sensor para facilitar el diagnóstico.

Configuración: panel administrativo para la gestión de recursos. Incluye CRUD del catálogo de antenas (tipo, rango de frecuencia, ganancia, código de inventario), administración de cuentas de usuario (roles y permisos), y configuración de parámetros globales del sistema.

Audio: reproductor de audio demodulado AM/FM, accesible desde el monitoreo en vivo o desde la vista de un sensor. Muestra métricas de modulación en tiempo real: profundidad para AM, excursión para FM.

Mecanismo de actualización en tiempo real

El frontend mantiene una conexión WebSocket con el backend para recibir actualizaciones sin necesidad de recarga manual. Los eventos recibidos incluyen cambios de estado de sensores, nuevas ubicaciones GPS, datos de espectro en tiempo real y confirmaciones de comandos. Estos eventos actualizan automáticamente los mapas, gráficas e indicadores en todas las pantallas relevantes, implementando un modelo de datos reactivo donde la interfaz refleja el estado del sistema en todo momento.

6.3.2. Backend

El backend es el centro de coordinación de la plataforma, implementado como un servidor Node.js con TypeScript sobre el framework Express. Su función principal es conectar a las personas usuarias (a través del frontend), los sensores RF en campo, la base de datos PostgreSQL y los servicios de postprocesamiento.

Tecnologías

- **Node.js 18+** como entorno de ejecución, seleccionado por su modelo de E/S no bloqueante que permite manejar múltiples conexiones concurrentes (frontend, sensores, WebSocket) en un solo proceso.
- **TypeScript** como lenguaje, compilado a JavaScript para producción, proporcionando verificación de tipos en tiempo de compilación.
- **Express** como framework HTTP, ofreciendo un modelo maduro de middleware, enrutamiento y manejo de solicitudes.

- **WebSocket nativo** (librería ws) para comunicación en tiempo real con el frontend, seleccionado sobre alternativas como Socket.IO por su menor sobrecarga de protocolo.
- **PostgreSQL 13+** con cliente pg y pool de conexiones, como sistema de base de datos relacional.
- **TimescaleDB** como extensión opcional de PostgreSQL para optimización de datos de series temporales.
- **JSON Web Tokens (JWT)** y librería jose para la gestión de sesiones y autenticación.
- **Swagger/OpenAPI** para la documentación interactiva de la API.

Capas de la arquitectura

El backend está organizado en seis capas lógicas con responsabilidades bien definidas:

Capa de entrada HTTP: expone el servidor en el puerto configurado (3000) y aplica middleware común: CORS para permitir solicitudes del frontend, registro (logging) de cada solicitud con método y ruta, y parsing de JSON con un límite ampliado de 50 MB para permitir la recepción de tramas espectrales de alta resolución enviadas por los sensores.

Capa de autenticación y autorización: valida la identidad de la persona usuaria mediante dos mecanismos complementarios. Para el acceso institucional desde el frontend, se integra con Azure AD utilizando el protocolo OpenID Connect y la librería jose para validación de tokens. Para el acceso administrativo local, se utilizan credenciales almacenadas en la base de datos con hash bcrypt y tokens JWT firmados por el servidor. Los middleware de autenticación se aplican de forma selectiva según la ruta: los endpoints de ingesta de datos desde sensores operan con validación por dirección MAC; los endpoints de administración requieren sesión validada con rol apropiado.

Capa de rutas y controladores: organiza la lógica de negocio en módulos independientes: autenticación, gestión de sensores y antenas (CRUD, asignación), recepción de datos de campo (estado, GPS, espectro, audio), control remoto de sensores, campañas de medición, reportes de cumplimiento y configuración del sistema.

Capa de modelos y acceso a datos: abstrae las consultas SQL mediante funciones tipadas que utilizan un pool de conexiones pg.Pool con hasta 20 conexiones simultáneas. Las operaciones que modifican múltiples tablas se ejecutan dentro de transacciones PostgreSQL para garantizar consistencia.

Capa de comunicación en tiempo real: mantiene un servidor WebSocket que transmite cinco tipos de eventos al frontend: **sensor_status** (estados de sensores), **sensor_gps** (ubicaciones), **sensor_data** (espectro en tiempo real), **sensor_configure** y **sensor_stop**

(comandos). Un segundo servidor WebSocket independiente gestiona el streaming de audio demodulado en formato Opus.

Capa de mantenimiento automático: ejecuta tareas programadas en intervalos regulares de 30 segundos. La tarea principal valida la antigüedad del último reporte de cada sensor activo y marca como offline aquellos que exceden el umbral de inactividad. Los cambios de estado se notifican al frontend mediante broadcast WebSocket.

API REST

La interfaz de programación del backend expone aproximadamente treinta endpoints organizados en ocho categorías funcionales, todas documentadas mediante especificación OpenAPI y accesibles a través de Swagger UI:

1. **Sensor Data:** endpoints POST para que los sensores envíen estado operativo, ubicación GPS, datos de espectro radioeléctrico y audio demodulado. Son los endpoints de mayor tráfico durante la operación normal.
2. **Sensor Query:** endpoints GET para consulta de datos históricos: último estado, última ubicación, últimos datos de espectro, datos por rango temporal y configuración activa, identificando cada sensor por su dirección MAC.
3. **Sensor Control:** endpoints POST para enviar comandos a los sensores: configuración de parámetros de escaneo, comando de detención y almacenamiento de configuración activa.
4. **Sensors Management:** CRUD completo de sensores más consultas por ID o MAC, y gestión de la relación sensor-antena (asignar, desasignar, consultar).
5. **Antennas Management:** CRUD completo del catálogo de antenas.
6. **Campaigns:** gestión de campañas con operaciones de creación, consulta, actualización, eliminación, inicio, detención y consulta de datos capturados.
7. **Reports:** generación de reportes de cumplimiento normativo que integran datos de campaña, ubicación y análisis del servicio de postprocesamiento.
8. **System:** endpoint raíz con versión de la API y directorio de endpoints.

6.3.3. Base de datos

La plataforma utiliza PostgreSQL como sistema de gestión de base de datos relacional. El esquema está organizado en cuatro dominios funcionales que agrupan quince tablas:

Dominio de identidad y acceso: almacena credenciales y roles de las personas usuarias (tabla `users`, con hash bcrypt para contraseñas) y un registro de auditoría de acciones sensibles (tabla `audit_logs`).

Dominio de sensores y configuración: mantiene el registro maestro de sensores RF identificados por MAC única (tabla `sensors`), el catálogo de antenas con sus parámetros técnicos (`antennas`), la relación muchos-a-muchos entre sensores y antenas con puerto de conexión (`sensor_antennas`), y los parámetros de adquisición por sensor: frecuencias, sample rate, ganancias, ventana, solapamiento y configuración de demodulación (`sensor_configurations`).

Dominio de datos de campo: almacena las series temporales generadas por los sensores: estado operativo con métricas de CPU, RAM, temperatura y latencia (`sensor_status`), ubicaciones GPS (`sensor_gps`), datos de espectro con arreglos de potencia, métricas de demodulación AM/FM y asociación opcional a campaña (`sensor_data`), y alertas operativas históricas (`sensor_history_alert`).

Dominio de campañas y reportes: define las campañas de medición con parámetros de programación y configuración espectral (`campaigns`), la asignación de sensores a campañas (`campaign_sensors`), el caché de reportes de cumplimiento en formato JSON (`compliance_reports_cache`), y parámetros globales del sistema en formato clave-valor (`system_configurations`).

La tabla `sensor_status`, que recibe actualizaciones periódicas de cada sensor y es la de mayor crecimiento, está configurada como hypertable de TimescaleDB, particionada por la columna `timestamp_ms`. Esta configuración permite compresión y retención automática de datos, y optimiza consultas temporales mediante índices compuestos por MAC y timestamp.

6.3.4. Despliegue

La plataforma se despliega en un servidor institucional privado de la ANE, accesible a través de la red interna de la entidad. La arquitectura de despliegue utiliza contenedores Docker coordinados por un proxy inverso Nginx:

- **Frontend:** imagen Docker construida en dos etapas: compilación con Node.js Alpine y servidor web con Nginx Alpine. Expone el puerto 80 con configuración para aplicaciones de página única (SPA) y compresión gzip.
- **Backend:** imagen Docker construida en dos etapas: compilación de TypeScript y ejecución con Node.js Alpine. Expone el puerto 3000.
- **Nginx:** actúa como punto único de entrada, enrutando el tráfico HTTP al frontend (archivos estáticos) y al backend (API REST en `/api/`, WebSocket en `/ws/`). La

configuración incluye timeouts extendidos de una hora para la generación de reportes de larga duración.

- **PostgreSQL**: base de datos con extensión TimescaleDB, ejecutándose en el mismo servidor. En producción se recomienda la imagen `timescale/timescaledb:latest-pg14`.

La configuración de entorno se gestiona mediante variables de entorno (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `PORT`) que permiten desplegar la misma imagen en entornos de desarrollo, pruebas y producción sin recompilación.

6.4. Entradas, salidas e interfaces

6.4.1. Entradas

La plataforma recibe datos de tres fuentes principales:

Desde el frontend: solicitudes HTTP de las personas usuarias para autenticarse, consultar información, configurar mediciones, gestionar campañas, revisar alertas y generar reportes. Estas solicitudes incluyen tokens de autenticación (JWT o Azure AD) y parámetros específicos según la operación.

Desde los sensores RF: datos de campo en formato JSON enviados mediante solicitudes POST:

- Estado del sensor: métricas de CPU por núcleo, RAM, disco, temperatura, latencia de red y logs del sistema operativo del dispositivo.
- Ubicación GPS: latitud, longitud y altitud.
- Datos de espectro: arreglo de potencias PSD (P_{xx}), frecuencia inicial y final en Hz, marca de tiempo en epoch milliseconds, coordenadas de captura, y métricas de demodulación AM (profundidad) o FM (excursión).
- Audio demodulado: tramas de audio PCM codificadas en base64 o en formato Opus.

Desde el servicio de postprocesamiento: resultados de análisis de emisiones en formato JSON, con frecuencias centrales detectadas, anchos de banda, potencias, cruce con licencias y evaluación de cumplimiento por municipio (código DANE).

6.4.2. Salidas

La plataforma entrega:

Al frontend: respuestas JSON a todas las consultas y operaciones, eventos WebSocket para actualización en tiempo real de estado de sensores, GPS, datos de espectro y audio.

A los sensores RF: configuraciones de adquisición (frecuencias, sample rate, ganancias, ventana, solapamiento, demodulación) y listas de campañas asignadas con todos los parámetros necesarios para ejecutar mediciones programadas.

A las personas usuarias: reportes de cumplimiento normativo estructurados con emisiones detectadas, comparación contra licencias, evaluación por parámetro (frecuencia central, ancho de banda, potencia) y estimación de intensidad de campo.

6.4.3. Interfaces

Las interfaces de comunicación entre componentes son:

- **Frontend – Backend:** API REST sobre HTTP con formato JSON. Autenticación mediante tokens JWT en cabecera **Authorization**. WebSocket para eventos en tiempo real y streaming de audio.
- **Sensores – Backend:** API REST sobre HTTP con formato JSON. Identificación del sensor por dirección MAC. Los sensores actúan como clientes HTTP que envían datos (POST) y consultan configuración (GET).
- **Backend – PostgreSQL:** conexión TCP nativa mediante el cliente pg sobre el puerto 5432, utilizando un pool de conexiones configurable.
- **Backend – Postprocesamiento:** invocación HTTP síncrona al servicio Python Flask expuesto en el puerto 8000, con formato JSON para solicitudes y respuestas.
- **Nginx – Frontend:** entrega de archivos estáticos (HTML, JS, CSS, assets) por HTTP en puerto 80.
- **Nginx – Backend:** proxy inverso HTTP y WebSocket, con timeouts extendidos para operaciones de larga duración.

6.5. Decisiones de diseño o implementación

6.5.1. Separación frontend/backend con API REST

Se decidió separar el frontend y el backend como aplicaciones independientes comunicadas por API REST, en lugar de una aplicación monolítica con renderizado en el servidor. Esta decisión se fundamenta en tres consideraciones:

1. **Despliegue independiente:** el frontend (archivos estáticos) y el backend (servidor Node.js) pueden escalarse, actualizarse y desplegarse de forma independiente. El frontend se sirve desde Nginx como archivos estáticos con caché agresiva; el backend maneja la lógica de negocio y la concurrencia.
2. **Separación de responsabilidades:** el equipo de frontend puede iterar sobre la interfaz sin afectar la lógica del backend, y viceversa. La API actúa como contrato estable entre ambas capas.
3. **Cliente único vs. múltiples clientes:** aunque actualmente el frontend web es el único cliente, la arquitectura REST permite que en el futuro otros clientes (aplicaciones móviles, scripts de automatización, dashboards externos) consuman la misma API sin modificar el backend.

6.5.2. WebSocket sobre polling para tiempo real

La elección de WebSocket como mecanismo de comunicación en tiempo real, en lugar de polling HTTP periódico, se justifica por el perfil de tráfico del sistema:

- Durante un monitoreo activo, un sensor puede enviar datos de espectro cada pocos segundos. Con polling, el frontend tendría que consultar al backend repetidamente para detectar nuevos datos, generando tráfico innecesario y latencia.
- WebSocket permite que el backend notifique proactivamente al frontend solo cuando hay datos nuevos, en un modelo push eficiente.
- La librería ws (WebSocket nativo) fue seleccionada sobre Socket.IO por su menor sobrecarga de protocolo: no requiere el handshake adicional de Socket.IO ni sus mecanismos de fallback a long-polling, que son innecesarios en un entorno de red controlada.

6.5.3. PostgreSQL y TimescaleDB para datos de series temporales

La decisión de adoptar PostgreSQL como base de datos principal —y específicamente TimescaleDB como extensión para tablas de series temporales— fue la de mayor impacto en la arquitectura del backend. El sistema comenzó utilizando SQLite, una base de datos sin servidor que almacena toda la información en un solo archivo.

SQLite ofrecía ventajas iniciales de simplicidad: sin necesidad de instalar ni configurar un servidor de base de datos separado, sin autenticación, con despliegue trivial. Sin embargo, a medida que el volumen de datos creció —particularmente en las tablas de estado de sensores y datos de espectro— se manifestaron limitaciones críticas:

- El límite práctico de aproximadamente 2 GB por archivo de base de datos comenzó a causar degradación y bloqueos.
- SQLite no soporta concurrencia real de escritura: múltiples sensores enviando datos simultáneamente generaban contención de bloqueo.
- Las consultas analíticas sobre grandes volúmenes de datos históricos (necesarias para reportes) tenían rendimiento deficiente.

La migración a PostgreSQL resolvió estas limitaciones y habilitó capacidades nuevas: tipos de datos nativos para valores geospaciales y series temporales, transacciones ACID completas en operaciones multi-tabla, índices compuestos para consultas por sensor y rango temporal, y compresión y retención automática de datos antiguos mediante TimescaleDB. La migración se realizó de forma progresiva, actualizando primero la capa de conexión, luego los modelos de datos y finalmente las rutas con consultas directas.

6.5.4. Microservicio de postprocesamiento en Python

El análisis espectral avanzado —detección de emisiones, estimación de piso de ruido por escalones, separación de regiones por valles, cruce con base de datos de licencias— se implementó como un servicio independiente en Python (Flask), separado del backend Node.js.

Esta decisión responde a la naturaleza del procesamiento requerido: el análisis involucra cálculos numéricos intensivos (FFT, integración de potencia, suavizado Savitzky-Golay, detección de picos, evaluación de cumplimiento) para los cuales el ecosistema Python ofrece librerías maduras y optimizadas (NumPy, SciPy, Pandas). Implementar estas mismas rutinas en Node.js habría requerido reescribir algoritmos complejos o depender de puentes a librerías nativas con mantenimiento incierto.

La separación como microservicio también permite escalar el postprocesamiento de forma independiente, asignar más recursos computacionales al servicio Python sin afectar la capacidad de respuesta del backend, y mantener ciclos de desarrollo separados para la lógica de análisis espectral.

6.5.5. Separación de canales WebSocket para datos y audio

El audio demodulado se transmite por un canal WebSocket independiente del canal de datos de sensores. Esta separación permite tratar cada flujo según sus requisitos específicos:

- El canal de datos transmite eventos discretos (estado, GPS, espectro) con frecuencia de segundos y payloads de tamaño moderado.

- El canal de audio transmite tramas continuas en formato Opus con frecuencia de milisegundos, requiriendo menor latencia y mayor throughput.
- La separación evita que la carga de streaming de audio afecte la entrega de datos de espectro durante un monitoreo, y viceversa.

6.6. Problemas presentados y ajustes realizados

A continuación se documentan los problemas identificados durante el desarrollo y operación de la plataforma, las evidencias recolectadas, el análisis de causas, las decisiones de ajuste tomadas y los resultados obtenidos.

Tabla 6.1: Problemas presentados, decisiones y ajustes realizados en la plataforma web.

Problema	Evidencia	Causa probable	Decisión o Resultado
Límite de capacidad de SQLite causaba bloqueos en producción al alcanzar 2 GB de datos acumulados	Logs de error del servidor, mediciones de tamaño de archivo de base de datos	SQLite no está diseñado para volúmenes de series temporales con múltiples escrituras concurrentes desde sensores	Migración completa a PostgreSQL con extensión TimescaleDB. Se reescribió la capa de conexión, modelos de datos y rutas con consultas directas manteniendo compatibilidad de formatos
Endpoint de campañas para sensores con nombre incorrecto y resultados duplicados en asignaciones múltiples	Logs de consulta de sensores físicos, pruebas de integración	Falta de deduplicación en consulta SQL de asignación sensor-campaña y nomenclatura inconsistente del endpoint	Corrección del nombre del endpoint, incorporación de lógica de deduplicación, adición de filtro opcional por estado y validación de campos requeridos
			Sensores reciben lista única de campañas asignadas con parámetros completos para ejecutar mediciones

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Timeouts en generación de reportes de cumplimiento para campañas con grandes volúmenes de datos	Errores HTTP 504 en frontend, logs de timeout en Nginx	Timeouts por defecto (30–60 s) insuficientes para el pipeline completo de consulta, post-procesamiento y armado de respuesta	Configuración de timeouts extendidos a 3600 s en servidor Node.js y proxy Nginx (<code>proxy_read_timeout</code> , <code>proxy_connect_timeout</code> , <code>proxy_send_timeout</code>)	Reportes generados exitosamente incluso para campañas con grandes volúmenes de datos acumulados
Sensores que dejaban de reportar permanecían con estado activo en la plataforma	Reportes de personas usuarias, inspección de estados inconsistentes en base de datos	Ausencia de mecanismo automático de verificación de frescura de datos de sensores	Implementación de tarea programada cada 30 s que marca como offline sensores sin reporte reciente y notifica cambios al frontend por WebSocket	El estado de sensores refleja fielmente su condición operativa real, eliminando falsos positivos
Carga inicial lenta del frontend por inclusión de Plotly.js en el bundle principal	Métricas de Lighthouse, reportes de personas usuarias sobre tiempo de carga	Plotly.js (3 MB) se incluía en el bundle principal, bloqueando renderización	Separación de Plotly.js en chunk independiente mediante code splitting en Vite; pantallas sin gráficos no esperan su carga	Mejora significativa en tiempo de carga inicial y puntuación de rendimiento sin afectar pantallas de monitoreo
Errores de Mixed Content en producción con HTTPS: conexiones WebSocket y API bloqueadas por navegador	Errores en consola del navegador, fallos en conexión WebSocket, datos en tiempo real sin actualizar	URLs del backend configuradas como absolutas (HTTP) desde página servida por HTTPS	Migración a URLs relativas (<code>/api</code> , <code>/ws</code>) en variables de entorno de compilación, delegando resolución al proxy Nginx	Eliminación de errores de Mixed Content, funcionamiento correcto en todos los entornos

Problema	Evidencia	Causa probable	pro-ajuste	Decisión o Resultado
Inconsistencia de sesión entre múltiples pestañas del navegador	Reportes de personas usuarias, pruebas con múltiples pestañas abiertas	Estado de autenticación y WebSocket gestionado por pestaña sin coordinación	Revisión del flujo de MSAL para propagar eventos de login/logout entre pestañas y manejo de renovación de tokens	Comportamiento consistente de sesión independientemente del número de pestañas abiertas

6.7. Validación, pruebas o evidencias

La plataforma web ha sido validada mediante múltiples estrategias de verificación a lo largo de su desarrollo y operación:

Pruebas funcionales manuales: cada pantalla del frontend y cada endpoint de la API han sido verificados manualmente durante el desarrollo, cubriendo los flujos principales de autenticación, monitoreo en vivo, creación de campañas, revisión de alertas y generación de reportes.

Pruebas de integración con sensores físicos: se ha verificado la comunicación extremo a extremo con sensores HackRF reales, incluyendo el envío de configuraciones, la recepción de datos de espectro y estado, y la transmisión de audio demodulado AM/FM. El simulador de sensores AM/FM documentado en el backend permite reproducir escenarios de prueba sin hardware físico.

Documentación interactiva de API: la especificación OpenAPI publicada a través de Swagger UI permite verificar la estructura de cada endpoint, los esquemas de solicitud y respuesta, y ejecutar pruebas directamente desde el navegador mediante la función “Try it out”.

Validación en producción: la plataforma se encuentra en operación en el servidor institucional de la ANE, accesible a través de la red interna. El uso continuo por parte del personal técnico constituye una validación operativa de las funcionalidades implementadas.

Logs de servidor: el backend registra cada solicitud HTTP con método, ruta, marca de tiempo y código de respuesta, permitiendo auditoría de la actividad y diagnóstico de problemas. Los logs de Nginx complementan esta trazabilidad con información de tiempos de respuesta y estado de las conexiones.

Mantenimiento automático verificado: la tarea programada de validación de estado de sensores se monitorea a través de los eventos WebSocket que notifican cambios de estado al frontend. Los sensores que dejan de reportar son marcados como offline de forma automática, y este comportamiento ha sido confirmado en operación.

6.8. Aprendizajes y transferencia de conocimiento

Los siguientes aprendizajes se derivan de la experiencia de diseño, implementación y operación de la plataforma web:

1. **Dimensionar la persistencia desde el inicio según la carga esperada:** SQLite fue adecuado para desarrollo y prototipado, pero mantener una base de datos sin servidor para una carga de producción con múltiples escritores concurrentes y grandes volúmenes de series temporales generó un cuello de botella que requirió una migración significativa. La enseñanza es que la elección de la base de datos debe basarse en el perfil de carga previsto —volumen, concurrencia, tipo de consultas— y no solo en la conveniencia de desarrollo.
2. **La comunicación en tiempo real debe ser push, no pull:** el uso de WebSocket para notificar al frontend sobre nuevos datos y cambios de estado evitó la necesidad de implementar lógica de polling, reduciendo la carga en el servidor y la latencia de visualización. Para sistemas donde los datos fluyen de forma continua desde múltiples fuentes, el patrón push es la estrategia correcta.
3. **Los timeouts deben calibrarse según la operación más costosa:** los valores por defecto de los servidores HTTP (30 segundos) no contemplan operaciones de análisis que involucran consultas complejas, procesamiento numérico y cruce con bases de datos externas. Identificar la operación de mayor duración —los reportes de cumplimiento— y configurar todos los timeouts de la cadena (servidor, proxy) de forma consistente evita fallos intermitentes difíciles de diagnosticar.
4. **La separación de canales por tipo de dato simplifica el manejo de calidades de servicio:** mantener datos de espectro y audio en canales WebSocket independientes permitió tratar cada flujo con sus propios requisitos de latencia y frecuencia, sin que la carga de streaming de audio afectara la entrega de datos de espectro.
5. **Las URLs relativas eliminan problemas de configuración por entorno:** migrar de URLs absolutas a relativas para API y WebSocket resolvió permanentemente los errores de Mixed Content y permitió que una misma compilación funcione en desarrollo, VPN y producción sin reconfiguración.

6. **La división de código es necesaria en aplicaciones con dependencias de visualización pesadas:** las librerías de gráficos científicos pueden dominar el tamaño del bundle. Planificar code splitting desde el inicio evita resolver problemas de rendimiento cuando la aplicación ya está en producción.
7. **El mantenimiento automático del estado elimina falsos positivos:** sin verificación periódica de la frescura de datos de sensores, los equipos desconectados aparecían como operativos. La automatización de esta verificación, con notificación por WebSocket, eliminó este problema y redujo la carga de monitoreo manual del personal.
8. **La documentación interactiva de API acelera la integración:** contar con Swagger UI permitió que el equipo de frontend y los desarrolladores de los sensores pudieran explorar y probar la API sin depender de documentación estática, reduciendo el tiempo de integración.

6.9. Limitaciones

La plataforma web en su estado actual presenta las siguientes limitaciones:

1. **Base de datos sin alta disponibilidad:** PostgreSQL opera en instancia única, sin replicación ni mecanismo de failover automático. Una falla del servidor de base de datos interrumpiría la totalidad de la plataforma.
2. **Escalamiento vertical del backend:** el backend está diseñado como un solo proceso Node.js. Si bien el pool de 20 conexiones soporta la concurrencia actual, la arquitectura no contempla escalamiento horizontal a múltiples instancias, lo que requeriría un backend compartido para WebSocket (como Redis) y balanceo de carga.
3. **Sin balanceo de carga:** el punto único de entrada a través de Nginx no está configurado para distribuir tráfico entre múltiples réplicas del backend.
4. **Dependencia de conectividad de sensores:** el modelo de operación asume que los sensores consultan periódicamente su configuración. Si un sensor pierde conectividad en el momento de consulta, puede no recibir una campaña programada hasta el siguiente ciclo.
5. **Postprocesamiento síncrono sin cola:** la generación de reportes invoca al servicio Python de forma síncrona. Si el servicio no está disponible, el reporte falla sin mecanismo de reintento automático o encolamiento.
6. **Tablas de series temporales sin optimización completa:** solo `sensor_status` está configurada como hypertable de TimescaleDB. Las tablas `sensor_data` y `sensor_gps`,

que también almacenan series temporales, pueden presentar degradación al crecer en volumen.

7. **Tamaño de dependencias de visualización:** Plotly.js (3 MB) sigue siendo una dependencia significativa que afecta la primera visita a las pantallas de monitoreo, incluso estando en un chunk separado.
8. **Sin pruebas automatizadas:** el desarrollo no incluye pruebas unitarias ni de integración automatizadas, dependiendo de validación manual para verificar cambios.
9. **Sin modo offline:** la aplicación requiere conectividad constante con el backend. No hay almacenamiento local de datos para consulta sin conexión.
10. **Sin diseño responsivo para dispositivos móviles:** la interfaz está optimizada para estaciones de escritorio, que es el contexto de uso principal. No se ha priorizado la adaptación a pantallas pequeñas.

6.10. Recomendaciones específicas

Con base en las limitaciones identificadas y la experiencia acumulada, se proponen las siguientes recomendaciones para la evolución de la plataforma:

1. **Alta disponibilidad de base de datos:** configurar replicación streaming en PostgreSQL con al menos una réplica en caliente, permitiendo continuidad operativa ante falla del servidor principal.
2. **Escalamiento horizontal del backend:** evaluar la introducción de Redis como backend compartido para WebSocket y como caché de datos frecuentes, habilitando múltiples instancias del servidor Node.js coordinadas detrás de un balanceador de carga.
3. **Optimización de todas las tablas de series temporales:** migrar `sensor_data` y `sensor_gps` a hypertables de TimescaleDB, y configurar políticas de compresión y retención automática para todas las tablas de series temporales.
4. **Resiliencia del postprocesamiento:** implementar una cola de trabajos (por ejemplo, Bull con Redis) para las solicitudes de reportes, con reintentos automáticos con backoff exponencial cuando el servicio Python no esté disponible.
5. **Pruebas automatizadas:** introducir pruebas unitarias con Vitest/Jest para el frontend y backend, y pruebas de integración para los flujos críticos (autenticación, monitoreo en vivo, creación de campañas, generación de reportes).

6. **Evaluación de alternativas de visualización:** analizar la viabilidad de reemplazar Plotly.js por ECharts o una implementación basada en Canvas/WebGL para reducir el tamaño del bundle sin perder capacidades de visualización.
7. **Monitoreo operacional:** instrumentar el backend con métricas de Node.js (consumo de memoria, event loop lag, latencia de respuesta, conexiones activas) y exportarlas a un sistema de monitoreo institucional.
8. **Internacionalización preparatoria:** si bien el contexto actual es exclusivamente colombiano, estructurar los textos de la interfaz de forma que una futura traducción no requiera reescribir componentes, utilizando una capa de localización (i18n).
9. **Revisión de seguridad:** auditar los endpoints de administración para verificar que todos requieran autenticación y autorización por rol, y documentar explícitamente el modelo de seguridad (endpoints públicos de ingesta, endpoints autenticados de administración).
10. **Documentación operativa:** elaborar guías de operación para el personal técnico de la ANE, cubriendo procedimientos de despliegue, respaldo, recuperación y diagnóstico de problemas comunes.

6.11. Repositorios

El software de la plataforma web está organizado en dos módulos dentro del repositorio del sistema. A continuación se presentan las fichas técnicas de cada componente.

Tabla 6.2: Repositorio asociado al frontend de la plataforma web.

Nombre del software	ANE-HX Frontend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Interfaz web para la operación y administración del sistema de monitoreo espectral
Entradas	Interacciones de la persona usuaria, datos en tiempo real por WebSocket, respuestas de API REST
Salidas	Visualizaciones de espectro y mapas, formularios de configuración, reportes de cumplimiento, streaming de audio
Dependencias	React 19, TypeScript, Vite 5, Tailwind CSS 3, Plotly.js, Leaflet, MSAL, Axios, Nginx
Estado actual	En producción en servidor institucional privado

Tabla 6.3: Repositorio asociado al backend de la plataforma web.

Nombre del software	ANE-HX Backend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Centro de coordinación: API REST, WebSocket, persistencia, gestión de campañas, alertas y reportes
Entradas	Solicitudes HTTP del frontend y sensores; datos de espectro, GPS, estado y audio desde sensores RF
Salidas	Respuestas JSON, eventos WebSocket, reportes de cumplimiento normativo
Dependencias	Node.js 18+, Express, TypeScript, PostgreSQL 13+, TimescaleDB (opcional), ws, Swagger/OpenAPI, Azure AD, servicio Python de postprocesamiento
Estado actual	En producción en servidor institucional privado

Análisis Espectral en la Nube y Localización Administrativa de Emisiones

Módulo de Postprocesamiento Espectral — Sistema de Monitoreo ANE

NOTA TÉCNICA: Este capítulo documenta el módulo de análisis espectral en la nube. El término “localización” en este contexto se refiere exclusivamente a localización administrativa (asociación DANE/municipio), NO a localización radioeléctrica real (triangulación, TDOA, AOA o multilateración RSSI). La posición física del transmisor no se estima en este módulo.

Tabla de Variables y Símbolos

Símbolo / Variable	Nombre completo	Unidad	Descripción
F_c	Frecuencia central	Hz / MHz	Frecuencia representativa de la emisión detectada
BW	Ancho de banda	Hz / kHz	Extensión espectral de la emisión
OBW	Occupied Bandwidth (Ancho de Banda Ocupado)	Hz / kHz	Intervalo que contiene el 99 % de la potencia de la emisión
RBW	Resolution Bandwidth (Ancho de Banda de Resolución)	Hz	RBW = Sample Rate / FFT size; resolución espectral del receptor
P_{xx}	Power Spectral Density	dBm / dB	Vector de amplitudes/PSD de la captura espectral
SNR	Signal-to-Noise Ratio	dB	Relación señal a ruido estimada
MER	Modulation Error Ratio	dB	Indicador estimado de calidad de modulación (solo diagnóstico)
BER	Bit Error Rate	–	Indicador estimado de tasa de error de bit (solo diagnóstico)
DANE	Código DANE	–	Identificador numérico de municipio colombiano (DANE)
FC_MARGIN_MHz	Tolerancia de frecuencia central	MHz	Valor por defecto: 0.1 MHz (100 kHz); configurable con <code>--delta_fc_khz</code>
BW_MARGIN_kHz	Tolerancia de ancho de banda	kHz	Valor por defecto: 10 kHz; configurable con <code>--delta_bw_khz</code>
umbral_db	Umbral de detección	dB	dB sobre el piso de ruido para considerar candidato de emisión
power_dbm	Potencia estimada	dBm	Potencia integrada de la emisión; null si no es finita

6.12. Objetivo del Capítulo

Este capítulo documenta el módulo de postprocesamiento espectral desarrollado para analizar, en un entorno centralizado en la nube, las capturas generadas por el sistema de monitoreo SDR. El objetivo es explicar cómo el software recibe una traza espectral, organiza la información de frecuencia y amplitud, detecta emisiones candidatas y extrae

sus parámetros técnicos: frecuencia central (F_c), ancho de banda (BW / OBW), potencia (*power_dbm*), y opcionalmente indicadores de calidad (MER/BER estimado para TDT).

Adicionalmente, el módulo verifica el cumplimiento de emisiones detectadas contra una base de licencias y las asocia administrativamente a un municipio o código DANE.

6.13. Alcance y Exclusiones

6.13.1. Lo que implementa este módulo

- Localización administrativa: asociación de una emisión al municipio o código DANE del sensor que capturó la señal.
- Análisis espectral: detección, medición de F_c , BW, OBW, potencia, SNR.
- Comparación contra base de licencias CSV.
- Exportación de resultados en JSON.
- Operación vía CLI (`main.py`) y API REST (`server_flask.py`).

6.13.2. Lo que NO implementa este módulo

- Localización radioeléctrica real: no se realiza triangulación, TDOA, AOA ni multilateración RSSI. La posición física del transmisor NO se estima.
- Localización geográfica del sensor: no se usa GPS/geotag para estimar posición del emisor.
- Demodulación formal de señal: MER/BER se estiman desde traza espectral, no desde demodulación.
- Evaluación de vigencia de licencias: el código no filtra licencias vencidas en esta versión.

NOTA TÉCNICA: La distinción entre localización administrativa, localización del sensor y localización radioeléctrica real es fundamental para evitar conclusiones regulatorias incorrectas. Asociar una emisión a un municipio no prueba que el transmisor esté físicamente allí.

6.14. Relación con Requisitos ANE

Requisito ANE	Sección del capítulo	Estado
Detección de emisiones	Sección 8 — Análisis espectral	Implementado
Umbral de detección configurable	Sección 5 (<i>umbral_db</i>) / Sección 8	Implementado
Medición de frecuencia central	Sección 8.1	Implementado
Medición de ancho de banda (-x dB y OBW 99%)	Sección 8.2	Implementado
Potencia de canal	Sección 8.3	Implementado
Exportación de resultados JSON	Sección 6 / Sección 11	Implementado
Timestamp, geotag e identificador del sensor	Sección 13 — Trazabilidad	Parcial (timestamp implementado; geotag depende del backend)
Comparación contra bases de licencias	Sección 9	Implementado
DANE / municipio	Sección 10	Implementado
Reportes	Sección 13 / Sección 14	Parcial
Exactitud de frecuencia y amplitud	Sección 7 — RBW y resolución	Parcial (depende de RBW/hardware)
DANL y rango dinámico	Sección 7 — RBW y resolución	Pendiente (no caracterizado formalmente)
Evaluación de vigencia de licencias	Sección 9.3	Pendiente — Recomendado para versión futura
Localización radioeléctrica (TDOA/AOA)	N/A	Recomendado para versión futura

6.15. Arquitectura Funcional

El módulo sigue una cadena funcional desde el sensor SDR en el borde hasta la plataforma cloud:

Sensor SDR / Edge → Traza espectral (JSON) → API cloud (`server_flask.py`)
→ `payload_parser.py` → `SpectrumFrame` → Corrección/calibración → De-

tección de emisiones → Medición de parámetros (F_c , BW, P) → Comparación con licencias → Localización administrativa (DANE) → Base de datos / Reportes / Interfaz web

Archivo	Función	Capa
<code>main.py</code>	Ejecución desde consola; lee JSON o trama, interpreta argumentos	Edge / Local
<code>server_flask.py</code>	API REST Flask; endpoints POST /analyze y POST /analyze_batch	Cloud
<code>src/payload_parser.py</code>	Convierte JSON de entrada en SpectrumFrame interno; valida campos	Cloud
<code>src/spectrum_frame.py</code>	Estructura SpectrumFrame: amplitudes, frecuencias, resolución espectral (<code>bin_hz</code>)	Cloud
<code>src/processor.py</code>	Orquesta flujo principal; selecciona modo; aplica corrección; arma salida	Cloud
<code>src/spectral_analysis.py</code>	Detección de picos, segmentación, F_c , BW, OBW, potencia, MER/BER	Cloud
<code>src/calibration_io.py</code>	Lee CSV de licencias; normaliza municipios; interpola corrección por frecuencia	Cloud
<code>src/power_utils.py</code>	Integración de potencia en dominio lineal; fallback trapezoidal; manejo de NaN	Cloud

6.16. Entradas del Módulo

Entrada	Tipo	Requerido	Descripción
Pxx	Lista numérica (dBm/dB)	Sí	Vector de amplitudes/PSD de la captura espectral. Se trata como densidad espectral en función de frecuencia para cálculos de integración y OBW.
start_freq_hz	Numérico (Hz)	Sí	Frecuencia inicial de la captura
end_freq_hz	Numérico (Hz)	Sí	Frecuencia final de la captura
picos	Lista numérica (Hz)	No	Frecuencias específicas solicitadas; activa modo peaks
cumplimiento	Entero (0/1)	No	Activa modo compliance cuando no hay picos
corr	Ruta CSV	No	Archivo de corrección de ganancia por frecuencia
lic	Ruta CSV	No (req. para compliance)	Base de licencias para comparación normativa
dane / danes / municipio	Texto o lista	No (req. para compliance)	Filtro administrativo; requerido para evaluación de cumplimiento
umbral_db	Numérico (dB)	No	Umbral sobre piso de ruido; por defecto adaptativo
delta_fc_khz	Numérico (kHz)	No	Tolerancia F_c ; por defecto 100 kHz (FC_MARGIN_MHZ=0.1)
delta_bw_khz	Numérico (kHz)	No	Tolerancia BW; por defecto 10 kHz (BW_MARGIN_KHZ=10)

6.17. RBW, FFT y Resolución Espectral

6.17.1. Relación entre parámetros

La resolución espectral del sistema está determinada por la cadena de adquisición SDR. Los parámetros clave son:

- Sample rate (`sample_rate_hz`): por defecto 20 MHz (20 000 000 Hz); configurable por campaña.

- RBW (`rbw_hz`): se define en el backend como `campaign.resolution_khz * 1000` si está configurado; si no, valor por defecto de 10 000 Hz (10 kHz).
- FFT size: no se selecciona explícitamente en postprocesamiento. El módulo recibe `Pxx` ya calculado por el sensor. El número de bins es `len(Pxx)` y la resolución de bin se calcula como: `bin_hz = (end_freq_hz - start_freq_hz) / (N - 1)`.
- La fórmula teórica $\text{RBW} = \text{Sample Rate} / \text{FFT_size}$ aplica en el sensor; en post-procesamiento se usa la resolución implícita del vector `Pxx` recibido.

6.17.2. Distinción entre resolución de visualización y exactitud de medición

La resolución espectral determina la separación mínima entre frecuencias distinguibles, pero no es equivalente a la exactitud de medición. Un bin de 10 kHz RBW no implica que la frecuencia central se mida con error de 10 kHz; el centroide de potencia puede dar mayor precisión. Sin embargo:

- El hardware SDR con ancho instantáneo de 20 MHz limita el span analizable sin barrido.
- La exactitud de amplitud está limitada por la planicidad del receptor y se corrige con el archivo de corrección (ver Sección 7).
- A mayor RBW, menor tiempo de adquisición pero peor resolución para señales angostas.

6.18. Calibración y Corrección de Ganancia

6.18.1. Qué corrige el archivo de corrección

El archivo CSV de corrección (parámetro `--corr`) contiene dos columnas: Frecuencia (MHz) y Error (dB). Se aplica como:

$$\text{amplitudes_dbm_corregidas} = \text{amplitudes_dbm} + \text{correction_dB}(f)$$

Esto corrige la respuesta en frecuencia del sistema. Puede absorber: desbalance de respuesta del receptor SDR, no planicidad (ripple) espectral, pérdidas en cable/antena y offsets de ganancia dependientes de frecuencia.

6.18.2. Cómo se genera y periodicidad

- El archivo de corrección debe generarse externamente (no hay rutina automática en este repositorio).
- Se recomienda generarlo con una fuente de referencia calibrada (generador de señal o analizador de espectro certificado).
- Periodicidad recomendada: calibración semestral o ante cambio de hardware/antena.
- Si no se suministra archivo de corrección, Pxx se usa sin ajuste. Esto puede afectar la exactitud de potencia, umbral de detección y ancho de banda en receptores con respuesta no plana.

6.19. Análisis Espectral

6.19.1. Frecuencia Central

La frecuencia central se calcula como el centroide de potencia (baricentro energético) dentro de los límites de la emisión detectada, no como el punto de máxima amplitud. Esto garantiza mayor precisión en señales asimétricas (ej. FM con piloto estéreo).

Campo	Unidad	Descripción
fc_hz	Hz	Frecuencia central estimada en unidad base
fc_mhz	MHz	fc_hz escalado; para reportes
matched_peak_hz	Hz	Pico más cercano a la frecuencia solicitada (modo peaks)
delta_match_hz	Hz	Diferencia entre frecuencia solicitada y pico asociado

6.19.2. Ancho de Banda

Se calculan en paralelo dos métricas: OBW (Occupied Bandwidth, 99 % de la potencia) y ancho a -x dB respecto al pico. El sistema prefiere reportar OBW por ser más estable en señales OFDM, FM estéreo y señales con múltiples picos. Solo si OBW no es válido se usa el criterio -x dB.

Campo	Unidad	Descripción
bandwidth_xdb_hz	Hz	Ancho a -x dB respecto al pico
obw_hz	Hz	OBW: intervalo con 99 % de la potencia
bw_hz	Hz	Ancho de banda seleccionado para reporte (OBW preferido)
bw_khz	kHz	bw_hz en kHz
bw_medido_kHz	kHz	Nombre en modo compliance

6.19.3. Potencia

El cálculo de potencia NO suma directamente valores en dBm. El procedimiento correcto es:

1. Convertir cada muestra $P_{xx}[i]$ de dBm a dominio lineal: $P_{lin}[i] = 10^{(P_{xx}[i]/10)}$ [mW].
2. Sumar la contribución de todos los bins en la región de emisión: $P_{total_mW} = \sum(P_{lin}[i] * bin_hz)$.
3. Convertir de vuelta a dBm: $power_dbm = 10 \cdot \log_{10}(P_{total_mW})$.
4. Si la suma produce valor no finito (NaN, $-\infty$), **power_dbm** se reporta como **null**.
5. Fallback: si la suma directa falla, se intenta integración trapezoidal como segundo recurso.

Distinción importante: **power_dbm** es potencia integrada de la emisión. **channel_power_dbm** es potencia de canal calculada en la medición de parámetros. Ambas son distintas de la potencia de pico (amplitud máxima del bin).

6.19.4. Indicadores Estimados de Calidad (MER/BER)

ADVERTENCIA: MER y BER estimados desde traza espectral NO equivalen a medición formal con demodulación. Estos valores son indicadores de diagnóstico preliminar únicamente.

- Solo aplican a señales TDT (rango de frecuencias TDT Colombia y ancho de banda plausible para OFDM).
- Se estiman usando la relación señal/ruido en banda vs. fuera de banda, asumiendo modulación 64-QAM.

- Una medición formal de MER/BER requiere demodulación completa de la señal o equipo especializado certificado.
- Si la SNR es insuficiente para una estimación confiable, los campos `mer_db` y `ber_est` se omiten del resultado.

Campo	Unidad	Descripción
<code>mer_db</code>	dB	Indicador estimado de calidad de modulación (solo TDT, solo diagnóstico)
<code>ber_est</code>	–	Indicador estimado de tasa de error de bit (solo TDT, solo diagnóstico)
<code>f_center_hz</code>	Hz	Centro de la banda usada para el cálculo
<code>band_start_hz</code>	Hz	Límite inferior de la banda evaluada
<code>band_stop_hz</code>	Hz	Límite superior de la banda evaluada

6.20. Modos de Operación

Modo	Condición de activación	Descripción
<code>all_emissions</code>	<code>picos=[]</code> y <code>cumplimiento=0</code>	Detección automática: analiza toda la traza, detecta emisiones candidatas, mide F_c /BW/potencia.
<code>peaks</code>	<code>picos=[f1, f2, ...]</code> (sin importar <code>cumplimiento</code>)	Análisis por picos: analiza frecuencias específicas entregadas por el usuario.
<code>compliance</code>	<code>picos=[]</code> y <code>cumplimiento=1</code>	Cumplimiento normativo: detecta emisiones y las compara contra base de licencias con filtro DANE/municipio.

6.21. Tolerancias de Cumplimiento contra Licencias

6.21.1. Valores y configuración

Parámetro	Valor por defecto	CLI	Justificación técnica
FC_MARGIN_MHZ	0.1 MHz (100 kHz)	--delta_fc_khz	Margen conservador para SDR con RBW ~10 kHz y posibles derivas de calibración
BW_MARGIN_KHZ	10 kHz	--delta_bw_khz	Tolerancia menor al RBW típico; ajustable según servicio radioeléctrico

6.21.2. Criterios de cumplimiento técnico

Criterio	Regla	Comportamiento
Frecuencia Central (Cumple_FC)	$ f_{medida} - f_{nominal} \leq \text{FC_MARGIN_MHZ}$	Simétrico; cualquier desviación mayor a la tolerancia es incumplimiento
Ancho de Banda (Cumple_BW)	$\text{BW_medido} \leq \text{BW_nominal} + \text{BW_MARGIN_KHZ}$	Asimétrico: emitir con menor BW siempre cumple; solo falla si excede el nominal más la tolerancia
Potencia (Cumple_P)	$P_{medida} \leq P_{nominal}$	Sin margen de tolerancia; cualquier exceso es incumplimiento. Si $P_{medida}=\text{null}$, se reporta None

6.21.3. Manejo de múltiples licencias y licencias sin vigencia

- Si varias licencias coinciden en la misma ventana de DANE: el sistema selecciona una sola licencia usando $score = |\Delta f|$ como criterio principal y $|\Delta bw|$ como desempate secundario. Potencia no decide el match.
- Si hay empate exacto en score: se toma el primer registro que aparece en el CSV (índice más bajo).

- No se reportan múltiples licencias candidatas en esta versión.
- Vigencia de licencias: el código actual NO evalúa fechas ni filtra licencias vencidas. Si el CSV contiene columnas de fecha, no se usan. Esta funcionalidad queda pendiente.

6.21.4. Estados semánticos de licencia

Estado	Significado	Cuándo ocurre
«licencia»: «NO»	Búsqueda ejecutada; no se encontró licencia en la tolerancia	Se pasó filtro DANE/municipio y CSV de licencias, pero ningún registro coincide
«licencia»: null	No se intentó hacer matching	Faltó filtro dane/municipio O faltó CSV de licencias
«licencia»: <nombre>	Match encontrado	Licencia identificada; se evalúan Cumple_FC, Cumple_BW, Cumple_P

6.22. Formato de la Base de Licencias

El archivo CSV de licencias debe contener al menos los siguientes campos:

Campo CSV	Tipo	Descripción	Requerido
frecuencia	Numérico (MHz)	Frecuencia nominal de la licencia	Sí
ancho_de_banda	Numérico (kHz)	Ancho de banda autorizado	Sí
potencia	Numérico (dBm)	Potencia máxima autorizada	Sí
codigo_dane	Numérico	Código DANE del municipio	Sí (para filtro)
municipio	Texto	Nombre del municipio	Sí (para filtro texto)
servicio	Texto	Tipo de servicio radioeléctrico	Recomendado
operador	Texto	Nombre del operador o titular	Recomendado
vigencia	Fecha	Fecha de vencimiento de la licencia	Pendiente de implementar

6.23. Localización Administrativa

6.23.1. Qué es la localización administrativa

La localización administrativa consiste en asociar una emisión detectada al municipio o código DANE del sensor que realizó la captura. Esto permite contextualizar la medición dentro de una zona de análisis y filtrar la base de licencias por área geográfica.

6.23.2. Origen del parámetro territorial

Interfaz	Mecanismo	Referencia en código
API HTTP	Campo municipio en body/meta de la petición, o query param	<code>server_flask.py</code> (líneas 308–336)
CLI / consola	Argumento <code>--municipio</code> en terminal	<code>main.py</code> (líneas 150–235)
Backend de reportes	Objeto ubicacion con municipio y código DANE inyectado al invocar el servicio	<code>reports.ts</code> (líneas 253–271)

6.23.3. Jerarquía de prioridad

- Prioridad 1 — **dan**es (lista): si se entrega lista de códigos DANE, se ignoran los demás campos territoriales.
- Prioridad 2 — **dane** (único): si no hay lista, se evalúa un único código DANE.
- Compatibilidad: si **municipio** es numérico, se interpreta como código DANE.
- Protección contra falsos positivos: si no se especifica ningún filtro territorial, el procesador aborta el matching con el CSV de licencias.

6.23.4. Limitaciones de la localización administrativa

NOTA TÉCNICA: ADVERTENCIA REGULATORIA: Asociar una emisión a un municipio NO prueba que el transmisor esté físicamente en ese municipio. La medición depende de la ubicación del sensor, la cobertura de la señal, la propagación, la altura y el patrón de antena, y la base de licencias disponible. Emisiones provenientes de municipios cercanos pueden aparecer como detectadas en el municipio del sensor.

- La cobertura de una señal puede cruzar límites municipales, especialmente en zonas de montaña o con topografía favorable.
- Para determinar la ubicación física del transmisor se requiere localización radioelétrica real (TDOA, AOA, multilateración), no implementada en este módulo.

6.24. Salidas, API e Integración Cloud

6.24.1. Endpoints de API

Endpoint	Método	Descripción	Módulo
POST /analyze	POST	Procesa un único frame espectral	server_flask.py
POST /analyze_batch	POST	Procesa lote de frames	server_flask.py
GET /health	GET	Verificación de estado del servicio	server_flask.py
POST /api/sensor/data	POST	Envío de datos del sensor SDR remoto al backend Node	Backend Node
POST /api/sensor/gps	POST	Envío de geotag/GPS del sensor	Backend Node
POST /api/sensor/status	POST	Estado del sensor remoto	Backend Node

6.24.2. Campos de salida principales

Campo	Tipo	Descripción
mode	Texto	Modo seleccionado: all_emissions, peaks, compliance
num_emissions	Entero	Cantidad de emisiones o candidatos encontrados
umbral	Numérico	Umbral absoluto usado en la detección
results	Lista	Lista de resultados por emisión
results_by_dane	Dict	Resultados por DANE cuando se pasan múltiples códigos

Campo en results[]	Tipo	Descripción
status	Texto	emission o noise
fc_hz / fc_mhz	Numérico	Frecuencia central en Hz y MHz
bw_hz / bw_khz	Numérico	Ancho de banda en Hz y kHz
power_dbm	Numérico / null	Potencia integrada; null si no es finita
snr_db	Numérico	SNR estimado
Licencia	Texto / null	Nombre de licencia, 'NO', o null
Cumple_FC / Cumple_BW / Cumple_P	Bool / None	Resultado de cada criterio de cumplimiento
mer_db / ber_est	Numérico (opcional)	Indicadores estimados de calidad (solo TDT, solo diagnóstico)

6.25. Ejemplos de Entrada y Salida JSON

6.25.1. Análisis automático (all_emissions)

Entrada: {Pxx: [...], start_freq_hz: 88000000, end_freq_hz: 108000000, cumplimiento: 0}

Salida: {mode: all_emissions, num_emissions: 2, results: [{status: emission, fc_mhz: 95.1, bw_khz: 180.0, power_dbm: -45.2, ...}]}

6.25.2. Análisis por picos

Entrada: {Pxx: [...], start_freq_hz: 88e6, end_freq_hz: 108e6, picos: [95100000]}

Salida: {mode: peaks, results: [{status: emission, requested_pico: 95100000, fc_hz: 95098500, delta_match_hz: 1500}]}

6.25.3. Cumplimiento con DANE único

Entrada: {Pxx: [...], start_freq_hz: ..., end_freq_hz: ..., cumplimiento: 1, dane: 11001, lic: licencias.csv}

Salida: {mode: compliance, results: [{fc_mhz: 95.1, Licencia: OPERADOR_X, Cumple_FC: true, Cumple_BW: true, Cumple_P: false}]}

6.25.4. Sin emisión detectada

Salida: {mode: all_emissions; num_emissions: 0, results: [], umbral: -85.3}

6.25.5. Sin DANE — cumplimiento bloqueado

Salida: {mode: compliance; results: [{Licencia: null, reason: filtro_no_especificado}]}

6.26. Manejo de Errores y Estados de Excepción

Caso de error	Comportamiento del módulo	Salida
JSON incompleto (falta Pxx o frecuencias)	payload_parser rechaza la entrada	Error 400 / excepción con mensaje descriptivo
Frecuencias inválidas ($\text{start} \geq \text{end}$)	SpectrumFrame detecta incoherencia	Error con razón frecuencias_invalidas
Pxx vacío o con un solo bin	No se puede calcular RBW ni detectar emisiones	num_emissions=0; results=[]
DANE no encontrado en CSV de licencias	Filtro no retorna registros	Licencia: «NO» para todas las emisiones
Base de licencias no cargada (lic=None)	Matching no se ejecuta	Licencia: null; reason: filtro_no_especificado
Captura sin SNR suficiente para MER/BER	Cálculo de MER/BER se omite	mer_db y ber_est ausentes del resultado
Potencia no finita (NaN / $-\infty$)	power_utils detecta valor no finito	power_dbm: null
Emisión detectada sin licencia asociada	Match fallido tras búsqueda	Licencia: «NO»; Cumple_FC/BW/P: null
Emisión autorizada pero BW excedido	Cumple_BW = False	Cumple_BW: false; delta_bw reportado
Archivo corr inexistente o mal formado	Corrección se omite o error en carga	Advertencia en log; Pxx sin corregir

6.27. Almacenamiento, Trazabilidad y Reportes

6.27.1. Campos de trazabilidad forense

Para auditoría y análisis posterior, cada resultado debe conservar:

Campo	Descripción	Estado
timestamp	Fecha/hora de la captura (UTC ISO 8601)	Implementado (campo del sensor)
sensor_id	Identificador único del nodo SDR	Implementado (campo del backend)
geotag / GPS	Coordenadas del sensor (lat, lon, alt)	Parcial (disponible en POST /api/sensor/gps; no siempre en el resultado de análisis)
start_freq_hz end_freq_hz	/ Rango de frecuencias de la captura	Implementado
sample_rate_hz rbw_hz	/ Configuración de adquisición	Implementado (en backend/campaña)
mode / umbral	Configuración del análisis ejecutado	Implementado (en salida)
hash de la captura	Identificador único del frame analizado	Pendiente — Recomendado para versión futura
usuario o tarea programada	Quién o qué originó el análisis	Pendiente — Recomendado para versión futura

6.27.2. Campos que alimentan los reportes de administración de espectro

Campo de reporte	Fuente en el módulo
Frecuencia central medida (Hz/MHz)	fc_hz / fc_mhz
Ancho de banda medido (kHz)	bw_khz
Potencia medida (dBm)	power_dbm
Municipio / Código DANE	municipio / dane (entrada)
Estado de licencia	Licencia (campo de salida)
Estado de cumplimiento (FC/BW/P)	Cumple_FC / Cumple_BW / Cumple_P
Fecha y hora	timestamp
Sensor	sensor_id
Observaciones o alertas	reason / status

6.28. Seguridad de la Información

Aspecto de seguridad	Estado actual	Observación
TLS/HTTPS	Parcial — el backend arranca como HTTP en desarrollo	En producción se recomienda proxy/reverse-proxy con TLS. No hay HTTPS nativo en <code>app.ts</code> .
Autenticación JWT	Parcial — soporte JWT en <code>auth.ts</code> del backend Node	El router de sensor (<code>sensor.ts</code>) no aplica middleware JWT en esta versión.
API Key para sensores remotos	No implementado	No hay mecanismo de API Key explícito en sensor/postprocesamiento.
Control de acceso por rol	Parcial — definido en backend Node	Pendiente de aplicar en endpoints de sensor.
Protección de base de licencias CSV	No implementado formalmente	El archivo CSV se lee como archivo local; en producción debe restringirse el acceso.
Integridad de datos recibidos	No implementado	No hay verificación de hash o firma del payload espectral.
Trazabilidad de consultas	Parcial	Los logs de Flask registran requests; no hay auditoría estructurada.

6.29. Pruebas y Validación

6.29.1. Matriz de validación con señales de prueba

Tipo de señal	Objetivo	Resultado esperado
Señal de banda angosta (CW)	Verificar detección y medición F_c /BW	F_c correcta; BW $\sim$ RBW; power_dbm no null
Señal FM comercial	Verificar OBW en señal ancha asimétrica	OBW > ancho -3dB; F_c = centroide energético
Señal TDT/OFDM	Verificar OBW y estimación MER/BER	OBW $\sim$ 8 MHz; mer_db presente si SNR suficiente
Solo ruido (sin señal)	Verificar ausencia de falsos positivos	num_emissions=0 o status=noise para todos
Señal cercana al umbral	Verificar sensibilidad límite	Detección dependiente de umbral_db configurado
Señal saturada	Verificar robustez ante saturación	power_dbm puede ser null; status reportado
Dos emisiones cercanas	Verificar resolución espectral	Dos entradas en results si separación > RBW
Emisión autorizada con licencia	Verificar cumplimiento correcto	Licencia: nombre; Cumple_FC/BW/P: true
Emisión no autorizada (sin licencia)	Verificar detección de infracción	Licencia: «NO»
Emisión con BW excedido	Verificar Cumple_BW=False	Cumple_BW: false; delta_bw positivo

6.29.2. Métricas de desempeño y metas

Métrica	Definición	Meta sugerida
Error máximo de F_c (MAE)	Diferencia $ F_{c_medida} - F_{c_real} $	$< 1 \times \text{RBW}$
Error máximo de potencia (MAE)	Diferencia $ P_{medida} - P_{real} $	$< \pm 1 \text{ dB}$
Error máximo de BW (MAE)	Diferencia $ BW_{medido} - BW_{real} $	$< 2 \times \text{RBW}$
Tasa de falsos positivos (FPR)	Ruido clasificado como emisión / total ruido	$< 5 \%$
Tasa de falsos negativos (FNR)	Emisiones reales no detectadas / total emisiones	$< 5 \%$
F1-Score de detección	Media armónica precisión/recall	$> 0,90$
Latencia de procesamiento	Tiempo desde recepción Pxx hasta salida results	$< 500 \text{ ms por frame}$
Capturas por minuto (API)	Rendimiento sostenido de /analyze	Según capacidad del servidor
Consumo máximo de RAM	RAM en análisis multijurisdicción (múltiples DANE)	Definir según infraestructura

6.30. Limitaciones del Módulo

[Implementado] Análisis espectral: F_c , BW, OBW, potencia, SNR.

[Implementado] Comparación contra licencias con filtro DANE/municipio.

[Parcial] Trazabilidad: timestamp y sensor_id implementados; hash de captura pendiente.

[Parcial] Seguridad: JWT en backend; TLS depende de infraestructura externa.

[Pendiente] Evaluación de vigencia de licencias (fechas en CSV no se evalúan).

[Pendiente] Reporte de múltiples licencias candidatas para una misma emisión.

[Pendiente] Hash/firma del payload espectral para integridad de datos.

[Recomendado] Localización radioeléctrica real (TDOA, AOA, multilateración RSSI).

[Recomendado] Rutina automática de generación del archivo de corrección de ganancia.

[Recomendado] Modelos de IA/ML para detección robusta en baja SNR.

[Recomendado] Compresión de trazas y procesamiento por eventos en el borde.

6.31. Recomendaciones

- Implementar evaluación de vigencia de licencias: agregar columna de fecha al CSV y filtrar registros vencidos antes del matching.
- Agregar hash SHA-256 del payload JSON de cada captura para trazabilidad forense.
- Aplicar middleware JWT al router de sensor (`sensor.ts`) para autenticar nodos remotos.
- Configurar TLS/HTTPS en producción mediante proxy inverso (nginx, traefik).
- Antes de migrar a Deep Learning, evaluar mejoras clásicas: CFAR optimizado, mejor estimación de piso de ruido por segmentos, compresión de trazas en el borde.
- Validar con datasets etiquetados (señales reales con ground truth conocido) antes de reportar métricas de desempeño al ANE.
- Documentar y estandarizar el proceso de generación del archivo de corrección de ganancia (periodicidad semestral recomendada).

6.32. Criterios de Aceptación Verificables

Criterio	Condición de verificación
Detección con umbral configurable	<code>umbral_db</code> de 3, 5 u otro valor produce diferente número de detecciones en señal sintética cerca del umbral
Reporte de F_c en Hz y MHz	Salida contiene <code>fc_hz</code> (entera) y <code>fc_mhz</code> (decimal) para toda emisión detectada
Cálculo OBW 99 %	<code>obw_hz</code> presente y diferente de <code>bandwidth_xdb_hz</code> cuando la región es válida
Potencia integrada correcta	<code>power_dbm</code> no es suma directa de <code>Pxx</code> en dBm; verificable con señal sintética de potencia conocida
Exportación en JSON	Salida de <code>/analyze</code> es JSON válido con todos los campos documentados
Bloqueo sin DA-NE/municipio	Con <code>cumplimiento=1</code> y sin <code>dane/municipio</code> : <code>Licencia=null</code> y <code>reason=filtro_no_especificado</code>
Bloqueo sin base de licencias	Con <code>cumplimiento=1</code> y sin <code>lic</code> : <code>Licencia=null</code>

6.33. Tabla de Pendientes para Completar el Capítulo

Tema	Estado actual	Información faltante	Prioridad	Evidencia
Vigencia de licencias	No implementado	Lógica de filtrado por fecha en CSV	Alta	Prueba de vigencia de licencias
Hash de captura	No implementado	Implementación de SHA-256 en parser	Media	Campo hash en JSON
TLS en producción	Depende de infra	Configuración de proxy HTTPS	Alta	Certificado de endpoint
JWT en sensor router	Parcial	Middleware auth en sensor.ts	Alta	401 ante token
Reporte múltiples licencias	No implementado	Cambio en lógica de matching	Media	results_cancun
Rutina de calibración	No automatizada	Script de generación de CSV corr	Media	Archivo de calibración con equipo
Validación con datasets reales	No realizada	Dataset etiquetado con ground truth	Alta	Reporte de FNR, F1, etc.
DANL y rango dinámico	No caracterizado	Medición formal del receptor	Media	Tabla de datos de ruido

6.34. Conclusión Técnica

El módulo implementa análisis espectral en la nube, detección de emisiones, medición de parámetros técnicos y comparación administrativa contra licencias. Quedan pendientes mejoras de trazabilidad, vigencia de licencias y localización radioeléctrica real.

Capítulo 7

Protocolos de prueba edge-cloud

7.1. Objetivo del capítulo

Este capítulo presenta el protocolo de pruebas propuesto para validar el funcionamiento integral de la plataforma de monitoreo espectral ANE-HX, considerando tanto el componente de borde *edge* –sensor, receptor SDR, procesamiento local, almacenamiento temporal y telemetría– como el componente *cloud* –plataforma web, gestión de campañas, almacenamiento centralizado, visualización, reportes y administración de usuarios–.

El objetivo no es únicamente verificar que cada módulo funcione de manera aislada, sino comprobar que el flujo completo de monitoreo espectral sea trazable, repetible, técnicamente coherente y útil para procesos de vigilancia, análisis y transferencia de conocimiento.

Desde el enfoque pedagógico del informe final, este capítulo busca que un nuevo integrante del equipo pueda entender qué se debe probar, por qué se debe probar, cómo debe ejecutarse cada prueba y qué evidencia debe conservarse para respaldar los resultados obtenidos.

7.2. Alcance de las pruebas

El protocolo cubre las pruebas necesarias para verificar que el sistema pueda ejecutar campañas de monitoreo espectral, adquirir datos de radiofrecuencia, procesar señales localmente, transmitir resultados hacia la plataforma central, visualizar mediciones, generar históricos, exportar información y soportar operación remota.

El alcance incluye los siguientes aspectos:

- Configuración de sensores y selección de antenas.
- Parametrización de frecuencia central, ancho de banda de monitoreo, sample rate, tamaño FFT, RBW, VBW y umbrales.
- Adquisición de datos IQ y estimación de densidad espectral de potencia.

- Validación de detección de emisiones.
- Cálculo de métricas espectrales: frecuencia central, ancho de banda, potencia integrada, ancho de banda ocupado y desviación respecto a valores nominales.
- Evaluación de conectividad entre sensor y plataforma.
- Supervisión de variables de salud del sensor: CPU, RAM, disco, temperatura, estado del proceso SDR, GPS, NTP y red.
- Programación y ejecución de campañas de monitoreo.
- Almacenamiento local y centralizado de datos.
- Exportación de resultados.
- Validación de roles, usuarios, trazabilidad y reportes.

No todas las pruebas descritas en este capítulo deben entenderse como pruebas de aceptación contractual estricta. Algunas corresponden a pruebas funcionales, otras a pruebas técnicas de ingeniería, otras a pruebas de diagnóstico y otras a pruebas pedagógicas orientadas a la transferencia de conocimiento.

7.3. Arquitectura bajo prueba

La arquitectura bajo prueba se entiende como una cadena completa de adquisición, procesamiento, transmisión, almacenamiento y análisis. El sistema puede representarse conceptualmente mediante los siguientes bloques:

1. **Frente de radiofrecuencia:** antenas, conmutador RF, conectores, acoples, filtros y receptor SDR.
2. **Nodo edge:** computador embebido, receptor SDR, motor de adquisición, procesamiento local, almacenamiento temporal y telemetría.
3. **Canal de comunicación:** Ethernet, WiFi o red móvil, con verificación de disponibilidad, latencia y estabilidad.
4. **Backend institucional o cloud:** recepción de datos, base de datos, administración de campañas, gestión de sensores y procesamiento adicional.
5. **Interfaz de usuario:** visualización espectral, mapa, campañas, historial, alertas, usuarios, reportes y exportaciones.

La validación debe comprobar que estos bloques funcionen de manera coordinada. Un sensor puede adquirir correctamente, pero si no transmite datos con timestamp, geolocalización, identificador único y parámetros de adquisición, la medición pierde trazabilidad. De forma equivalente, una interfaz puede visualizar datos, pero si no conserva los parámetros técnicos de captura, no permite auditoría posterior ni reproducción del análisis.

7.4. Criterios generales de aceptación

Para cada prueba se deben registrar, como mínimo, los siguientes elementos:

- Identificador de prueba.
- Módulo evaluado.
- Requerimiento asociado.
- Condiciones iniciales.
- Procedimiento paso a paso.
- Resultado esperado.
- Resultado observado.
- Evidencia: captura, archivo JSON, archivo CSV, log, gráfica, reporte o fotografía.
- Estado: aprobada, aprobada con observaciones, fallida o no ejecutada.
- Responsable y fecha de ejecución.

Se recomienda clasificar los resultados de la siguiente manera:

- **Aprobada:** el comportamiento observado coincide con el resultado esperado.
- **Aprobada con observaciones:** la funcionalidad opera, pero existen restricciones, desviaciones menores o recomendaciones.
- **Fallida:** el sistema no cumple el criterio definido o presenta comportamiento no reproducible.
- **No ejecutada:** la prueba no pudo realizarse por falta de equipo, datos, acceso o condiciones controladas.

7.5. Estrategia general de pruebas

La estrategia propuesta combina pruebas funcionales, pruebas de integración, pruebas metrológicas, pruebas de operación remota y pruebas pedagógicas. Esta división permite diferenciar entre errores de software, fallas de comunicación, limitaciones del hardware SDR y problemas de operación.

7.5.1. Pruebas funcionales

Las pruebas funcionales verifican que las opciones visibles para el usuario operen de acuerdo con lo esperado. Incluyen inicio de sesión, selección de sensores, configuración de campañas, visualización de mediciones, consulta de históricos, generación de reportes y exportación de datos.

7.5.2. Pruebas de integración edge

Las pruebas de integración edge evalúan la interacción entre el hardware y el software local del sensor. Incluyen comunicación con el SDR, selección de antenas, adquisición de muestras IQ, generación de PSD, almacenamiento local, lectura de GPS, sincronización horaria y monitoreo de recursos del sistema.

7.5.3. Pruebas de integración edge-cloud

Las pruebas de integración edge-cloud verifican la comunicación entre el sensor y la plataforma central. Incluyen registro del sensor, envío de telemetría, envío de resultados espectrales, ejecución remota de campañas, recuperación ante pérdida de conectividad y consistencia de timestamps.

7.5.4. Pruebas metrológicas y espectrales

Las pruebas metrológicas y espectrales permiten conocer los límites reales del sistema de medición. Incluyen sensibilidad, DANL, rango dinámico, exactitud de frecuencia, precisión de amplitud, estabilidad del piso de ruido, medición de ancho de banda y detección de emisiones.

7.5.5. Pruebas pedagógicas

Las pruebas pedagógicas verifican que los procedimientos puedan ser reproducidos por nuevos usuarios técnicos. El objetivo es que el informe final no sea solamente un documento de cierre, sino también una guía de transferencia de conocimiento para operación, mantenimiento y evolución de la plataforma.

7.6. Matriz general de pruebas

La Tabla 7.1 presenta una matriz general de pruebas recomendadas para el sistema edge-cloud. Los estados deben actualizarse de acuerdo con la evidencia real disponible.

Tabla 7.1: Matriz general de pruebas edge-cloud.

ID	Prueba	Módulo	Resultado esperado
PR-EC-001	Registro e identificación del sensor	Plataforma y edge	El sensor aparece registrado con identificador único, ubicación, estado y metadatos técnicos.
PR-EC-002	Heartbeat del sensor	Edge-cloud	El sensor reporta periódicamente estado de CPU, RAM, disco, temperatura, red, GPS y proceso SDR.
PR-EC-003	Verificación de conectividad	Red	La plataforma identifica si el sensor está en línea o fuera de línea y registra latencia cuando aplique.
PR-EC-004	Sincronización temporal	Edge, NTP y GPS	Los timestamps de adquisición son coherentes con la hora del sistema.
PR-EC-005	Selección de sensor activo	Interfaz	El usuario puede seleccionar un sensor disponible y habilitar el panel de adquisición.
PR-EC-006	Selección de antena	Hardware RF	La antena seleccionada corresponde al puerto RF activado.

ID	Prueba	Módulo	Resultado esperado
PR-EC-007	Configuración de frecuencia central	SDR	El sistema permite configurar una frecuencia válida dentro del rango técnico del hardware.
PR-EC-008	Configuración de sample rate	SDR y DSP	El sistema acepta valores permitidos y rechaza configuraciones fuera del rango soportado.
PR-EC-009	Configuración de FFT y RBW	DSP	El RBW mostrado coincide con la relación entre sample rate y tamaño FFT.
PR-EC-010	Adquisición IQ local	Edge SDR	El sensor captura muestras IQ y registra una salida válida sin corrupción.
PR-EC-011	Cálculo de PSD	DSP edge	La PSD generada presenta eje de frecuencia, unidades de potencia y número de bins coherentes.
PR-EC-012	Visualización espectral	Interfaz cloud	La plataforma muestra la traza espectral correspondiente al sensor y configuración seleccionados.
PR-EC-013	Detección por umbral	Algoritmo espectral	El sistema diferencia piso de ruido y ocupación de señal mediante un umbral configurable.
PR-EC-014	Medición de frecuencia central	Algoritmo espectral	La frecuencia medida de una emisión conocida se mantiene dentro de la tolerancia definida.
PR-EC-015	Medición de ancho de banda	Algoritmo espectral	El sistema calcula ancho de banda por método X dB y por OBW cuando aplique.
PR-EC-016	Medición de potencia de canal	Algoritmo espectral	El sistema integra potencia sobre el ancho de canal definido y reporta un valor trazable.
PR-EC-017	Corrección de DC offset	DSP edge	La componente central espuria se reduce respecto a la señal cruda.

ID	Prueba	Módulo	Resultado esperado
PR-EC-018	Corrección de balance IQ	DSP edge	Se reduce la imagen espectral y mejora la calidad de la PSD.
PR-EC-019	Ejecución de campaña programada	Plataforma y edge	La campaña se ejecuta en el horario configurado y genera resultados históricos.
PR-EC-020	Cancelación de campaña	Plataforma	Un usuario autorizado puede cancelar una campaña programada antes de su ejecución.
PR-EC-021	Persistencia ante pérdida de red	Edge	Si se pierde conectividad, el sensor conserva resultados temporalmente cuando sea posible.
PR-EC-022	Almacenamiento centralizado	Backend	Los resultados quedan asociados a sensor, timestamp, ubicación y parámetros de adquisición.
PR-EC-023	Consulta de histórico	Plataforma	El usuario puede consultar campañas y mediciones anteriores con filtros básicos.
PR-EC-024	Exportación CSV y JSON	Plataforma	La plataforma exporta resultados completos y legibles para procesamiento offline.
PR-EC-025	Validación de roles	Seguridad	Los permisos dependen del rol del usuario.
PR-EC-026	Alertas de salud del sensor	Plataforma y telemetría	La plataforma genera alertas cuando una variable supera un umbral definido.
PR-EC-027	Prueba de mapa geográfico	Interfaz y geolocalización	El sensor aparece ubicado correctamente en el mapa.
PR-EC-028	Prueba de DANL	Caracterización RF	Se documenta el nivel de ruido propio del sistema por banda o configuración.
PR-EC-029	Prueba de rango dinámico	Caracterización RF	Se identifica el intervalo entre señal mínima detectable y máxima señal sin distorsión evidente.

ID	Prueba	Módulo	Resultado esperado
PR-EC-030	Prueba de exactitud de frecuencia	Caracterización RF	La diferencia entre frecuencia inyectada y frecuencia medida queda dentro de la tolerancia definida.
PR-EC-031	Prueba de precisión de amplitud	Caracterización RF	La diferencia entre potencia de referencia y potencia medida se registra en dB.
PR-EC-032	Prueba de documentación pedagógica	Transferencia de conocimiento	Un usuario técnico puede reproducir una prueba siguiendo el procedimiento documentado.

7.7. Protocolos detallados de prueba

7.7.1. PR-EC-001: registro e identificación del sensor

Objetivo: verificar que cada sensor pueda ser registrado en la plataforma con un identificador único y metadatos suficientes para trazabilidad.

Condiciones iniciales:

- Plataforma cloud activa.
- Usuario con permisos de administración.
- Sensor energizado y con conectividad disponible.

Procedimiento:

1. Ingresar a la plataforma con usuario administrador.
2. Acceder al módulo de configuración o registro de dispositivos.
3. Crear o editar un sensor.
4. Asociar nombre, identificador, ubicación, antenas disponibles y descripción técnica.
5. Guardar la configuración.
6. Verificar que el sensor aparezca en el listado general.

Resultado esperado: el sensor queda registrado, aparece disponible para campañas y conserva sus metadatos técnicos.

Evidencia sugerida: captura del formulario de registro, captura del listado de sensores y registro en base de datos.

7.7.2. PR-EC-002: heartbeat y salud del sensor

Objetivo: comprobar que el nodo edge reporte periódicamente su estado operativo.

Variables mínimas a registrar:

- Estado en línea o fuera de línea.
- Uso de CPU.
- Uso de RAM.
- Uso de disco.
- Temperatura.
- Estado del proceso SDR.
- Estado de red.
- Estado GPS o coordenadas.
- Hora del sistema.

Procedimiento:

1. Encender el sensor.
2. Verificar conectividad IP.
3. Ingresar al panel de sensores.
4. Observar la actualización periódica del estado.
5. Desconectar temporalmente la red del sensor.
6. Verificar que la plataforma detecte el cambio de estado.
7. Reconectar el sensor.
8. Confirmar recuperación del estado en línea.

Resultado esperado: la plataforma refleja correctamente el estado operativo del sensor y permite detectar fallas de conectividad o recursos.

7.7.3. PR-EC-003: configuración de adquisición SDR

Objetivo: validar que la plataforma pueda enviar parámetros de adquisición al sensor y que el sensor los aplique correctamente.

Parámetros bajo prueba:

- Frecuencia central.
- Ancho de banda de monitoreo.
- Sample rate.
- Ganancia de recepción, si aplica.
- Tamaño FFT.
- Antena seleccionada.
- Duración de adquisición.

Procedimiento:

1. Seleccionar un sensor en línea.
2. Seleccionar una antena compatible con la banda de prueba.
3. Configurar una frecuencia central conocida.
4. Configurar sample rate y tamaño FFT.
5. Iniciar adquisición.
6. Revisar logs del sensor.
7. Verificar que la configuración aplicada coincide con la configuración solicitada.

Resultado esperado: el sensor aplica los parámetros y genera una medición válida sin errores de configuración.

7.7.4. PR-EC-004: validación de RBW

Objetivo: verificar que el valor de RBW mostrado por la plataforma sea coherente con los parámetros DSP.

El RBW debe calcularse como:

$$RBW = \frac{F_s}{N_{FFT}}$$

donde F_s es el sample rate y N_{FFT} es el tamaño de la FFT.

Procedimiento:

1. Configurar un sample rate conocido.
2. Configurar un tamaño FFT conocido.
3. Iniciar adquisición.
4. Consultar el RBW mostrado en la interfaz o archivo de salida.
5. Calcular manualmente el RBW.
6. Comparar ambos valores.

Resultado esperado: el RBW reportado coincide con el cálculo teórico, dentro de una tolerancia asociada al redondeo de visualización.

7.7.5. PR-EC-005: adquisición IQ y generación de PSD

Objetivo: comprobar que el sensor capture datos IQ y genere una PSD correctamente indexada en frecuencia.

Procedimiento:

1. Configurar una banda con actividad espectral conocida.
2. Ejecutar una adquisición.
3. Confirmar la existencia de archivo IQ, buffer o salida procesada.
4. Verificar número de muestras.
5. Generar PSD.

6. Confirmar que el eje de frecuencia corresponde a la frecuencia central y al ancho de banda configurados.
7. Verificar que la potencia se encuentre en unidades consistentes.

Resultado esperado: la PSD debe contener vector de frecuencias, vector de potencia, metadatos de adquisición y timestamp.

7.7.6. PR-EC-006: detección de emisiones por umbral

Objetivo: validar que el algoritmo pueda distinguir entre piso de ruido y emisiones presentes.

Procedimiento:

1. Capturar una banda con al menos una emisión visible.
2. Estimar el piso de ruido.
3. Definir un umbral de detección, por ejemplo $NF + 3$ dB o $NF + 5$ dB.
4. Identificar bins por encima del umbral.
5. Agrupar bins contiguos como emisiones.
6. Registrar frecuencia central, ancho de banda y potencia.

Resultado esperado: el algoritmo detecta emisiones reales y evita clasificar fluctuaciones aleatorias de ruido como señales.

7.7.7. PR-EC-007: medición de ancho de banda ocupado

Objetivo: comprobar que el sistema mida ancho de banda usando criterios X dB y OBW.

Procedimiento:

1. Seleccionar una emisión estable.
2. Calcular el punto de máxima potencia.
3. Aplicar el método X dB para $X = 3$, $X = 10$ o $X = 26$, según configuración.
4. Calcular OBW al 99% integrando potencia acumulada.

5. Comparar ambos resultados.
6. Registrar diferencias y condiciones de medición.

Resultado esperado: el sistema reporta ancho de banda de forma consistente y diferencia explícitamente el método usado.

7.7.8. PR-EC-008: programación y ejecución de campañas

Objetivo: verificar que una campaña programada desde la plataforma sea ejecutada por el sensor correspondiente.

Procedimiento:

1. Crear una campaña con sensor, antena, frecuencia, ancho de banda, duración y periodicidad.
2. Guardar la campaña.
3. Verificar que aparece como programada.
4. Esperar la hora de ejecución.
5. Confirmar inicio de adquisición en el sensor.
6. Verificar recepción de datos en la plataforma.
7. Revisar histórico de campaña.

Resultado esperado: la campaña se ejecuta automáticamente y genera registros consultables.

7.7.9. PR-EC-009: recuperación ante pérdida de conectividad

Objetivo: evaluar la tolerancia del sistema ante desconexiones temporales entre edge y cloud.

Procedimiento:

1. Iniciar una campaña o adquisición.
2. Interrumpir temporalmente la conectividad del sensor.
3. Verificar que el sensor no pierda la adquisición local, si el diseño lo permite.

4. Restaurar la conectividad.
5. Confirmar si los resultados pendientes son transmitidos.
6. Revisar logs de error y recuperación.

Resultado esperado: el sistema debe registrar la pérdida de conexión, conservar datos críticos y recuperar operación sin intervención manual, cuando sea técnicamente posible.

7.7.10. PR-EC-010: exportación de datos

Objetivo: validar que los resultados puedan descargarse para análisis offline.

Formatos mínimos sugeridos:

- CSV para análisis tabular.
- JSON para interoperabilidad técnica.
- XML si se requiere integración con sistemas institucionales.

Procedimiento:

1. Seleccionar una campaña terminada.
2. Descargar resultados en CSV.
3. Descargar resultados en JSON.
4. Verificar que los archivos incluyan sensor, timestamp, coordenadas, parámetros de adquisición y resultados espectrales.
5. Abrir los archivos en una herramienta externa.

Resultado esperado: los archivos exportados son legibles, completos y consistentes con la medición mostrada en plataforma.

7.8. Pruebas metrológicas recomendadas

7.8.1. DANL

El DANL permite estimar el nivel promedio de ruido mostrado por el sistema en ausencia de señales intencionales. Esta prueba es fundamental para conocer la sensibilidad práctica del sistema.

Procedimiento sugerido:

1. Terminar el puerto RF en $50\ \Omega$.
2. Configurar frecuencia central y sample rate.
3. Realizar varias capturas.
4. Promediar la PSD.
5. Registrar el nivel de ruido por banda.
6. Repetir para distintas frecuencias representativas.

Resultado esperado: se obtiene una curva o tabla de DANL por frecuencia y configuración.

7.8.2. Exactitud de frecuencia

Procedimiento sugerido:

1. Inyectar una señal senoidal de frecuencia conocida.
2. Configurar el sensor alrededor de esa frecuencia.
3. Medir la frecuencia de pico.
4. Calcular el error absoluto y relativo en ppm.

$$Error_{Hz} = f_{medida} - f_{referencia}$$

$$Error_{ppm} = \frac{f_{medida} - f_{referencia}}{f_{referencia}} \times 10^6$$

Resultado esperado: el error de frecuencia queda documentado y puede compararse entre sensores.

7.8.3. Precisión de amplitud

Procedimiento sugerido:

1. Inyectar una señal de potencia conocida.
2. Medir la potencia reportada por el sistema.
3. Repetir para varios niveles de potencia.
4. Calcular error en dB.

$$Error_{dB} = P_{medida} - P_{referencia}$$

Resultado esperado: se obtiene una tabla de error de amplitud por nivel de potencia.

7.8.4. Rango dinámico

Procedimiento sugerido:

1. Estimar el DANL del receptor.
2. Inyectar señales de amplitud creciente.
3. Identificar el nivel máximo antes de compresión, saturación o aparición significativa de espurios.
4. Calcular la diferencia entre el nivel máximo lineal y el nivel mínimo detectable.

Resultado esperado: se documenta el rango dinámico práctico del sistema bajo condiciones de prueba.

7.9. Pruebas de instalación y operación de campo

Las pruebas de campo deben validar que el sensor opere en condiciones reales de instalación. Se recomienda incluir:

- Verificación de alimentación estable.
- Estado de sellado y protección ambiental.

- Correcta instalación de antenas.
- Confirmación de GPS.
- Conectividad IP.
- Prueba de adquisición corta.
- Verificación de reporte en plataforma.
- Registro fotográfico del emplazamiento.

Estas pruebas son importantes porque un sistema puede funcionar correctamente en laboratorio, pero presentar fallas en campo por alimentación inestable, mala conectividad, ubicación deficiente de antenas, ausencia de sincronización temporal o interferencias locales.

7.10. Problemas, decisiones y ajustes esperados

La Tabla [7.2](#) resume problemas típicos que pueden aparecer durante la validación del sistema, junto con causas probables y ajustes recomendados.

Tabla 7.2: Problemas frecuentes durante las pruebas edge-cloud.

Problema	Evidencia	Causa probable	Ajuste recomendado
Pérdida temporal de conectividad del sensor	Sensor aparece fuera de línea o sin heartbeat	Red inestable, pérdida de enlace WiFi, LTE o Ethernet	Implementar persistencia local y reintento de transmisión.
Espurio en frecuencia central	Pico persistente en el centro de la PSD	DC offset del receptor de conversión directa	Aplicar corrección DC o estrategia de sintonía con desplazamiento.
Imágenes espectrales	Componentes reflejadas alrededor de la portadora	Desbalance de amplitud o fase entre ramas I y Q	Aplicar corrección de balance IQ y documentar mejora.
Medición inestable del piso de ruido	Variación fuerte del umbral entre capturas	Ganancia no controlada, entorno RF variable o promediado insuficiente	Fijar ganancia, aumentar promediado y registrar condiciones de prueba.
Inconsistencia entre campaña y datos recibidos	Resultados sin metadatos completos	Falta de asociación entre adquisición, sensor y campaña	Exigir identificador de campaña, sensor, timestamp y configuración en cada paquete.
Errores de usuario en configuración	Frecuencias o anchos de banda fuera de rango	Falta de validación en interfaz	Agregar validaciones y mensajes pedagógicos.

7.11. Evidencias mínimas por prueba

Cada prueba debe acompañarse de evidencia verificable. Se recomienda conservar, como mínimo, los siguientes tipos de evidencia:

- Capturas de pantalla de la interfaz.
- Logs del sensor.
- Logs de backend.
- Archivos de salida en CSV o JSON.
- Gráficas de PSD.

- Fotografías del montaje físico.
- Tabla de resultados observados.
- Comentarios del responsable de la prueba.

Una posible estructura de carpetas para organizar evidencias es la siguiente:

```
section/img/test_protocols/
  PR-EC-001-sensor-registration.png
  PR-EC-002-heartbeat.png
  PR-EC-005-psd-example.png
  PR-EC-008-campaign-execution.png

test-results/
  PR-EC-005-psd-output.json
  PR-EC-010-export.csv
  PR-EC-014-frequency-accuracy.csv
  PR-EC-028-danl-results.csv
```

Para cada evidencia se debe indicar:

- Nombre del archivo.
- Prueba asociada.
- Fecha.
- Responsable.
- Descripción breve.
- Interpretación del resultado.

7.12. Transferencia de conocimiento

El capítulo debe cumplir una función pedagógica: permitir que un nuevo integrante del equipo entienda qué se prueba, por qué se prueba y cómo interpretar los resultados.

Tabla 7.3: Aprendizajes transferibles asociados a las pruebas.

Tema	Aprendizaje obtenido	Prueba asociada
RBW y FFT	El RBW depende directamente del sample rate y del tamaño FFT; no es un parámetro independiente.	PR-EC-004
Piso de ruido y umbral	El umbral debe definirse por encima del piso de ruido para evitar falsos positivos.	PR-EC-006
DC offset	Los SDR de conversión directa pueden presentar espurios en frecuencia central que afectan la detección.	PR-EC-017
Balance IQ	El desbalance IQ genera imágenes espectrales que pueden confundirse con emisiones reales.	PR-EC-018
Trazabilidad	Toda medición debe conservar sensor, timestamp, ubicación y parámetros de adquisición.	PR-EC-022
Flujo edge-cloud	El sistema no se valida únicamente en el sensor ni únicamente en la web; debe validarse el flujo completo.	PR-EC-001 a PR-EC-027
Caracterización RF	DANL, exactitud de frecuencia, precisión de amplitud y rango dinámico permiten conocer los límites reales del sistema.	PR-EC-028 a PR-EC-031

7.13. Recomendaciones para futuras pruebas

Se recomienda que las pruebas futuras incorporen mayor automatización, especialmente en los siguientes aspectos:

- Generación automática de reportes de prueba.
- Ejecución periódica de pruebas de conectividad y heartbeat.
- Validación automática de estructura JSON.
- Comparación automática entre configuración solicitada y configuración aplicada.
- Pruebas con señales sintéticas controladas.
- Pruebas de regresión después de cada actualización del software.
- Banco de pruebas con generador RF, atenuadores calibrados y carga de 50 Ω .
- Registro sistemático de temperatura, ganancia, frecuencia, antena y condiciones ambientales.

También se recomienda separar claramente tres niveles de evidencia:

1. **Evidencia de funcionamiento:** capturas de interfaz, logs y archivos exportados.
2. **Evidencia técnica:** mediciones, gráficas de PSD, cálculos de error y resultados de caracterización.
3. **Evidencia pedagógica:** explicación del procedimiento, interpretación de resultados y recomendaciones de uso.

7.14. Conclusiones del capítulo

El protocolo de pruebas edge-cloud permite validar la plataforma ANE-HX como un sistema integral de monitoreo espectral, no como una suma aislada de componentes. La correcta operación depende de la coordinación entre el receptor SDR, el procesamiento local, la conectividad, la plataforma central, la visualización, el almacenamiento histórico y la capacidad de exportar resultados.

Las pruebas propuestas cubren el flujo completo desde la adquisición RF hasta la consulta de resultados en la plataforma. Además, incorporan pruebas metrológicas necesarias para entender las limitaciones reales del sistema, tales como DANL, rango dinámico, exactitud de frecuencia y precisión de amplitud.

Desde el enfoque pedagógico del informe final, este capítulo debe servir como guía para que futuros usuarios, desarrolladores o funcionarios puedan reproducir pruebas, interpretar evidencias y comprender por qué cada validación es necesaria dentro de un sistema institucional de monitoreo del espectro.

Capítulo 8

Integración general del sistema

8.1. Propósito del capítulo

[[Placeholder]]

8.2. Contexto dentro del sistema general

[[Placeholder]]

8.3. Desarrollo técnico

[[Placeholder]]

8.4. Entradas, salidas e interfaces

[[Placeholder]]

8.5. Decisiones de diseño o implementación

[[Placeholder]]

8.6. Problemas presentados y ajustes realizados

[[Placeholder]]

8.7. Validación, pruebas o evidencias

[[Placeholder]]

8.8. Aprendizajes y transferencia de conocimiento

[[Placeholder]]

8.9. Limitaciones

[[Placeholder]]

8.10. Recomendaciones específicas

[[Placeholder]]

Capítulo 9

Conclusiones generales

9.1. Síntesis de resultados técnicos

[[Placeholder]]

9.2. Logros del diseño e implementación

[[Placeholder]]

9.3. Resultados de la integración

[[Placeholder]]

9.4. Lecciones aprendidas

[[Placeholder]]

9.5. Cumplimiento de objetivos

[[Placeholder]]

Capítulo 10

Recomendaciones

10.1. Mejoras de arquitectura

[[Placeholder]]

10.2. Mejoras de hardware

[[Placeholder]]

10.3. Mejoras de software

[[Placeholder]]

10.4. Mejoras de documentación

[[Placeholder]]

10.5. Reducción de riesgos identificados

[[Placeholder]]

Capítulo 11

Referencias

[[Placeholder]]

Apéndice A

Material de apoyo: uso de agentes de IA para documentación, desarrollo y revisión técnica del proyecto

A.1. Nota de alcance y propósito

Este capítulo no describe un módulo funcional o componente obligatorio del sistema ANE. Se presenta exclusivamente como **material de apoyo metodológico** que documenta las técnicas de asistencia técnica utilizadas por el equipo de trabajo. El objetivo es proporcionar una guía sobre cómo el uso responsable de Inteligencia Artificial (IA) ha apoyado la documentación, revisión de código y organización de la información durante el ciclo de vida del proyecto.

A.2. Contexto y diferenciación funcional

Es fundamental diferenciar entre el uso de la IA como herramienta de apoyo al desarrollo y su posible rol como componente funcional del sistema.

- **IA como herramienta de apoyo:** Metodología aplicada para mejorar la productividad, coherencia documental y calidad del código mediante revisión asistida.
- **IA como componente del sistema:** Funcionalidades de análisis inteligente de señales o gestión autónoma que se plantean como propuestas futuras y no forman parte de la arquitectura operativa actual de ANE.

A.3. Metodología de trabajo: Ecosistema de Agentes

El equipo emplea herramientas de IA aprobadas institucionalmente para asistir en tareas técnicas. Aunque el enfoque es agnóstico al modelo, se documentan a continuación los pasos de habilitación para las herramientas utilizadas como referencia durante el proyecto.

A.3.1. Habilitación del Entorno (Ejemplos CLI)

Para permitir una interacción directa desde la terminal y facilitar la automatización, se utilizaron los siguientes agentes:

- **GitHub Copilot CLI:** Utilizado para asistencia en codificación y gestión de repositorios.
 - Instalación: `npm install -g @github/copilot`
 - Autenticación: `copilot /login`
- **Gemini CLI:** Utilizado para tareas multimodales, análisis de arquitectura y gestión de habilidades.
 - Instalación: `npm install -g @google/gemini-cli`
 - Inicio: `gemini`

A.3.2. Arquitectura Multi-Agente Asistida

La arquitectura multi-agente se define aquí como una **metodología de trabajo asistido**, no como una implementación automática dentro del sistema. Se sugieren los siguientes roles para la colaboración:

- **Agente Documental:** Apoyo en redacción técnica, generación de tablas y coherencia de estilo.
- **Agente de Código:** Revisión de scripts, detección de errores lógicos y sugerencias de optimización.
- **Agente de Pruebas:** Generación de casos de prueba y protocolos de validación.
- **Agente de Arquitectura:** Análisis de dependencias y revisión de diagramas de diseño.
- **Agente de Señales/RF:** Apoyo en el análisis de logs y procesamiento teórico de espectro.

- **Agente de Revisión:** Validación cruzada de respuestas para minimizar alucinaciones.

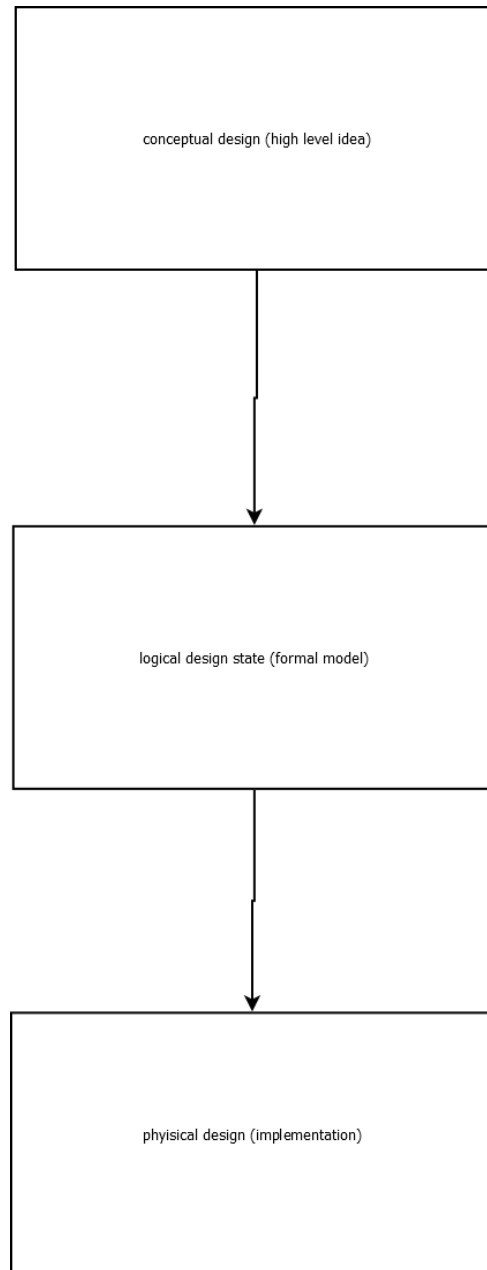


Figura A.1: Diseño conceptual del sistema multi-agente: Relación metodológica entre roles de asistencia. Fuente: Elaboración propia.

A.4. Gobernanza y Uso Responsable de IA

El uso de herramientas de IA en el proyecto ANE está sujeto a las siguientes restricciones de seguridad y ética:

- **Revisión Humana Obligatoria:** Ningún resultado generado por IA se considera definitivo sin la validación técnica de un responsable humano.
- **Confidencialidad:** Está estrictamente prohibido cargar información sensible, credenciales, llaves de API o datos institucionales restringidos en herramientas externas.
- **No Normatividad:** La IA no es una fuente normativa; las decisiones técnicas se basan en estándares de ingeniería y requerimientos del proyecto.
- **Validación de Código:** No se acepta código sugerido sin pruebas unitarias y validación en el entorno de desarrollo.

A.5. Trazabilidad Metodológica

A continuación, se presenta la trazabilidad del uso de IA frente a las actividades de apoyo del proyecto.

Tabla A.1: Trazabilidad de actividades de apoyo con IA.

Actividad de Apoyo	Aplicación en el Proyecto	Estado
Documentación Técnica	Generación de estructuras y tablas iniciales.	Aplicado
Revisión de Código	Análisis de scripts de adquisición edge.	Parcial
Generación de Pruebas	Creación de protocolos de validación HW.	Sugerido
Análisis de Errores	Interpretación de logs de comunicación cloud.	Aplicado
Organización de Info.	Clasificación de requisitos y dependencias.	Recomendado

A.6. Flujo de Trabajo Colaborativo (Paso a Paso)

Para asegurar la calidad y trazabilidad, se sigue el siguiente flujo:

1. **Definir Tarea:** Clarificar el objetivo técnico y las restricciones.
2. **Preparar Contexto:** Seleccionar archivos, logs o requisitos relevantes.

3. **Consultar IA:** Utilizar prompts estructurados (ROLE-OBJECTIVE-CONTEXT).
4. **Revisar Respuesta:** Identificar posibles errores o inconsistencias.
5. **Validar Técnicamente:** Probar el código o verificar los datos en el sistema real.
6. **Integrar y Documentar:** Subir al repositorio (GitHub) indicando la asistencia recibida y quién validó.

A.7. Trazabilidad de Prompts y Relación con GitHub

Cada interacción significativa con la IA debe ser trazable. Se recomienda asociar los prompts importantes a:

- **GitHub Issues:** Documentar el prompt usado para resolver un bug específico.
- **Commits/PRs:** Indicar en el mensaje del commit si se utilizó asistencia de IA y quién realizó la revisión final.
- **Archivo de Evidencias:** Registro del prompt, respuesta y decisión tomada.

A.8. Problemas, Limitaciones y Ajustes

La IA presenta limitaciones inherentes que deben ser gestionadas mediante supervisión constante.

Tabla A.2: Gestión de problemas y limitaciones de la IA.

Problema/Riesgo	Manifestación	Causa	Ajuste/Acción	Resultado
Alucinaciones	Librerías in-existentes o comandos falsos.	Falta de datos en el modelo.	Validación cruzada y búsqueda en docs oficiales.	Eliminación de errores técnicos.
Exposición de Datos	Riesgo de subir credenciales.	Descuido en la preparación de contexto.	Uso de <code>.gitignore</code> y limpieza de prompts.	Integridad de secretos mantenida.
Dependencia	Pérdida de criterio técnico propio.	Uso excesivo sin razonamiento.	Rotación de tareas y revisión por pares.	Mantenimiento del conocimiento.

A.9. Métricas de Utilidad y Evidencias

Para evaluar la efectividad de esta metodología, se consideran las siguientes métricas simples:

- **Tareas Apoyadas:** Número de capítulos o módulos revisados con asistencia.
- **Propuestas Aceptadas/Descartadas:** Porcentaje de sugerencias de IA integradas tras revisión.
- **Tiempo Ahorrado:** Estimación del impacto en la velocidad de documentación y depuración.

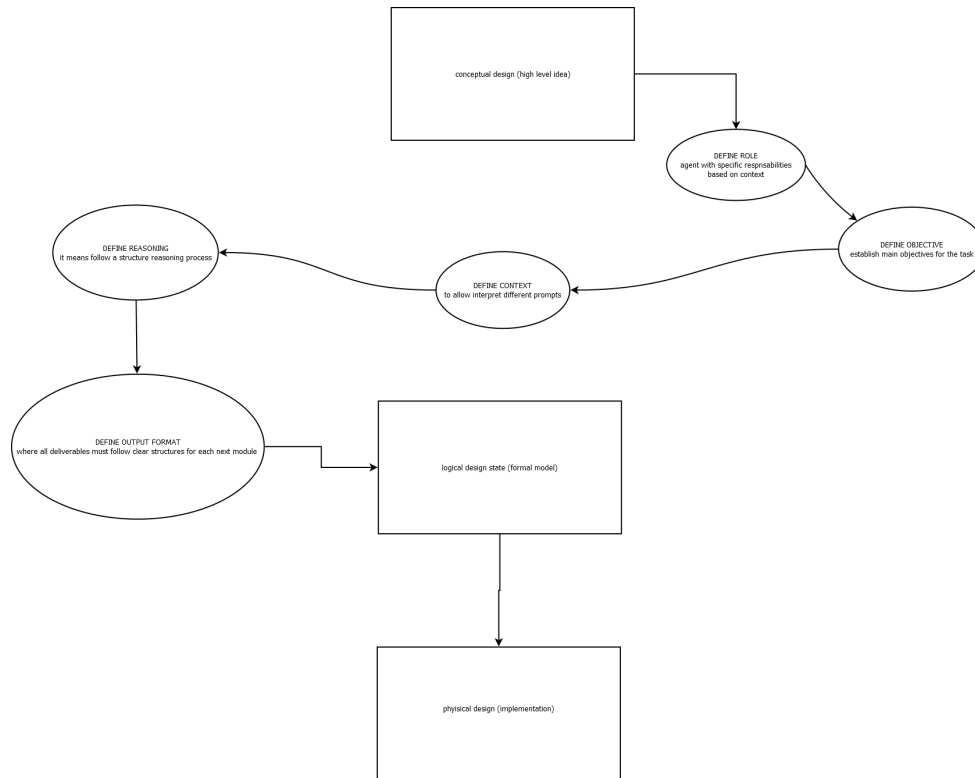


Figura A.2: Flujo de trabajo y validación humana en la integración de resultados. Fuente: Elaboración propia.

A.10. Conclusión del capítulo

La Inteligencia Artificial se propone y utiliza como un multiplicador de productividad para el equipo técnico. Sin embargo, se enfatiza que la responsabilidad final de todas las

decisiones, la seguridad del código y la veracidad de la documentación recae exclusivamente en los ingenieros responsables del proyecto. La validación humana es el pilar central que garantiza la integridad del sistema ANE.

Apéndice B

Diagramas adicionales

B.1. Diagramas de flujo de la plataforma

Este anexo complementa el Capítulo 6 (Plataforma web en la nube) presentando los diagramas de flujo y arquitectura interna de los tres componentes principales que conforman la plataforma: el frontend de aplicación de página única, el backend de servicios y orquestación, y el servicio de postprocesamiento para análisis espectral. Estos diagramas detallan la estructura interna, los flujos de datos y las interacciones entre los módulos que componen cada uno de estos componentes.

B.1.1. Frontend

El frontend de la plataforma web está desarrollado con React 19, TypeScript y Vite. El siguiente diagrama presenta la organización de componentes, el flujo de navegación entre pantallas y las conexiones con los servicios externos: API REST mediante Axios, WebSocket para datos en tiempo real y streaming de audio demodulado.



Figura B.1: Diagrama de arquitectura y flujo del frontend de la plataforma web. Fuente: Elaboración propia.

B.1.2. Backend

El backend implementa un servidor Node.js con Express y TypeScript, organizado en seis capas lógicas. El diagrama a continuación ilustra la arquitectura interna del backend, incluyendo las capas de entrada HTTP, autenticación y autorización, rutas y controladores, modelos de acceso a datos, comunicación en tiempo real mediante WebSocket y las tareas de mantenimiento automático.



Figura B.2: Diagrama de arquitectura y flujo del backend de la plataforma web. Fuente: Elaboración propia.

B.1.3. Postprocesamiento

El servicio de postprocesamiento es un microservicio independiente implementado en Python con Flask, responsable del análisis espectral avanzado. El siguiente diagrama muestra el flujo de procesamiento desde la recepción de datos espectrales hasta la generación de resultados de análisis, incluyendo la detección de emisiones, la estimación del piso de ruido, el cruce con la base de datos de licencias y la evaluación de cumplimiento normativo.

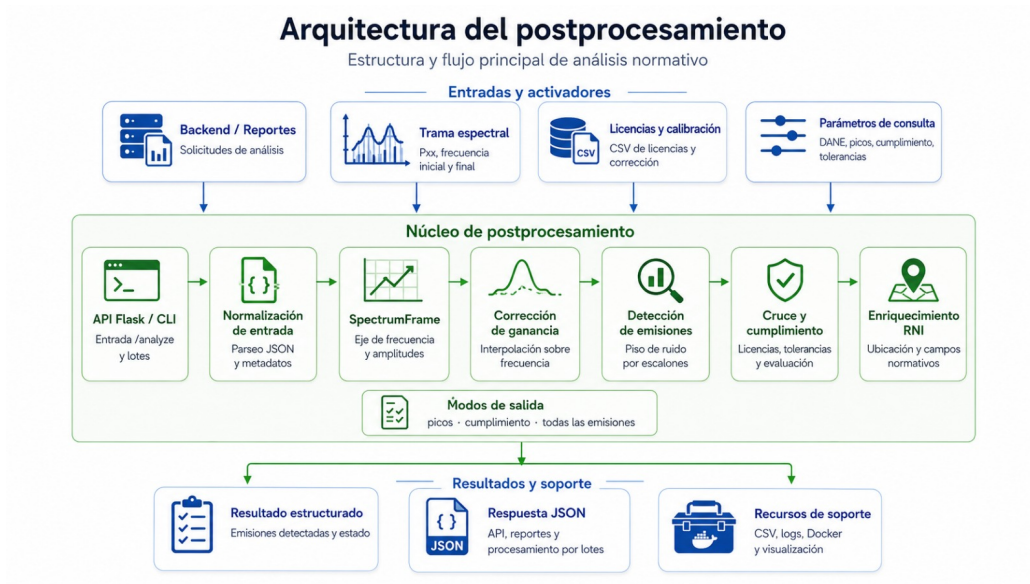


Figura B.3: Diagrama de arquitectura y flujo del servicio de postprocesamiento. Fuente: Elaboración propia.

B.2. Diagramas de despliegue y conectividad

[[Placeholder]]

B.3. Secuencias operativas

[[Placeholder]]

B.4. Esquemas detallados de componentes

[[Placeholder]] “‘latex

Apéndice C

Manuales y documentos de apoyo

C.1. Propósito del capítulo

Este capítulo consolida los manuales, informes técnicos, anexos, videos de apoyo y documentos externos que acompañan el informe final del proyecto. Su objetivo es dejar una referencia organizada de la documentación disponible en la carpeta compartida de Google Drive denominada *Documentación ANE*.

Los documentos referenciados en este capítulo no hacen parte editable del repositorio L^AT_EX. Deben entenderse como material externo de consulta, soporte técnico, guía operativa, evidencia documental o material complementario para instalación, operación, administración, pruebas y mantenimiento del sistema.

Por esta razón, el presente capítulo no reproduce el contenido completo de dichos archivos, sino que identifica su nombre, propósito, relación con el proyecto, enlace de consulta y condición de documento no editable.

C.2. Carpeta documental externa

La documentación de apoyo se encuentra disponible en la siguiente carpeta de Google Drive:

- **Carpeta Documentación ANE:** https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link

Los archivos almacenados en esta carpeta se consideran documentos externos no editables desde el presente informe. Cualquier actualización, reemplazo, corrección o control de ver-

sión debe realizarse directamente en la carpeta documental de Google Drive, manteniendo trazabilidad sobre los cambios.

C.3. Inventario de documentos disponibles

La Tabla C.1 presenta el inventario de documentos, manuales, informes y videos disponibles en la carpeta compartida.

Tabla C.1: Manuales, informes, videos y documentos de apoyo disponibles en Google Drive

Archivo	Propósito dentro del proyecto	Enlace de consulta	Estado
acceptance_test 1.0.pdf	Documento de pruebas de aceptación del sistema. Debe utilizarse como referencia para validar funcionalidades, criterios de aceptación, cumplimiento de requisitos, evidencias de prueba y cierre técnico de entregables. Se relaciona principalmente con los capítulos de protocolos de prueba, validación y trazabilidad.	https://drive.google.com/drive/folders/1Ne_Gym73XcdlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
Antena TDT ensamble.mp4	Video de apoyo para el ensamble físico de la antena TDT. Debe consultarse como material práctico para instalación, montaje, verificación física y apoyo al capítulo de hardware o cadena de recepción RF.	https://drive.google.com/drive/folders/1Ne_Gym73XcdlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
Antena VHF_UHF ensamble.mp4	Video de apoyo para el ensamble físico de la antena VHF/UHF. Debe utilizarse como referencia práctica para montaje, conexión, despliegue y revisión física de antenas asociadas al sistema de monitoreo.	https://drive.google.com/drive/folders/1Ne_Gym73XcdlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable

Archivo	Propósito dentro del proyecto	Enlace de consulta	Estado
Informe Técnico_Análisis De Antenas.docx	Informe técnico de apoyo para el análisis de antenas, criterios de selección, comportamiento esperado, condiciones de uso, bandas de operación y consideraciones asociadas al sistema de monitoreo. Debe consultarse como soporte del capítulo de hardware, adquisición RF y diseño del sistema.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
Manual_del_Usuario.pdf	Manual orientado al usuario final u operador del sistema. Debe servir como guía para el uso cotidiano de la plataforma, consulta de información, interacción con la interfaz, revisión de resultados, visualización de mediciones y operación básica del sistema.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
Anexo.pdf	Documento complementario que agrupa información adicional, detalles de soporte, evidencias, material ampliado o contenido auxiliar que no se desarrolla completamente en el cuerpo principal del informe. Debe consultarse como anexo externo.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
Manual_del_administrador de la plataforma.pdf	Manual de la plataforma para el administrador de la plataforma. Debe utilizarse como referencia para gestión de usuarios, configuración, administración de servicios, revisión de parámetros, permisos, mantenimiento lógico, operación administrativa y supervisión del sistema.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable

Archivo	Propósito dentro del proyecto	Enlace de consulta	Estado
Documentación__soporte_del_sistema_sobre_monitoreo_multiproposito.pdf	Documento de soporte del sistema de monitoreo multipropósito. Debe utilizarse como referencia general para entender la arquitectura, componentes, flujo de información, funcionalidades principales, criterios técnicos, elementos de soporte, operación general y alcance funcional del sistema.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
manual_de_instalacion.pdf	Guía de instalación del sistema. Debe servir como referencia para despliegue inicial, preparación del entorno, instalación de componentes, configuración básica, verificación posterior a la instalación y puesta en marcha. Se relaciona con los capítulos de hardware, software de borde y plataforma cloud.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable
informe_IQ_Balance.pdf	Informe técnico relacionado con balance I/Q, calibración o análisis de señales complejas en sistemas SDR. Debe consultarse como soporte para los capítulos de adquisición SDR, procesamiento digital de señales, calidad de datos, calibración y validación técnica de mediciones.	https://drive.google.com/drive/folders/1Ne_Gym73XcDlqQFBKUjwLtbypIQ1od_t?usp=drive_link	No editable

C.4. Relación de los documentos con los capítulos del informe

Los documentos externos deben utilizarse como soporte de los capítulos técnicos del informe final. La relación sugerida es la siguiente:

- **Capítulo de diseño del sistema:** debe apoyarse en el documento Documentación__soporte_del el archivo Anexo.pdf y los documentos técnicos relacionados.

- **Capítulo de hardware y cadena RF:** debe apoyarse en el Informe Técnico_Análisis De Antenas.docx, en los videos Antena TDT ensamble.mp4 y Antena VHF_UHF ensamble.mp4, así como en el manual_de_instalacion.pdf.
- **Capítulo de adquisición SDR:** debe apoyarse en el informe_IQ_Balance.pdf, en la documentación de soporte del sistema de monitoreo y en los criterios técnicos relacionados con adquisición, calibración y calidad de señal.
- **Capítulo de plataforma web o cloud:** debe apoyarse en el Manual_del_administrador_de_la_plataforma.pdf, el Manual_del_Usuario.pdf y la documentación de soporte del sistema de monitoreo multipropósito.
- **Capítulo de operación del sistema:** debe apoyarse principalmente en el Manual_del_Usuario.pdf.
- **Capítulo de instalación y despliegue:** debe apoyarse en el manual_de_instalacion.pdf y en los videos de ensamble de antenas.
- **Capítulo de mantenimiento y administración:** debe apoyarse en el Manual_del_administrador_de_la_plataforma.pdf, el manual_de_instalacion.pdf y la documentación de soporte del sistema.
- **Capítulo de protocolos de prueba:** debe apoyarse en el archivo acceptance_test 1.0.pdf, el cual debe servir como referencia para criterios de aceptación, validación funcional y evidencia de cumplimiento.
- **Anexos y material complementario:** deben apoyarse en el archivo Anexo.pdf.

C.5. Clasificación funcional de la documentación

Para facilitar la consulta, los documentos externos se pueden clasificar funcionalmente de la siguiente manera:

- **Documentos de operación:**
 - Manual_del_Usuario.pdf
- **Documentos de administración:**
 - Manual_del_administrador_de_la_plataforma.pdf
- **Documentos de instalación y despliegue:**
 - manual_de_instalacion.pdf
 - Antena TDT ensamble.mp4
 - Antena VHF_UHF ensamble.mp4

- **Documentos técnicos de soporte:**
 - Informe Técnico_ Análisis De Antenas.docx
 - informe_IQ_Balance.pdf
 - Documentación__soporte_del_sistema_de_monitoreo_multiproposito.pdf
- **Documentos de pruebas y aceptación:**
 - acceptance_test 1.0.pdf
- **Material complementario:**
 - Anexo.pdf

C.6. Requisitos previos de consulta e instalación

Antes de aplicar los procedimientos descritos en los manuales externos, se recomienda verificar los siguientes aspectos:

- Contar con acceso de lectura a la carpeta de Google Drive *Documentación ANE*.
- Confirmar que los documentos consultados corresponden a la versión vigente o aceptada por el equipo del proyecto.
- Identificar el subsistema al que corresponde cada documento: hardware, SDR, plataforma, administración, instalación, operación, pruebas o soporte.
- Disponer del entorno técnico requerido para aplicar los procedimientos: nodo de borde, plataforma cloud, equipo SDR, red de comunicaciones, credenciales administrativas o interfaz de usuario.
- Validar que los procedimientos de instalación, administración o mantenimiento sean coherentes con el alcance real del prototipo o sistema implementado.
- Evitar copiar en el informe credenciales, direcciones internas, configuraciones sensibles o información operativa que deba mantenerse bajo control de acceso.
- Verificar que los videos de ensamble sean usados como apoyo práctico y no como sustituto de los procedimientos técnicos escritos.

C.7. Instrucciones de operación cotidiana

Para consultar y utilizar la documentación de apoyo durante la operación del sistema, se recomienda el siguiente procedimiento:

1. Abrir la carpeta compartida *Documentación ANE* desde el enlace indicado en este capítulo.
2. Identificar el documento correspondiente al tipo de actividad que se desea realizar.
3. Para operación funcional del sistema, consulta de resultados, navegación de la plataforma y uso por parte del operador, consultar el archivo:
 - `Manual_del_Usuario.pdf`
4. Para tareas de administración, gestión de usuarios, configuración, revisión de servicios, permisos o mantenimiento lógico, consultar el archivo:
 - `Manual_del_administrador_de_la_plataforma.pdf`
5. Para instalación, despliegue inicial, preparación del entorno y puesta en marcha, consultar el archivo:
 - `manual_de_instalacion.pdf`
6. Para montaje físico de antenas, consultar los videos:
 - `Antena TDT ensamble.mp4`
 - `Antena VHF_UHF ensamble.mp4`
7. Para aspectos técnicos relacionados con antenas, criterios de selección, análisis RF o comportamiento esperado, consultar el archivo:
 - `Informe Técnico_ Análisis De Antenas.docx`
8. Para aspectos relacionados con adquisición SDR, señales I/Q, balance I/Q o calibración, consultar el archivo:
 - `informe_IQ_Balance.pdf`
9. Para información general de soporte del sistema de monitoreo multipropósito, consultar el archivo:
 - `Documentación__soporte_del_sistema_de_monitoreo_multiproposito.pdf`
10. Para pruebas de aceptación, criterios de validación y cierre funcional, consultar el archivo:

- `acceptance_test 1.0.pdf`

11. Para información complementaria o material ampliado, consultar el archivo:

- `Anexo.pdf`

12. Registrar cualquier inconsistencia entre el documento externo y el informe final para revisión posterior.

C.8. Solución de problemas frecuentes

La Tabla C.2 resume problemas comunes asociados a la consulta o uso de los documentos externos.

Tabla C.2: Problemas frecuentes en la consulta de documentos externos

Problema	Causa probable	Acción recomendada
No se puede abrir la carpeta de Drive.	El usuario no tiene permisos de lectura o el enlace fue modificado.	Solicitar acceso al responsable de la carpeta documental o verificar que el enlace compartido siga vigente.
No se puede abrir un documento específico.	El archivo puede haber sido movido, eliminado, renombrado o tener permisos restringidos.	Ingresa primero a la carpeta principal y ubicar el documento por nombre. Si no aparece, solicitar confirmación al responsable documental.
No se puede reproducir un video de ensamble.	El archivo de video puede requerir permisos adicionales, conexión estable o reproducción desde el visor de Google Drive.	Abrir el video directamente desde la carpeta de Drive, verificar permisos de lectura y descargarlo solo si el responsable documental lo permite.
El nombre del documento en el informe no coincide exactamente con el archivo en Drive.	Puede existir cambio de nombre, duplicidad, error de escritura o diferencia entre versiones.	Verificar el nombre completo directamente en Drive y actualizar la Tabla C.1.

Problema	Causa probable	Acción recomendada
Existen varias versiones de un mismo documento.	Pueden haberse generado copias de trabajo, versiones preliminares o documentos duplicados.	Usar únicamente la versión validada por el equipo del proyecto y registrar la fecha de consulta.
El contenido del manual externo no coincide con el sistema implementado.	El manual puede corresponder a una versión anterior del sistema o a una configuración distinta.	Registrar la diferencia y validar con el responsable técnico antes de aplicar el procedimiento.
El documento contiene configuraciones sensibles.	Algunos manuales pueden incluir rutas, parámetros, direcciones, usuarios de ejemplo o configuraciones internas.	No copiar información sensible en el informe final. Mantener los documentos externos con permisos controlados.
El enlace apunta a la carpeta general y no al archivo específico.	No se dispone todavía del enlace individual del archivo o se decidió conservar una referencia general a la carpeta documental.	Copiar desde Google Drive el enlace individual del archivo y reemplazarlo en la tabla de inventario documental.

C.9. Recomendaciones para mantenimiento documental

Para conservar la coherencia entre el informe final y los documentos externos, se recomienda:

- Mantener la carpeta *Documentación ANE* como repositorio documental externo de consulta.
- No duplicar el contenido completo de los manuales dentro del archivo $\text{\LaTeX}$.
- Conservar los documentos externos como archivos no editables desde el informe.
- Agregar enlaces individuales a cada documento cuando estén disponibles.
- Actualizar este capítulo si se agregan, eliminan, renombran o reemplazan documentos en la carpeta de Drive.
- Verificar periódicamente que los enlaces de consulta funcionen correctamente.
- Evitar reemplazos silenciosos de documentos. Si se actualiza un archivo, se recomienda conservar control de versión o fecha.

- Relacionar cada documento con el capítulo técnico correspondiente para mejorar la trazabilidad.
- No incluir en el informe credenciales, parámetros internos sensibles o información de acceso restringido.
- Distinguir entre documentos técnicos, manuales de usuario, manuales de administrador, videos de ensamble, pruebas de aceptación y anexos complementarios.

C.10. Control de trazabilidad documental

Para fortalecer la trazabilidad del informe, cada documento externo debe asociarse con al menos uno de los siguientes elementos:

- Requisito del sistema.
- Subsistema técnico.
- Procedimiento de instalación.
- Procedimiento de operación.
- Procedimiento de administración.
- Procedimiento de mantenimiento.
- Procedimiento de prueba o aceptación.
- Evidencia de validación.
- Decisión de diseño.
- Material complementario o anexo.

La trazabilidad permite identificar qué documento respalda una afirmación, configuración, prueba o procedimiento incluido en el informe. Por tanto, cuando un documento externo sea utilizado como respaldo de una afirmación técnica, debe indicarse en el capítulo correspondiente mediante una referencia textual, una nota, una tabla de trazabilidad o una mención explícita al archivo consultado.

C.11. Nota sobre enlaces individuales

En la versión actual de este capítulo se referencia la carpeta principal de Google Drive para todos los documentos. Esto permite acceder al conjunto completo de archivos disponibles en la carpeta *Documentación ANE*. Sin embargo, para mejorar la trazabilidad documental, se recomienda reemplazar progresivamente el enlace general por el enlace individual de cada archivo.

El procedimiento sugerido es:

1. Abrir la carpeta *Documentación ANE*.
2. Hacer clic derecho sobre cada archivo.
3. Seleccionar la opción de compartir o copiar enlace.
4. Verificar que el permiso permita consulta de lectura.
5. Reemplazar en la Tabla [C.1](#) el enlace general de la carpeta por el enlace individual del archivo.
6. Conservar el estado *No editable* para indicar que el documento es una fuente externa de consulta.

C.12. Observación final

Los manuales, informes, videos y documentos de soporte incluidos en este capítulo deben considerarse parte del ecosistema documental del proyecto, pero no sustituyen el contenido técnico principal del informe. Su función es complementar, respaldar y facilitar la consulta de información específica relacionada con instalación, operación, administración, pruebas, análisis técnico y mantenimiento del sistema de monitoreo. ““

Apéndice D

Resultados de pruebas

D.1. Tablas de métricas y mediciones

[[Placeholder]]

D.2. Evidencias visuales de ejecución

[[Placeholder]]

D.3. Comparación entre escenarios de prueba

[[Placeholder]]

D.4. Observaciones y anomalías detectadas

[[Placeholder]]